



Uniwersytet  
Wrocławski



University of Wrocław

# Pokor Fanclub

Olaf Surgut, Rafał Mańczyk, Krzysztof Olejnik

ICPC 2025

4 September

1 Contest

2 Mathematics

3 Data structures

4 Numerical

5 Number theory

6 Combinatorial

7 Graph

8 Geometry

9 Strings

10 Various

Contest (1)

template.cpp 42 lines

```
#include "bits/stdc++.h"
using namespace std;
#define int long long
#define ll long long
#define ld long double
#define endl '\n'
#define st first
#define nd second
#define mp make_pair
#define pb push_back
#define eb emplace_back
#define sz(x) (int)(x).size()
#define all(x) begin(x),end(x)
#define FOR(i,l,r) for(int i=(l);i<=(r);i++)
#define ROF(i,r,l) for(int i=(r);i>=(l);i--)
auto& operator<<(auto &o, pair<auto, auto> p) {
    return o << "(" << p.st << ", " << p.nd << ")";
}
auto operator<<(auto &o, auto x)->decltype(end(x), o) {
    o << "{"; int i=0; for (auto e : x) o << ", "+!i++ << e;
    return o << "}";
}
#ifdef LOCAL
#define debug(x...) cerr << "[" #x "]: ", []{(auto...$) { \
    ((cerr << $ << "; ",...) << endl; }(x)
#else
#define debug(...)
#endif
#define rep(i,a,b) for(int i=(a);i<(b); i++)
using pii = pair<int, int>;
using vi = vector<int>;
const int inf = 1e9+7;
signed main() {
    cin.tie(0)->sync_with_stdio(0);
    return 0;
}
```

pokorrc 31 lines

```
# path to the bits/stdc++.h for fast compilation
$ g++ a.cpp -H 2> 2>(head)
# Slow compilation
c() {
    g++ $1.cpp -o $1 -std=gnu20 -Wall -Wshadow -Wextra \
    -fsanitize=undefined,address -g -gdb3 -DLOCAL -I$HOME/bits
}
# Quick compilation
nc() {
    g++ $1.cpp -o $1 -std=gnu20 -O2 -g -static
}
# Time and memory usage
r() { command time -f "%Us %M KB" ./$1; }
# Hash of a file, ignores all whitespaces and comments
```

```
1 haszuj() {
2     cpp -dD -P -fpreprocessed $1.cpp | tr -d '[:space:]' | \
3     md5sum | cut -c-6
4 }
5 # Tester script given brut and gen in the folder
6 testuj() {
7     for ((i=0;;i++)); do
8         ./gen > $1.test
9         diff -bwq <./>$1 < $1.test) <(/brute < $1.test) || break
10        echo "OK $i"
11    done
12 }
```

7 .vimrc 2 lines

```
syntax on
set nu hls is ts=4 si sw=4
```

9 hash.sh 3 lines

```
11 # Hashes a file, ignoring all whitespace and comments. Use for
12 # verifying that code was correctly typed.
13 cpp -dD -P -fpreprocessed | tr -d '[:space:]' | md5sum |cut -c-6
```

18 Mathematics (2)

22 2.1 Equations

23 
$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by  $x = -b/2a$ .

$$\begin{aligned} ax + by &= e & x &= \frac{ed - bf}{ad - bc} \\ cx + dy &= f & y &= \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation  $Ax = b$ , the solution to a variable  $x_i$  is given by

$$x_i = \frac{\det A'_i}{\det A}$$

where  $A'_i$  is  $A$  with the  $i$ 'th column replaced by  $b$ .

2.2 Recurrences

If  $a_n = c_1a_{n-1} + \dots + c_ka_{n-k}$ , and  $r_1, \dots, r_k$  are distinct roots of  $x^k - c_1x^{k-1} - \dots - c_k$ , there are  $d_1, \dots, d_k$  s.t.

$$a_n = d_1r_1^n + \dots + d_kr_k^n.$$

Non-distinct roots  $r$  become polynomial factors, e.g.

$$a_n = (d_1n + d_2)r^n.$$

2.3 Trigonometry

$$\sin(v + w) = \sin v \cos w + \cos v \sin w$$

$$\cos(v + w) = \cos v \cos w - \sin v \sin w$$

$$\tan(v + w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$

$$\sin v + \sin w = 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$\cos v + \cos w = 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where  $V, W$  are lengths of sides opposite angles  $v, w$ .

$$a \cos x + b \sin x = r \cos(x - \phi)$$

$$a \sin x + b \cos x = r \sin(x + \phi)$$

where  $r = \sqrt{a^2 + b^2}, \phi = \text{atan2}(b, a)$ .

2.4 Geometry

Side lengths:  $a, b, c$

Semiperimeter:  $p = \frac{a + b + c}{2}$

Area:  $A = \sqrt{p(p - a)(p - b)(p - c)}$

Circumradius:  $R = \frac{abc}{4A}$

Inradius:  $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles):

$$m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc \left[ 1 - \left( \frac{a}{b + c} \right)^2 \right]}$$

Law of sines:  $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines:  $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Law of tangents:  $\frac{a + b}{a - b} = \frac{\tan \frac{\alpha + \beta}{2}}{\tan \frac{\alpha - \beta}{2}}$

With side lengths  $a, b, c, d$ , diagonals  $e, f$ , diagonals angle  $\theta$ , area  $A$  and magic flux  $F = b^2 + d^2 - a^2 - c^2$ :

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is  $180^\circ$ ,  $ef = ac + bd$ , and  $A = \sqrt{(p - a)(p - b)(p - c)(p - d)}$ .

2.5 Derivatives/Integrals

$$\int \sqrt{a^2 + x^2} dx = \frac{x}{2} \sqrt{a^2 + x^2} + \frac{a^2}{2} \ln(x + \sqrt{a^2 + x^2})$$
$$\int \sqrt{a^2 - x^2} dx = \frac{x}{2} \sqrt{a^2 - x^2} + \frac{a^2}{2} \arcsin \frac{x}{|a|}$$
$$\int \frac{dx}{\sqrt{a^2 - x^2}} = \arcsin \frac{x}{|a|} = -\arccos \frac{x}{|a|}$$
$$\int \frac{dx}{\sqrt{a^2 + x^2}} = \ln(x + \sqrt{a^2 + x^2})$$

Sub  $s = \tan(x/2)$  to get:  $dx = \frac{2 \, ds}{1 + s^2}$

$$\sin x = \frac{2s}{1 + s^2}, \, \cos x = \frac{1 - s^2}{1 + s^2}$$
$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

(Integration by parts)  $\int \tan ax = -\frac{\ln |\cos ax|}{a}$

$$\int x \sin ax = \frac{\sin ax - ax \cos ax}{a^2}$$
$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2} \text{erf}(x), \, \int x e^{ax} dx = \frac{e^{ax}}{a^2} (ax - 1)$$
$$\frac{d}{dx} \tan x = 1 + \tan^2 x, \, \frac{d}{dx} \arctan x = \frac{1}{1 + x^2}$$

Curve length:  $\int_a^b \sqrt{1 + (f'(x))^2} dx$

When  $X(t), Y(t) : \int_a^b \sqrt{(X'(t))^2 + (Y'(t))^2} dt$

Solid of revolution vol:  $\pi \int_a^b (f(x))^2 dx$

Surface area:  $2\pi \int_a^b |f(x)| \sqrt{1 + (f'(x))^2} dx$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Sums

$$c^a + c^{a+1} + \dots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$
$$1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(2n+1)(n+1)}{6}$$
$$1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4}$$
$$1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.7 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, (-\infty < x < \infty)$$
$$\ln(1 + x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots, (-1 < x \leq 1)$$
$$\sqrt{1 + x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \dots, (-1 \leq x \leq 1)$$
$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, (-\infty < x < \infty)$$
$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots, (-\infty < x < \infty)$$

2.8 Probability theory

Let  $X$  be a discrete random variable with probability  $p_X(x)$  of assuming the value  $x$ . It will then have an expected value (mean)  $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$  and variance  $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$  where  $\sigma$  is the standard deviation. If  $X$  is instead continuous it will have a probability density function  $f_X(x)$  and the sums above will instead be integrals with  $p_X(x)$  replaced by  $f_X(x)$ .

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent  $X$  and  $Y$ ,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

2.8.1 Discrete distributions

Binomial distribution

The number of successes in  $n$  independent yes/no experiments, each which yields success with probability  $p$  is  $\text{Bin}(n, p)$ ,  $n = 1, 2, \dots$ ,  $0 \leq p \leq 1$ .

$$p(k) = \binom{n}{k} p^k (1 - p)^{n - k}$$

$$\mu = np, \sigma^2 = np(1 - p)$$

$\text{Bin}(n, p)$  is approximately  $\text{Po}(np)$  for small  $p$ .

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability  $p$  is  $\text{Fs}(p)$ ,  $0 \leq p \leq 1$ .

$$p(k) = p(1 - p)^{k - 1}, k = 1, 2, \dots$$

$$\mu = \frac{1}{p}, \sigma^2 = \frac{1 - p}{p^2}$$

2.9 Other theorems

Dirichelt's approximation theorem: For any real  $r$  and  $N$ , with  $1 \leq N$ , there exist integers  $p$  and  $q$  such that  $1 \leq q \leq N$  and

$$|q\alpha - p| \leq \frac{1}{[N] + 1} < \frac{1}{N}$$

$$0 < |\alpha - \frac{p}{q}| < \frac{1}{q^2}$$

Stirling approximation :

$$n! \approx \sqrt{2\pi n} (\frac{n}{e})^n e^{\frac{1}{12n}}$$

Stirling Numbers(permutation  $|P| = n$  with  $k$  cycles):

$$S(n, k) = \text{coefficient of } x^k \text{ in } \Pi_{i=0}^{n-1} (x + i)$$

Stirling Numbers(Partition  $n$  elements into  $k$  non-empty set):

$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

Pick's Theorem :  $A = i + b/2 - 1$

$A$ : Area,  $i$ : grid number in the inner,  $b$ : grid number on the side

$$C_n = \binom{2n}{n} / (n + 1)$$
$$C_n^{n+m} - C_{n+1}^{n+m} = (m + n)! \frac{n-m+1}{n+1} \quad \text{for } n \geq m$$
$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$
$$C_0 = 1 \quad \text{and} \quad C_{n+1} = 2 \binom{2n+1}{n+2} C_n$$
$$C_0 = 1 \quad \text{and} \quad C_{n+1} = \sum_{i=0}^n C_i C_{n-i} \quad \text{for } n \geq 0$$

Euler Characteristic:

$$\text{planar graph: } V - E + F - C = 1$$

$$\text{convex polyhedron: } V - E + F = 2$$

$V, E, F, C$ : number of vertices, edges, faces(regions), and components

Kirchhoff's theorem:

$A_{ii} = \text{deg}(i), A_{ij} = (i, j) \in E \text{ ?} - 1 : 0$ , Deleting any one row, one column, and cal the  $\det(A)$

Polya' theorem ( $c$  is number of color,  $m$  is the number of cycle size):  
 $(\sum_{i=1}^m c^{gcd(i, m)})/m$

Wilson's theorem :

$(p-1)! \equiv -1(mod\ p)$

Euler’s totient function:  
 $A^{B^C} mod\ p = pow(A,pow(B,C,p-1))mod\ p$

### Data structures (3)

PBDS.h  
Description: Policy Based Data Structures 460200, 17 lines

```
// Order Statistics Tree: Caution: Not a multiset!
#include <bits/extc++.h>
using namespace __gnu_pbds;
template <class T> using Tree = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
Tree<int> t, t2;
auto it = t.insert(10).first; // it == t.upper_bound(9);
t.order_of_key(10); // # of entries strictly smaller than key
t.join(t2); // fast only if max(T) < min(T2) or min(T) > max(T2)
// Hash Table: faster but can lead to MLE (1.5x worse performance),
    initial capacity must = 2^k
struct chash { // large odd number for C
    const uint64_t C = 1l(4e18 * acos(0)) | 71;
    ll operator()(ll x) const { return __builtin_bswap64(x * C); }
};
gp_hash_table<ll, int, chash> h({}, {}, {}, {}, {1 << 16}); //
    cc_hash_table also exists if needed
```

HashMap.h  
Description: Hash map with mostly the same API as unordered\_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided). d77092, 7 lines

```
#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
    const uint64_t C = 1l(4e18 * acos(0)) | 71;
    ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll,int,chash> h({}, {}, {}, {}, {1<<16});
```

LazySegmentTree.h  
Description: Segment tree with ability to add or set values of large intervals, and compute max of intervals. Can be changed to other things. Use with a bump allocator for better performance, and SmallPtr or implicit indices to save memory.  
Usage: Node\* tr = new Node(v, 0, sz(v));  
Time: O(log N). 34ecf5, 50 lines

```
"../various/BumpAllocator.h"
const int inf = 1e9;
struct Node {
    Node *l = 0, *r = 0;
    int lo, hi, mset = inf, madd = 0, val = -inf;
    Node(int lo,int hi):lo(lo),hi(hi){} // Large interval of -inf
    Node(vi& v, int lo, int hi) : lo(lo), hi(hi) {
        if (lo + 1 < hi) {
            int mid = lo + (hi - lo)/2;
            l = new Node(v, lo, mid); r = new Node(v, mid, hi);
            val = max(l->val, r->val);
        }
        else val = v[lo];
    }
    int query(int L, int R) {
        if (R <= lo || hi <= L) return -inf;
        if (L <= lo && hi <= R) return val;
        push();
        return max(l->query(L, R), r->query(L, R));
    }
    void set(int L, int R, int x) {
        if (R <= lo || hi <= L) return;
        if (L <= lo && hi <= R) mset = val = x, madd = 0;
        else {
            push(), l->set(L, R, x), r->set(L, R, x);
            val = max(l->val, r->val);
        }
    }
    void add(int L, int R, int x) {
        if (R <= lo || hi <= L) return;
        if (L <= lo && hi <= R) {
            if (mset != inf) mset += x;
            else madd += x;
            val += x;
        }
    }
}
```

```
    else {
        push(), l->add(L, R, x), r->add(L, R, x);
        val = max(l->val, r->val);
    }
}
void push() {
    if (!l) {
        int mid = lo + (hi - lo)/2;
        l = new Node(lo, mid); r = new Node(mid, hi);
    }
    if (mset != inf)
        l->set(lo,hi,mset), r->set(lo,hi,mset), mset = inf;
    else if (madd)
        l->add(lo,hi,madd), r->add(lo,hi,madd), madd = 0;
}
};
```

HilbertOrder.h  
Description: Useful speed up for MoQueries.  
Time: O(log N) 8cec340, 14 lines

```
ll hilbertOrder(int x, int y, int pow = 20, int rotate = 0) {
    if(pow == 0) return 0;
    int hpow = 1 << (pow - 1);
    int seg = x < hpow ? (y < hpow ? 0:3) : (y < hpow ? 1:2);
    seg = (seg + rotate) & 3;
    const int rotateDelta[4] = {3, 0, 0, 1};
    int nx = x & (x ^ hpow), ny = y & (y ^ hpow);
    int nrot = (rotate + rotateDelta[seg]) & 3;
    ll subSquareSize = 1ll << (pow * 2 - 2);
    ll ans = seg * subSquareSize;
    ll add = hilbertOrder(nx, ny, pow - 1, nrot);
    ans += seg == 1 || seg == 2 ? add : (subSquareSize - add-1);
    return ans;
}
```

UnionFindRollback.h  
Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().  
Usage: int t = uf.time(); ...; uf.rollback(t);  
Time: O(log(N)) de4ad0, 21 lines

```
struct RollbackUF {
    vi e; vector<pii> st;
    RollbackUF(int n) : e(n, -1) {}
    int size(int x) { return e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : find(e[x]); }
    int time() { return sz(st); }
    void rollback(int t) { i --> t;
        for (int i = time(); i --> t; )
            e[st[i].first] = st[i].second;
        st.resize(t);
    }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        st.push_back({a, e[a]});
        st.push_back({b, e[b]});
        e[a] += e[b]; e[b] = a;
        return true;
    }
};
```

LineContainer.h  
Description: Container where you can add lines of the form kx+m, and query maximum values at points x. Useful for dynamic programming (“convex hull trick”).  
Time: O(log N) 8ec1c7, 30 lines

```
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};
struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
}
```

```
void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y) isect(x, y = erase(y)))
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
}
ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
};
```

Treap.h  
Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.  
Time: O(log N), increase works in O(log N log U) amortized 949abd, 113 lines

```
struct Treap {
    struct Node {
        int ch[2] = {0, 0}, size = 0;
        int val = 0, mini = 1e9, sum = 0; // Subtree aggregates
        bool flip = 0; int add = 0; // Lazy tags
        Node() {}
        Node(int v) : size(1), val(v), mini(v), sum(v) {}
    };
    vector<Node> t;
    Treap() : t(1) {}
    void pull(int v) {
        auto [l, r] = t[v].ch;
        t[v].size = t[l].size + 1 + t[r].size;
        t[v].mini = min({t[l].mini, t[v].val, t[r].mini});
        t[v].sum = t[l].sum + t[v].val + t[r].sum;
    }
    int apply(int v, bool flip, int add) {
        if (!v) return 0;
        // t.pb(t[v]), v = SZ(t) - 1; // <- persistency
        if (flip) t[v].flip ^= 1, swap(t[v].ch[0], t[v].ch[1]);
        t[v].val += add; t[v].mini += add;
        t[v].sum += add * t[v].size;
        t[v].add += add;
        return v;
    }
    void push(int v) {
        FOR(i, 0, 2)
            t[v].ch[i] = apply(t[v].ch[i], t[v].flip, t[v].add);
        t[v].add = t[v].flip = 0;
    }
    int rank(int v, int u) {
        static mt19937 gen(2137);
        return int(gen() % (t[v].size + t[u].size)) < t[v].size;
    }
    pii split(int v, int k) {
        if (!v) return {0, 0};
        push(v);
        auto [l, r] = t[v].ch;
        if (k <= t[l].size) {
            // if (k <= t[v].val) { // <- by values
            auto [p, q] = split(l, k);
            t[v].ch[0] = q, pull(v);
            return {p, v};
        }
        else {
            auto [p, q] = split(r, k - t[l].size - 1);
            // auto [p, q] = split(r, k); // <- by values
            t[v].ch[1] = p, pull(v);
            return {v, q};
        }
    }
    int merge(int v, int u) {
        if (!v || !u) return v ^ u;
        push(v), push(u);
        if (rank(v, u)) {
            t[v].ch[1] = merge(t[v].ch[1], u);
            return pull(v), v;
        }
        else {
            t[u].ch[0] = merge(v, t[u].ch[0]);
            return pull(u), u;
        }
    }
    void insert(int &v, int pos, int val) {
        // if (v) t.pb(t[v]), v = SZ(t) - 1; // <- persistency
        auto [p, q] = split(v, pos);
        t.pb(Node(val)); int u = SZ(t) - 1;
        // t.pb(Node(pos)); int u = SZ(t) - 1; // <- by values
        v = merge(merge(p, u), q);
    }
    void erase(int &v, int l, int r) {
```

```
// if(v) t.pb(t[v]), v = SZ(t) - 1; // <- persistency
auto [p, q] = split(v, l);
auto [u, s] = split(q, r - l + 1);
// auto [u, s] = split(q, r + 1); // <- by values
v = merge(p, s);
}
void modify(int &v, int l, int r, bool flip, int add) {
// if(v) t.pb(t[v]), v = SZ(t) - 1; // <- persistency
auto [p, q] = split(v, l);
auto [u, s] = split(q, r - l + 1);
// auto [u, s] = split(q, r + 1); // <- by values
u = apply(u, flip, add);
v = merge(merge(p, u), s);
}
pii get(int &v, int l, int r) {
// if(v) t.pb(t[v]), v = SZ(t) - 1; // <- persistency
auto [p, q] = split(v, l);
auto [u, s] = split(q, r - l + 1);
// auto [u, s] = split(q, r + 1); // <- by values
int mini = t[u].mini, sum = t[u].sum;
v = merge(merge(p, u), s);
return {mini, sum};
}
// only when by values
int join(int v, int u) {
if(!v || !u) return v ^ u;
if(!rank(v, u)) swap(v, u);
auto [p, q] = split(u, t[v].val);
push(v);
t[v].ch[0] = join(t[v].ch[0], p);
t[v].ch[1] = join(t[v].ch[1], q);
return pull(v), v;
}
// only when by values, persistency destroys complexity
void increase(int &v, int l, int r, int increase) {
// if(v) t.pb(t[v]), v = SZ(t) - 1; // <- persistency
auto [p, q] = split(v, l);
auto [u, s] = split(q, r + 1);
u = apply(u, 0, increase);
v = join(merge(p, s), u);
}
};
```

## LiChao.h

**Description:** LiChaoTree (insert line, min on interval)

```
struct LiChao_min{
    struct line{
        LL m, c;
        line(LL _m=0, LL _c=0) { m = _m; c = _c; }
        LL eval(LL x) { return m * x + c; }
    };
    struct node{
        node *l, *r; line f;
        node(line v) { f = v; l = r = NULL; }
    };
    typedef node* pnode;
    pnode root; int sz;
#define mid ((l+r)>>1)
    void insert(line &v, int l, int r, pnode &nd){
        if(!nd) { nd = new node(v); return; }
        LL trl = nd->f.eval(l), trr = nd->f.eval(r);
        LL vl = v.eval(l), vr = v.eval(r);
        if(trl <= vl && trr <= vr) return;
        if(trl > vl && trr > vr) { nd->f = v; return; }
        if(trl > vl) swap(nd->f, v);
        if(nd->f.eval(mid) < v.eval(mid)) insert(v, mid + 1, r, nd->r);
        else swap(nd->f, v), insert(v, l, mid, nd->l);
    }
    LL query(int x, int l, int r, pnode &nd){
        if(!nd) return LLONG_MAX;
        if(l == r) return nd->f.eval(x);
        if(mid >= x) return min(nd->f.eval(x), query(x, l, mid, nd->l));
        return min(nd->f.eval(x), query(x, mid + 1, r, nd->r));
    }
}
/* -sz <= query_x <= sz */
void init(int _sz) { sz = _sz + 1; root = NULL; }
void add_line(LL m, LL c){ line v(m, c); insert(v, -sz, sz, root); }
LL query(LL x) { return query(x, -sz, sz, root); }
};
```

## FenwickTree.h

**Description:** Computes partial sums  $a[0] + a[1] + \dots + a[\text{pos} - 1]$ , and updates single elements  $a[i]$ , taking the difference between the old and new value.

**Time:** Both operations are  $\mathcal{O}(\log N)$ .

```
struct FT {
```

```
vector<ll> s;
FT(int n) : s(n) {}
void update(int pos, ll dif) { // a[pos] += dif
    for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
}
ll query(int pos) { // sum of values in [0, pos)
    ll res = 0;
    for (; pos > 0; pos &= pos - 1) res += s[pos-1];
    return res;
}
int lower_bound(ll sum) { // min pos st sum of [0, pos] >= sum
    // Returns n if no sum is >= sum, or -1 if empty sum is.
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >= 1) {
        if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
            pos += pw, sum -= s[pos-1];
    }
    return pos;
}
};
```

## FenwickTree2d.h

**Description:** Computes sums  $a[i,j]$  for all  $i < I, j < J$ , and increases single elements  $a[i,j]$ . Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).

**Time:**  $\mathcal{O}(\log^2 N)$ . (Use persistent segment trees for  $\mathcal{O}(\log N)$ .)

```
*FenwickTree.h*
157f07, 22 lines

struct FT2 {
    vector<vi> ys; vector<FT> ft;
    FT2(int limx) : ys(limx) {}
    void fakeUpdate(int x, int y) {
        for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
    }
    void init() {
        for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
    }
    int ind(int x, int y) {
        return (int)(lower_bound(all(ys[x]), y) - ys[x].begin()); }
    void update(int x, int y, ll dif) {
        for (; x < sz(ys); x |= x + 1)
            ft[x].update(ind(x, y), dif);
    }
    ll query(int x, int y) {
        ll sum = 0;
        for (; x; x &= x - 1)
            sum += ft[x-1].query(ind(x-1, y));
        return sum;
    }
};
```

## WaveletTree.h

**Description:** Wavelet tree. Supports fast kth order statistics on ranges (no updates).

**Time:**  $\mathcal{O}(\log N)$

```
587095, 35 lines

struct WaveletTree {
    vector<vi> seq, left;
    int len;
    WaveletTree() {}
    // time and space: O((n+maxVal) log maxVal)
    // Values are expected to be in [0;maxVal).
    WaveletTree(const vi& elems, int maxVal) {
        for (len = 1; len < maxVal; len *= 2);
        seq.resize(len*2); left.resize(len*2);
        seq[1] = elems; build(1, 0, len);
    }
    void build(int i, int b, int e) {
        if (i >= len) return;
        int m = (b+e) / 2;
        left[i].pb(0);
        for(auto &x : seq[i]) {
            left[i].pb(left[i].back() + (x < m));
            seq[i*2 + (x >= m)].pb(x);
        }
        build(i*2, b, m); build(i*2+1, m, e);
    } // Find k-th (0 indexed) smallest element in [begin;end)
    int kth(int begin, int end, int k, int i=1) {
        if (i >= len) return seq[i][0];
        int x = left[i][begin], y = left[i][end];
        if (k < y-x) return kth(x, y, k, i*2);
        return kth(begin-x, end-y, k-y+x, i*2+1);
    } // Count number of elements >= vb and < ve
    int count(int begin, int end, int vb, int ve, int i = 1, int b = 0, int e = -1) {
        if (e < 0) e = len;
        if (b >= ve || vb >= e) return 0;

```

```
        if (b >= vb && e <= ve) return end-begin;
        int m = (b+e) / 2; int x = left[i][begin], y = left[i][end];
        return count(x, y, vb, ve, i*2, b, m) + count(begin-x, end-y, vb, ve, i*2+1,m,e);
    }
};
```

## RMQ.h

**Description:** Range Minimum Queries on an array. Returns  $\min(V[a], V[a + 1], \dots V[b - 1])$  in constant time.

**Usage:** RMQ rmq(values);

rmq.query(inclusive, exclusive);

**Time:**  $\mathcal{O}(|V| \log |V| + Q)$

```
510c32, 16 lines

template<class T>
struct RMQ {
    vector<vector<T>> jmp;
    RMQ(const vector<T>& V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            rep(j,0,sz(jmp[k]))
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
    T query(int a, int b) {
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};
```

## SlopeTrick.h

**Description:** Linear functions container. Not tested!

**Time:**  $\mathcal{O}(\log N)$  per operation

```
c81b01, 63 lines

struct SlopeTrick {
    const ll INF = 3e18;
    ll min_f, add_l, add_r;
    priority_queue<ll> L;
    priority_queue<ll, vector<ll>, greater<>> R;
    void push_R(ll a) { R.push(a - add_r); }
    ll top_R() const { return SZ(R) ? R.top() + add_r : INF; }
    ll pop_R() {
        ll val = top_R(); if(SZ(R)) R.pop();
        return val;
    }
    void push_L(ll a) { L.push(a - add_l); }
    ll top_L() const { return SZ(L) ? L.top() + add_l : -INF; }
    ll pop_L() {
        ll val = top_L(); if(SZ(L)) L.pop();
        return val;
    }
    int size() { return SZ(L) + SZ(R); }
    // use only functions below!
    SlopeTrick() : min_f(0), add_l(0), add_r(0) {}
    struct Query { ll lx, rx, min_f; };
    // return min f(x)
    Query query() const { return Query(top_L(),top_R(),min_f); }
    // f(x) += a
    void add_all(ll a) { min_f += a; }
    // add \_ f(x) += max(a - x, 0)
    void add_a_minus_x(ll a) {
        min_f += max(0ll, a - top_R()); push_R(a); push_L(pop_R());
    }
    // add / f(x) += max(x - a, 0)
    void add_x_minus_a(ll a) {
        min_f += max(0ll, top_L() - a); push_L(a); push_R(pop_L());
    }
    // add \_ f(x) += abs(x - a)
    void add_abs(ll a) { add_a_minus_x(a); add_x_minus_a(a); }
    // \_> \_ f {new} (x) = min f(y) (y <= x)
    void clear_right() { while(SZ(R)) R.pop(); }
    // \_> / f {new} (x) = min f(y) (y >= x)
    void clear_left() { while(SZ(L)) L.pop(); }
    // \_> \_ f {new} (x) = min f(y) (x-b <= y <= x-a)
    void shift(ll a, ll b) {
        assert(a <= b); add_l += a; add_r += b;
    }
    // \. -> . \ f {new} (x) = f(x - a)
    void shift(ll a) { shift(a, a); }
    // L, R is destroyed
    ll get(ll x) {
        ll ret = min_f;
        while(SZ(L)) ret += max(0ll, pop_L() - x);
        while(SZ(R)) ret += max(0ll, x - pop_R());
        return ret;
    }
    void merge(SlopeTrick &o) {

```

```
    if(SZ(o) > size()) {
        swap(o.L, L); swap(o.R, R);
        swap(o.add_l, add_l); swap(o.add_r, add_r);
        swap(o.min_f, min_f);
    }
    while(SZ(o.R)) add_x_minus_a(o.pop_R());
    while(SZ(o.L)) add_a_minus_x(o.pop_L());
    min_f += o.min_f;
}
};
```

## MoQueries.h

**Description:** Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change step to add/remove the edge  $(a, c)$  and remove the initial add call (but keep in).

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <b>Time:</b> $\mathcal{O}(N\sqrt{Q})$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | a12ef4, 49 lines |
| <pre>void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1) void del(int ind, int end) { ... } // remove a[ind] int calc() { ... } // compute current answer vi mo(vector&lt;pii&gt; Q) {     int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)     vi s(sz(Q)), res = s; #define K(x) pii(x.first/blk, x.second ^ ~(x.first/blk &amp; 1))     iota(all(s), 0);     sort(all(s), [&amp;](int s, int t){ return K(Q[s]) &lt; K(Q[t]); });     for (int qi : s) {         pii q = Q[qi];         while (L &gt; q.first) add(--L, 0);         while (R &lt; q.second) add(R++, 1);         while (L &lt; q.first) del(L++, 0);         while (R &gt; q.second) del(--R, 1);         res[qi] = calc();     }     return res; } vi moTree(vector&lt;array&lt;int, 2&gt;&gt; Q, vector&lt;vi&gt;&amp; ed, int root=0){     int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)     vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);     add(0, 0), in[0] = 1;     auto dfs = [&amp;](int x, int p, int dep, auto&amp; f) -&gt; void {         par[x] = p;         L[x] = N;         if (dep) I[x] = N++;         for (int y : ed[x]) if (y != p) f(y, x, !dep, f);         if (!dep) I[x] = N++;         R[x] = N;     };     dfs(root, -1, 0, dfs); #define K(x) pii(I[x[0]] / blk, I[x[1]] ^ ~(I[x[0]] / blk &amp; 1))     iota(all(s), 0);     sort(all(s), [&amp;](int s, int t){ return K(Q[s]) &lt; K(Q[t]); });     for (int qi : s) rep(end,0,2) {         int &amp;a = pos[end], b = Q[qi][end], i = 0; #define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \                 else { add(c, end); in[c] = 1; } a = c; }         while (! (L[b] &lt;= L[a] &amp;&amp; R[a] &lt;= R[b]))             I[i++] = b, b = par[b];         while (a != b) step(par[a]);         while (i--&gt;) step(I[i]);         if (end) res[qi] = calc();     }     return res; } };</pre> |                  |

## Numerical (4)

### 4.1 Polynomials and recurrences

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <b>Polynomial.h</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                       | c9b7b0, 17 lines |
| <pre>struct Poly {     vector&lt;double&gt; a;     double operator()(double x) const {         double val = 0;         for (int i = sz(a); i--;) (val *= x) += a[i];         return val;     }     void diff() {         rep(i,1,sz(a)) a[i-1] = i*a[i];         a.pop_back();     }     void divroot(double x0) {         double b = a.back(), c; a.back() = 0;         for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;         a.pop_back();     } };</pre> |                  |

};

## PolyRoots.h

**Description:** Finds the real roots to a polynomial.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <b>Usage:</b> polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                  |
| <b>Time:</b> $\mathcal{O}(n^2\log(1/\epsilon))$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                  |
| <b>"Polynomial.h"</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | b00bfe, 23 lines |
| <pre>vector&lt;double&gt; polyRoots(Poly p, double xmin, double xmax) {     if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }     vector&lt;double&gt; ret;     Poly der = p;     der.diff();     auto dr = polyRoots(der, xmin, xmax);     dr.push_back(xmin-1);     dr.push_back(xmax+1);     sort(all(dr));     rep(i,0,sz(dr)-1) {         double l = dr[i], h = dr[i+1];         bool sign = p(l) &gt; 0;         if (sign ^ (p(h) &gt; 0)) {             rep(it,0,60) { // while (h - l &gt; 1e-8)                 double m = (l + h) / 2, f = p(m);                 if ((f &lt;= 0) ^ sign) l = m;                 else h = m;             }             ret.push_back((l + h) / 2);         }     }     return ret; } };</pre> |                  |

## PolyInterpolate.h

**Description:** Given  $n$  points  $(x[i], y[i])$ , computes an  $n-1$ -degree polynomial  $p$  that passes through them:  $p(x) = a[0] * x^0 + \dots + a[n-1] * x^{n-1}$ . For numerical precision, pick  $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \dots n-1$ .

|                                                                                                                                                                                                                                                                                                                                                                             |                  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <b>Time:</b> $\mathcal{O}(n^2)$                                                                                                                                                                                                                                                                                                                                             | 08bf48, 13 lines |
| <pre>typedef vector&lt;double&gt; vd; vd interpolate(vd x, vd y, int n) {     vd res(n), temp(n);     rep(k,0,n-1) rep(i,k+1,n)         y[i] = (y[i] - y[k]) / (x[i] - x[k]);     double last = 0; temp[0] = 1;     rep(k,0,n) rep(i,0,n) {         res[i] += y[k] * temp[i];         swap(last, temp[i]);         temp[i] -= last * x[k];     }     return res; } };</pre> |                  |

## BerlekampMassey.h

**Description:** Recovers any  $n$ -order linear recurrence relation from the first  $2n$  terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size  $\leq n$ .

**Usage:** berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <b>Time:</b> $\mathcal{O}(N^2)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                  |
| <b>"../number-theory/ModPow.h"</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 96548b, 20 lines |
| <pre>vector&lt;ll&gt; berlekampMassey(vector&lt;ll&gt; s) {     int n = sz(s), L = 0, m = 0;     vector&lt;ll&gt; C(n), B(n), T;     C[0] = B[0] = 1;     ll b = 1;     rep(i,0,n) { ++m;         ll d = s[i] % mod;         rep(j,L+1) d = (d + C[j] * s[i - j]) % mod;         if (!d) continue;         T = C; ll coef = d * modpow(b, mod-2) % mod;         rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;         if (2 * L &gt; i) continue;         L = i + 1 - L; B = T; b = d; m = 0;     }     C.resize(L + 1); C.erase(C.begin());     for (ll&amp; x : C) x = (mod - x) % mod;     return C; } };</pre> |                  |

## LinearRecurrence.h

**Description:** Generates the  $k$ 'th term of an  $n$ -order linear recurrence  $S[i] = \sum_j S[i-j-1]tr[j]$ , given  $S[0 \dots \geq n-1]$  and  $tr[0 \dots n-1]$ . Faster than matrix multiplication. Useful together with Berlekamp–Massey.

**Usage:** linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <b>Time:</b> $\mathcal{O}(n^2\log k)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | f4e444, 26 lines |
| <pre>typedef vector&lt;ll&gt; Poly; ll linearRec(Poly S, Poly tr, ll k) {     int n = sz(tr);     auto combine = [&amp;](Poly a, Poly b) {         Poly res(n + 2 + 1);         rep(i,0,n+1) rep(j,0,n+1)             res[i + j] = (res[i + j] + a[i] * b[j]) % mod;         for (int i = 2 * n; i &gt; n; --i) rep(j,0,n)             res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;         res.resize(n + 1);         return res;     };     Poly pol(n + 1), e(pol);     pol[0] = e[1] = 1;     for (int k; k; k /= 2) {         if (k % 2) pol = combine(pol, e);         e = combine(e, e);     }     ll res = 0;     rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;     return res; } };</pre> |                  |

### 4.2 Optimization

## GoldenSectionSearch.h

**Description:** Finds the argument minimizing the function  $f$  in the interval  $[a, b]$  assuming  $f$  is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is  $\epsilon$ ps. Works equally well for maximization with a small change in the code. See Ternary-Search.h in the Various chapter for a discrete version.

**Usage:** double func(double x) { return 4+x+.3\*x\*x; }

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| double xmin = gss(-1000,1000,func);                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                  |
| <b>Time:</b> $\mathcal{O}(\log((b-a)/\epsilon))$                                                                                                                                                                                                                                                                                                                                                                                                                                               | 31d45b, 14 lines |
| <pre>double gss(double a, double b, double (*f)(double)) {     double r = (sqrt(5)-1)/2, eps = 1e-7;     double x1 = b - r*(b-a), x2 = a + r*(b-a);     double f1 = f(x1), f2 = f(x2);     while (b-a &gt; eps)         if (f1 &lt; f2) { //change to &gt; to find maximum             b = x2; x2 = x1; f2 = f1;             x1 = b - r*(b-a); f1 = f(x1);         } else {             a = x1; x1 = x2; f1 = f2;             x2 = a + r*(b-a); f2 = f(x2);         }     return a; } };</pre> |                  |

## Integrate.h

**Description:** Simple integration of a function over an interval using Simpson's rule. The error should be proportional to  $h^4$ , although in practice you will want to verify that the result is stable to desired precision when epsilon changes.

|                                                                                                                                                                                                                                         |                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
|                                                                                                                                                                                                                                         | 4756fc, 7 lines |
| <pre>template&lt;class F&gt; double quad(double a, double b, F f, const int n = 1000) {     double h = (b - a) / 2 / n, v = f(a) + f(b);     rep(i,1,n*2)         v += f(a + i*h) * (i&amp;1 ? 4 : 2);     return v * h / 3; } };</pre> |                 |

## IntegrateAdaptive.h

**Description:** Fast integration using an adaptive Simpson's rule.

**Usage:** double sphereVolume = quad(-1, 1, [](double x) {

|                                                                                                                                                                                                                                                                                      |                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| return quad(-1, 1, [&](double y) {<br>return quad(-1, 1, [&](double z) {<br>return x*x + y*y + z*z < 1; });});});                                                                                                                                                                    | 92dd79, 15 lines |
| <pre>typedef double d; #define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6 template &lt;class F&gt; d rec(F&amp; f, d a, d b, d eps, d S) {     d c = (a + b) / 2;     d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;     if (abs(T - S) &lt;= 15 * eps    b - a &lt; 1e-10)</pre> |                  |

```
        return T + (T - S) / 15;
    return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S2);
}
template<class F>
d quad(d a, d b, F f, d eps = 1e-8) {
    return rec(f, a, b, eps, S(a, b));
}
```

### Simplex.h

**Description:** Solves a general linear maximization problem: maximize  $c^T x$  subject to  $Ax \leq b, x \geq 0$ . Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of  $c^T x$  otherwise. The input vector is set to an optimal  $x$  (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that  $x = 0$  is viable.  
**Usage:** vvd A = {{1,-1}, {-1,1}, {-1,-2}};  
vd b = {1,1,-4}, c = {-1,-1}, x;  
T val = LPSolver(A, b, c).solve(x);  
**Time:**  $\mathcal{O}(NM \times \#\text{pivots})$ , where a pivot may be e.g. an edge relaxation.  $\mathcal{O}(2^n)$  in the general case.

```
#define T double
typedef vector<T> vd;
typedef vector<vd> vvd;
const T eps = 1e-8, inf = 1./0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j
struct LPSolver {
    int m, n;
    vi N, B;
    vvd D;
    LPSolver(const vvd& A, const vd& b, const vd& c) :
        m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
        rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
        rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i]; }
        rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
        N[n] = -1; D[m+1][n] = 1;
    }
    void pivot(int r, int s) {
        T *a = D[r].data(), inv = 1 / a[s];
        rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
            T *b = D[i].data(), inv2 = b[s] * inv;
            rep(j,0,n+2) b[j] -= a[j] * inv2;
            b[s] = a[s] * inv2;
        }
        rep(j,0,n+2) if (j != s) D[r][j] *= inv;
        rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    }
    bool simplex(int phase) {
        int x = m + phase - 1;
        for (;;) {
            int s = -1;
            rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
            if (D[x][s] >= -eps) return true;
            int r = -1;
            rep(i,0,m) {
                if (D[i][s] <= eps) continue;
                if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
                    < MP(D[r][n+1] / D[r][s], B[r])) r = i;
            }
            if (r == -1) return false;
            pivot(r, s);
        }
    }
    T solve(vd &x) {
        int r = 0;
        rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
        if (D[r][n+1] < -eps) {
            pivot(r, n);
            if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
            rep(i,0,m) if (B[i] == -1) {
                int s = 0;
                rep(j,1,n+1) ltj(D[i]);
                pivot(i, s);
            }
        }
        bool ok = simplex(1); x = vd(n);
        rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
        return ok ? D[m][n+1] : inf;
    }
};
```

### 4.3 Matrices

#### Determinant.h

**Description:** Calculates determinant of a matrix. Destroys the matrix.  
**Time:**  $\mathcal{O}(N^3)$

```
double det(vector<vector<double>>& a) {
    int n = sz(a); double res = 1;
    rep(i,0,n) {
        int b = i;
        rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
        if (i != b) swap(a[i], a[b]), res *= -1;
        res *= a[i][i];
        if (res == 0) return 0;
        rep(j,i+1,n) {
            double v = a[j][i] / a[i][i];
            if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
        }
    }
    return res;
}
```

#### IntDeterminant.h

**Description:** Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.  
**Time:**  $\mathcal{O}(N^3)$

```
const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
    int n = sz(a); ll ans = 1;
    rep(i,0,n) {
        rep(j,i+1,n) {
            while (a[j][i] != 0) { // gcd step
                ll t = a[i][i] / a[j][i];
                if (t) rep(k,i,n)
                    a[i][k] = (a[i][k] - a[j][k] * t) % mod;
                swap(a[i], a[j]);
                ans *= -1;
            }
        }
        ans = ans * a[i][i] % mod;
        if (!ans) return 0;
    }
    return (ans + mod) % mod;
}
```

#### SolveLinear.h

**Description:** Solves  $A * x = b$ . If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in  $A$  and  $b$  is lost.  
**Time:**  $\mathcal{O}(n^2 m)$

```
typedef vector<double> vd;
const double eps = 1e-12;
int solveLinear(vector<vd>& A, vd& b, vd& x) {
    int n = sz(A), m = sz(x), rank = 0, br, bc;
    if (n) assert(sz(A[0]) == m);
    vi col(m); iota(all(col), 0);
    rep(i,0,n) {
        double v, bv = 0;
        rep(r,i,n) rep(c,i,m)
            if ((v = fabs(A[r][c])) > bv)
                br = r, bc = c, bv = v;
        if (bv <= eps) {
            rep(j,i,n) if (fabs(b[j]) > eps) return -1;
            break;
        }
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j,0,n) swap(A[j][i], A[j][bc]);
        bv = 1/A[i][i];
        rep(j,i+1,n) {
            double fac = A[j][i] * bv;
            b[j] -= fac * b[i];
            rep(k,i+1,m) A[j][k] -= fac*A[i][k];
        }
        rank++;
    }
    x.assign(m, 0);
    for (int i = rank; i--;) {
        b[i] /= A[i][i];
        x[col[i]] = b[i];
        rep(j,0,i) b[j] -= A[j][i] * b[i];
    }
    return rank; // (multiple solutions if rank < m)
}
```

#### SolveLinear2.h

**Description:** To get all uniquely determined values of  $x$  back from SolveLinear, make the following changes:

```
"SolveLinear.h"
08e495, 7 lines

rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
    rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
    x[col[i]] = b[i] / A[i][i];
fail;; }
```

#### SolveLinearBinary.h

**Description:** Solves  $Ax = b$  over  $\mathbb{F}_2$ . If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys  $A$  and  $b$ .  
**Time:**  $\mathcal{O}(n^2 m)$

```
typedef bitset<1000> bs;
int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
    int n = sz(A), rank = 0, br;
    assert(m <= sz(x));
    vi col(m); iota(all(col), 0);
    rep(i,0,n) {
        for (br=i; br<n; ++br) if (A[br].any()) break;
        if (br == n) {
            rep(j,i,n) if(b[j]) return -1;
            break;
        }
        int bc = (int)A[br]._Find_next(i-1);
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j,0,n) if (A[j][i] != A[j][bc]) {
            A[j].flip(i); A[j].flip(bc);
        }
        rep(j,i+1,n) if (A[j][i]) {
            b[j] ^= b[i];
            A[j] ^= A[i];
        }
        rank++;
    }
    x = bs();
    for (int i = rank; i--;) {
        if (!b[i]) continue;
        x[col[i]] = i;
        rep(j,0,i) b[j] ^= A[j][i];
    }
    return rank; // (multiple solutions if rank < m)
}
```

#### MatrixInverse.h

**Description:** Invert matrix  $A$ . Returns rank; result is stored in  $A$  unless singular ( $\text{rank} < n$ ). Can easily be extended to prime moduli; for prime powers, repeatedly set  $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$  where  $A^{-1}$  starts as the inverse of  $A \bmod p$ , and  $k$  is doubled in each step.  
**Time:**  $\mathcal{O}(n^3)$

```
int matInv(vector<vector<double>>& A) {
    int n = sz(A); vi col(n);
    vector<vector<double>> tmp(n, vector<double>(n));
    rep(i,0,n) tmp[i][i] = 1, col[i] = i;
    rep(i,0,n) {
        int r = i, c = i;
        rep(j,i,n) rep(k,i,n)
            if (fabs(A[j][k]) > fabs(A[r][c]))
                r = j, c = k;
        if (fabs(A[r][c]) < 1e-12) return i;
        A[i].swap(A[r]); tmp[i].swap(tmp[r]);
        rep(j,0,n)
            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
        swap(col[i], col[c]);
        double v = A[i][i];
        rep(j,i+1,n) {
            double f = A[j][i] / v;
            A[j][i] = 0;
            rep(k,i+1,n) A[j][k] -= f*A[i][k];
            rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
        }
        rep(j,i+1,n) A[i][j] /= v;
        rep(j,0,n) tmp[i][j] /= v;
        A[i][i] = 1;
    }
    for (int i = n-1; i > 0; --i) rep(j,0,i) {
        double v = A[j][i];
        rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
    }
}
```



```
rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
return n;
}
```

### MatrixInverse-mod.h

**Description:** Invert matrix  $A$  modulo a prime. Returns rank; result is stored in  $A$  unless singular (rank < n). For prime powers, repeatedly set  $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$  where  $A^{-1}$  starts as the inverse of  $A \bmod p$ , and k is doubled in each step.

**Time:**  $\mathcal{O}(n^3)$

"/number-theory/ModPow.h"0b7b13, 37 lines

```
int matInv(vector<vector<ll>>& A) {
    int n = sz(A); vi col(n);
    vector<vector<ll>> tmp(n, vector<ll>(n));
    rep(i,0,n) tmp[i][i] = 1, col[i] = i;
    rep(i,0,n) {
        int r = i, c = i;
        rep(j,i,n) rep(k,i,n) if (A[j][k]) {
            r = j; c = k; goto found;
        }
        return i;
    found:
        A[i].swap(A[r]); tmp[i].swap(tmp[r]);
        rep(j,0,n)
            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
        swap(col[i], col[c]);
        ll v = modpow(A[i][i], mod - 2);
        rep(j,i+1,n) {
            ll f = A[j][i] * v % mod;
            A[j][i] = 0;
            rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k]) % mod;
            rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i][k]) % mod;
        }
        rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
        rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
        A[i][i] = 1;
    }
    for (int i = n-1; i > 0; --i) rep(j,0,i) {
        ll v = A[j][i];
        rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][k]) % mod;
    }
    rep(i,0,n) rep(j,0,n)
        A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] < 0)*mod;
    return n;
}
```

### Tridiagonal.h

**Description:**  $x = \text{tridiagonal}(d, p, q, b)$  solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i, \, 1 \leq i \leq n,$$

where  $a_0, a_{n+1}, b_i, c_i$  and  $d_i$  are known.  $a$  can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \dots, -1, 1\}, \{0, c_1, c_2, \dots, c_n\},$$

$$\{b_1, b_2, \dots, b_n, 0\}, \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.

If  $|d_i| > |p_i| + |q_{i-1}|$  for all  $i$ , or  $|d_i| > |p_{i-1}| + |q_i|$ , or the matrix is positive definite, the algorithm is numerically stable and neither tr nor the check for `diag[i] == 0` is needed.

**Time:**  $\mathcal{O}(N)$

8f9fa8, 26 lines

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b) {
    int n = sz(b); vi tr(n);
    rep(i,0,n-1) {
        if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
            b[i+1] -= b[i] * diag[i+1] / super[i];
            if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
            diag[i+1] = sub[i]; tr[i+1] = 1;
        } else {
            diag[i+1] -= super[i]*sub[i]/diag[i];
            b[i+1] -= b[i]*sub[i]/diag[i];
        }
    }
}
```

```
    }
}
for (int i = n; i--;) {
    if (tr[i]) {
        swap(b[i], b[i-1]);
        diag[i-1] = diag[i];
        b[i] /= super[i-1];
    } else {
        b[i] /= diag[i];
        if (i) b[i-1] -= b[i]*super[i-1];
    }
}
return b;
}
```

### 4.4 Fourier transforms

#### FastFourierTransform.h

**Description:** `fft(a)` computes  $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$  for all  $k$ .  $N$  must be a power of 2. Useful for convolution: `conv(a, b) = c`, where  $c[x] = \sum a[i]b[x-i]$ . For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by  $n$ , reverse(start+1, end), FFT back. Rounding is safe if  $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$  (in practice  $10^{16}$ ; higher for random inputs). Otherwise, use NTT/FFTMod.

**Time:**  $\mathcal{O}(N \log N)$  with  $N = |A| + |B|$  ( $\sim 1s$  for  $N = 2^{22}$ )

00ced6, 35 lines

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1); // (^ 10% faster if double)
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
            a[i + j + k] = a[i + j] - z;
            a[i + j] += z;
        }
}
vd conv(const vd& a, const vd& b) {
    if (a.empty() || b.empty()) return {};
    vd res(sz(a) + sz(b) - 1);
    int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
    vector<C> in(n), out(n);
    copy(all(a), begin(in));
    rep(i,0,sz(b)) in[i].imag(b[i]);
    fft(in);
    for (C& x : in) x *= x;
    rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
    return res;
}
```

#### FastFourierTransformMod.h

**Description:** Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as  $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$  (in practice  $10^{16}$  or higher). Inputs must be in  $[0, \text{mod})$ .

**Time:**  $\mathcal{O}(N \log N)$ , where  $N = |A| + |B|$  (twice as slow as NTT or FFT)

"FastFourierTransform.h"b82773, 22 lines

```
typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    vl res(sz(a) + sz(b) - 1);
    int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);
    rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
    rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(L), fft(R);
    rep(i,0,n) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }
    fft(outl), fft(outs);
    rep(i,0,sz(res)) {
        ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
        ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
}
```

```
    }
    return res;
}
```

### NumberTheoreticTransform.h

**Description:** `ntt(a)` computes  $\hat{f}(k) = \sum_x a[x]g^{xk}$  for all  $k$ , where  $g = \text{root}^{(\text{mod}-1)/N}$ .  $N$  must be a power of 2. Useful for convolution modulo specific nice primes of the form  $2^a b + 1$ , where the convolution result has size at most  $2^a$ . For arbitrary modulo, see FFTMod. `conv(a, b) = c`, where  $c[x] = \sum a[i]b[x-i]$ . For manual convolution: NTT the inputs, multiply pointwise, divide by  $n$ , reverse(start+1, end), NTT back. Inputs must be in  $[0, \text{mod})$ .

**Time:**  $\mathcal{O}(N \log N)$

"/number-theory/ModPow.h"ced03d, 35 lines

```
const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.
typedef vector<ll> vl;
void ntt(vl &a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vl rt(2, 1);
    for (static int k = 2, s = 2; k < n; k *= 2, s++) {
        rt.resize(n);
        ll z[] = {1, modpow(root, mod >> s)};
        rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
            a[i + j + k] = ai - z + (z > ai ? mod : 0);
            ai += (ai + z >= mod ? z - mod : z);
        }
}
vl conv(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
        n = 1 << B;
    int inv = modpow(n, mod - 2);
    vl L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    rep(i,0,n)
        out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
}
```

### FastSubsetTransform.h

**Description:** Transform to a basis with fast convolutions of the form  $c[z] = \sum_{s=x \oplus y} a[x] \cdot b[y]$ , where  $\oplus$  is one of AND, OR, XOR. The size of  $a$  must be a power of two.

**Time:**  $\mathcal{O}(N \log N)$

464cf3, 16 lines

```
void FST(vi& a, bool inv) {
    for (int n = sz(a), step = 1; step < n; step *= 2) {
        for (int i = 0; i < n; i += 2 * step) rep(j,i,step) {
            int &u = a[j], &v = a[j + step]; tie(u, v) =
                inv ? pii(v - u, u) : pii(v, u + v); // AND
                inv ? pii(v, u - v) : pii(u + v, u); // OR
                pii(u + v, u - v); // XOR
        }
        if (inv) for (int& x : a) x /= sz(a); // XOR only
    }
}
vi conv(vi a, vi b) {
    FST(a, 0); FST(b, 0);
    rep(i,0,sz(a)) a[i] *= b[i];
    FST(a, 1); return a;
}
```

## Number theory (5)

### 5.1 Modular arithmetic

#### ModInverse.h

**Description:** Pre-computation of modular inverses. Assumes  $\text{LIM} \leq \text{mod}$  and that `mod` is a prime.

6f684f, 3 lines

```
const ll mod = 1000000007, LIM = 200000;
```



```
ll s inv = new ll[LIM] - 1; inv[1] = 1;
rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

|                                                                                                                                                                                                       |                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| ModPow.h                                                                                                                                                                                              | b83e45, 8 lines |
| <pre>const ll mod = 1000000007; // faster if const ll modpow(ll b, ll e) {     ll ans = 1;     for (; e; b = b * b % mod, e /= 2)         if (e &amp; 1) ans = ans * b % mod;     return ans; }</pre> |                 |

|                                                                                                                                                                                                                                                |                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| ModLog.h                                                                                                                                                                                                                                       |                  |
| <p><b>Description:</b> Returns the smallest <math>x &gt; 0</math> s.t. <math>a^x = b \pmod m</math>, or <math>-1</math> if no such <math>x</math> exists. <code>modLog(a,1,m)</code> can be used to calculate the order of <math>a</math>.</p> |                  |
| <b>Time:</b> $\mathcal{O}(\sqrt{m})$                                                                                                                                                                                                           | c040b8, 11 lines |

```
ll modLog(ll a, ll b, ll m) {
    ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;
    while (j <= n && (e = f = e * a % m) != b % m)
        A[e * b % m] = j++;
    if (e == b % m) return j;
    if (__gcd(m, e) == __gcd(m, b))
        rep(i,2,n+2) if (A.count(e = e * f % m))
            return n * i - A[e];
    return -1;
}
```

|                                                                                                                                                  |                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| ModSum.h                                                                                                                                         |                  |
| <p><b>Description:</b> Sums of mod'ed arithmetic progressions.</p>                                                                               |                  |
| <p><code>modsum(to, c, k, m) = <math>\sum_{i=0}^{to-1} (ki + c) \% m</math>.</code> <code>divsum</code> is similar but for floored division.</p> |                  |
| <b>Time:</b> $\log(m)$ , with a large constant.                                                                                                  | 5c5bc5, 16 lines |

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
ull divsum(ull to, ull c, ull k, ull m) {
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m; c %= m;
    if (!k) return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}
ll modsum(ull to, ll c, ll k, ll m) {
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

|                                                                                                                                                               |                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| ModMulLL.h                                                                                                                                                    |                  |
| <p><b>Description:</b> Calculate <math>a \cdot b \pmod c</math> (or <math>a^b \pmod c</math>) for <math>0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}</math>.</p> |                  |
| <b>Time:</b> $\mathcal{O}(1)$ for <code>modmul</code> , $\mathcal{O}(\log b)$ for <code>modpow</code>                                                         | bbb8df, 11 lines |

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1) ans = modmul(ans, b, mod);
    return ans;
}
```

|                                                                                                                                                                                  |                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| ModSqrt.h                                                                                                                                                                        |                  |
| <p><b>Description:</b> Tonelli-Shanks algorithm for modular square roots. Finds <math>x</math> s.t. <math>x^2 = a \pmod p</math> (<math>-x</math> gives the other solution).</p> |                  |
| <p><b>Time:</b> <math>\mathcal{O}(\log^2 p)</math> worst case, <math>\mathcal{O}(\log p)</math> for most <math>p</math></p>                                                      |                  |
| "ModPow.h"                                                                                                                                                                       | 19a793, 24 lines |

```
ll sqrt(ll a, ll p) {
    a %= p; if (a < 0) a += p;
    if (a == 0) return 0;
    assert(modpow(a, (p-1)/2, p) == 1); // else no solution
    if (p % 4 == 3) return modpow(a, (p+1)/4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
    ll s = p - 1, n = 2;
    int r = 0, m;
    while (s % 2 == 0)
        ++r, s /= 2;
    while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
    ll x = modpow(a, (s + 1) / 2, p);
```

```
ll b = modpow(a, s, p), g = modpow(n, s, p);
for (; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)
        t = t * t % p;
    if (m == 0) return x;
    ll gs = modpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
}
```

|                                                                                    |                  |
|------------------------------------------------------------------------------------|------------------|
| 5.2 Primality                                                                      |                  |
| FastEratosthenes.h                                                                 |                  |
| <p><b>Description:</b> Prime sieve for generating all primes smaller than LIM.</p> |                  |
| <b>Time:</b> LIM=1e9 $\approx 1.5s$                                                | 6b2912, 20 lines |

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
    const int S = (int)round(sqrt(LIM)), R = LIM / 2;
    pr = {2}, sieve(S+1); pr.reserve((int)(LIM/log(LIM)*1.1));
    vector<pii> cp;
    for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
        cp.push_back({i, i * i / 2});
        for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
    }
    for (int L = 1; L <= R; L += S) {
        array<bool, S> block{};
        for (auto &[p, idx] : cp)
            for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
        rep(i,0,min(S, R - L))
            if (!block[i]) pr.push_back((L + i) * 2 + 1);
    }
    for (int i : pr) isPrime[i] = 1;
    return pr;
}
```

|                                                                                                                                                                                                |                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| MillerRabin.h                                                                                                                                                                                  |                  |
| <p><b>Description:</b> Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to <math>7 \cdot 10^{18}</math>; for larger numbers, use Python and extend A randomly.</p> |                  |
| <p><b>Time:</b> 7 times the complexity of <math>a^b \pmod c</math>.</p>                                                                                                                        |                  |
| "ModMulLL.h"                                                                                                                                                                                   | 60ded1, 12 lines |

```
bool isPrime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
        s = __builtin_ctzll(n-1), d = n >> s;
    for (ull a : A) { // ^ count trailing zeroes
        ull p = modpow(a%n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = modmul(p, p, n);
        if (p != n-1 && i != s) return 0;
    }
    return 1;
}
```

|                                                                                                                                                                  |                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| Factor.h                                                                                                                                                         |                  |
| <p><b>Description:</b> Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -&gt; {11, 19, 11}).</p> |                  |
| <p><b>Time:</b> <math>\mathcal{O}(n^{1/4})</math>, less for numbers with small factors.</p>                                                                      |                  |
| "ModMulLL.h", "MillerRabin.h"                                                                                                                                    | d8d98d, 18 lines |

```
ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, g;
    auto f = [&i](ull x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((g = modmul(prd, max(x,y) - min(x,y), n)) > 1) prd = g;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}
vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
```

|                                                                                                                                                                                                  |                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| Pell.h                                                                                                                                                                                           |                  |
| <p><b>Description:</b> <math>\mathcal{O}(\log n)</math>, <code>pell(n)</code> oblicza rozwiązanie fundamentalne <math>x^2 - ny^2 = 1</math>, zwraca <math>(0, 0)</math> jeżeli nie istnieje.</p> |                  |
|                                                                                                                                                                                                  | 930a36, 27 lines |

```
pair<LL, LL> pell(LL n) {
    LL s = LL(sqrt1(n));
    if (s * s == n) return {0, 0};
    LL m = 0, d = 1, a = s;
    __int128 num1 = 1, num2 = a, den1 = 0, den2 = 1;
    while (num2 * num2 - n * den2 * den2 != 1) {
        m = d * a - m;
        d = (n - m * m) / d;
        a = (s + m) / d;
        if (num2 > (1ll << 62) / a) return {0, 0};
        tie(num1, num2) = pair(num2, a * num2 + num1);
        tie(den1, den2) = pair(den2, a * den2 + den1);
    }
    return {num2, den2};
}
vector<pair<LL, LL>> all_pell(LL n, LL limit) {
    auto [x0, y0] = pell(n);
    if (!x0) return {};
    vector<pair<LL, LL>> ret;
    __int128 x = x0, y = y0;
    while (x <= limit) {
        ret.emplace_back(x, y);
        if (y0 * y > (1ll << 62) / n) break;
        tie(x, y) = pair(x0 * x + n * y0 * y, x0 * y + y0 * x);
    }
    return ret;
}
```

|                                                                                                                                                                                                                          |                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| Pi.h                                                                                                                                                                                                                     |                  |
| <p><b>Description:</b> <math>\mathcal{O}(n^{\frac{3}{4}})</math>, liczba liczb pierwszych na przedziale <math>[1, n]</math>. <code>Pi pi(n); pi.query(d);</code> musi zachodzić <code>d</code> dzieli <code>n</code></p> |                  |
|                                                                                                                                                                                                                          | 5af6fc, 28 lines |

```
struct Pi {
    vector<LL> w, dp;
    int id(LL v) {
        if (v <= w.back() / v)
            return int(v - 1);
        return ssize(w) - int(w.back() / v);
    }
    Pi(LL n) {
        for (LL i = 1; i * i <= n; ++i) {
            w.push_back(i);
            if (n / i != i)
                w.emplace_back(n / i);
        }
        sort(w.begin(), w.end());
        for (LL i : w)
            dp.emplace_back(i - 1);
        for (LL i = 1; (i + 1) * (i + 1) <= n; ++i) {
            if (dp[i] == dp[i - 1])
                continue;
            for (int j = ssize(w) - 1; w[j] >= (i + 1) * (i + 1); --j)
                dp[j] -= dp[id(w[j] / (i + 1))] - dp[i - 1];
        }
        LL query(LL v) {
            assert(w.back() % v == 0);
            return dp[id(v)];
        }
};
```

|                                                                                                                                                                                                                                          |                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| FloorSum.h                                                                                                                                                                                                                               |                  |
| <p><b>Description:</b> <math>\mathcal{O}(\log a)</math>, liczy <math>\sum_{i=0}^{n-1} \left\lfloor \frac{a \cdot i + b}{c} \right\rfloor</math>. Działa dla <math>0 \leq a, b &lt; c</math> oraz <math>1 \leq c, n \leq 10^9</math>.</p> |                  |
|                                                                                                                                                                                                                                          | 78c6f7, 15 lines |

```
LL floor_sum(LL n, LL a, LL b, LL c) {
    LL ans = 0;
    if (a >= c) {
        ans += (n - 1) * n * (a / c) / 2;
        a %= c;
    }
    if (b >= c) {
        ans += n * (b / c);
        b %= c;
    }
    LL d = (a * (n - 1) + b) / c;
    if (d == 0) return ans;
    ans += d * (n - 1) - floor_sum(d, c, c - b - 1, a);
    return ans;
}
```

5.3 Divisibility

euclid.h

**Description:** Finds two integers  $x$  and  $y$ , such that  $ax + by = \gcd(a, b)$ . If you just need gcd, use the built in `__gcd` instead. If  $a$  and  $b$  are coprime, then  $x$  is the inverse of  $a \pmod b$ .

```
33ba8f, 5 lines
11 euclid(11 a, 11 b, 11 &x, 11 &y) {
    if (!b) return x = 1, y = 0, a;
    11 d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}
```

CRT.h

**Description:** Chinese Remainder Theorem. `crt(a, m, b, n)` computes  $x$  such that  $x \equiv a \pmod m, x \equiv b \pmod n$ . If  $|a| < m$  and  $|b| < n$ ,  $x$  will obey  $0 \leq x < \text{lcm}(m, n)$ . Assumes  $mn < 2^{62}$ .  
**Time:**  $\mathcal{O}(\log n)$

```
04d93a, 7 lines
11 crt(11 a, 11 m, 11 b, 11 n) {
    if (n > m) swap(a, b), swap(m, n);
    11 x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}
```

Min25.h

**Description:** Calculates prefsums of multiplicative function at each floor( $N/i$ ). `keys[id(N/i)]=N/i`. Remember about overflows. See example below.  
**Time:**  $\mathcal{O}\left(\frac{n^{3/4}}{\log n}\right)$

```
f4fd1a, 50 lines
vector<ll> global_primes; // global_primes[-1]>=sqrt(N)
template<typename T>
struct Min25 {
    11 N;
    vector<ll> keys, primes;
    Min25(11 N_) : N(N_) {
        for (11 l = 1; l <= N; ++l)
            keys.pb(l = N / (N / l));
        for (int i = 0; global_primes[i] * global_primes[i] <= N; ++i)
            primes.pb(global_primes[i]);
    }
    11 id(11 x) {
        11 id = x < N / x ? x - 1 : sz(keys) - N / x;
        assert(keys[id] == x);
        return id;
    }
    // f has to be TOTALLY multiplicative
    // pref(x) is regular prefix sum function of f
    vector<T> overPrimes(auto pref) {
        vector<T> dp(sz(keys));
        rep(i, sz(keys))
            dp[i] = pref(keys[i]) - T(1);
        for (11 p : primes) {
            auto fp = dp[p - 1] - dp[p - 2];
            for (int i = sz(keys) - 1; i >= 0 && p * p <= keys[i]; --i)
                dp[i] = dp[i] - (dp[id(keys[i] / p)] - dp[p - 2]) * fp;
        }
        return dp;
    }
    // dp are prefix sums of f over primes
    // f(p, k, p**k) calculates f on primes powers
    void fullSum(vector<T> &dp, auto f) {
        for (11 p : primes | views::reverse) {
            for (int i = sz(keys) - 1; i >= 0 && p * p <= keys[i]; --i) {
                for (11 k = 1, q = p; q * p <= keys[i]; ++k, q *= p)
                    dp[i] = dp[i] + f(p, k + 1, q * p) + f(p, k, q) * (dp[id(keys[i] / q)] - dp[p - 1]);
            }
        }
        for (auto &v : dp) v = v + T(1);
    }
}
vector<ll> exampleUsage(Min25<ll> &m) { // OVERFLOWS!
    auto primeCnt = m.overPrimes([](11 x){return x; });
    auto primeSum = m.overPrimes([](11 x){return x*(x+1)/2; });
    vector<ll> phi; rep(i, sz(m.keys))
        phi.pb(primeSum[i] - primeCnt[i]);
    m.fullSum(phi, [](int p,int k,11 pk){return pk-pk/p; });
    return phi; }
```

LinearSieve.h

**Description:** Usefull for computing values of multiplicative function and its prefix sums.  
**Time:**  $\mathcal{O}(N)$

```
308681, 29 lines
struct LinearSieve {
    vector<bool> isComposite; vi prime, cnt;
    vector<ll> phi, prefPhi;
    11 dPhi(11 x, int p, int a) { // x / phi(p^(a-1)) * phi(p^a)
        return x * (a == 1 ? p - 1 : p);
    }
    LinearSieve(int n) : isComposite(n), cnt(n), phi(n) {
        if(n > 1) phi[1] = 1;
        FOR(i, 2, n) {
            if(!isComposite[i]) {
                prime.pb(i), cnt[i] = 1, phi[i] = dPhi(1, i, 1);
            }
            FOR(j, 0, SZ(prime)) {
                if(i * prime[j] >= n) break;
                isComposite[i * prime[j]] = 1;
                if(i % prime[j] == 0) {
                    cnt[i * prime[j]] = cnt[i] + 1;
                    phi[i*prime[j]] = dPhi(phi[i], prime[j], cnt[i]+1);
                    break;
                }
                else {
                    cnt[i * prime[j]] = 1;
                    phi[i * prime[j]] = phi[i] * phi[prime[j]];
                }
            }
        }
        partial_sum(all(phi), back_inserter(prefPhi));
    }
};
```

5.4 Pisano period

$\pi(n)$  is a period of Fibbonacci sequence modulo  $n$ .  
 $\pi(nm) = \pi(n)\pi(m)$  for  $n \perp m, \pi(p^k) = p^{k-1}\pi(p)$ .

$$\pi(p) \begin{cases} = 3 & p = 2 \\ = 20 & p = 5 \\ \mid p - 1 & p \equiv_{10} \pm 1 \\ \mid 2(p + 1) & p \equiv_{10} \pm 3 \end{cases}$$

$F_i \equiv_p -F_{i+p+1}$  for  $p \equiv_{10} \pm 3. \pi(n) \leq 4n$  for  $n \neq 2 \cdot 5^r$ .

5.5 Morbius

Niech  $M(n) = \sum_{i=1}^n \mu(i)$ . Można policzyć  $M(n)$  w  $\mathcal{O}\left(n^{2/3} \cdot \log(\text{smth})\right)$ . Dla  $u = n^{1/3}$ , wystarczy spreprocesować  $M$  do  $n^{2/3}$  i obliczyć  $M(n)$  wzorem:

$$M(n) = M(u) - \sum_{m=1}^u \mu(m) \sum_{i=\lfloor \frac{u}{m} \rfloor + 1}^{\lfloor \frac{n}{m} \rfloor} M\left(\left\lfloor \frac{n}{mi} \right\rfloor\right).$$

$$\mu(p^k) = [k = 0] - [k = 1]$$

$$\sum_{d|n} \mu(d) = [n = 1]$$

$$\sum_{d|n} \phi(d) = n$$

$$\phi(p^k) = p^k - p^{k-1}$$

5.6 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with  $m > n > 0, k > 0, m \perp n$ , and either  $m$  or  $n$  even.

5.7 Pythagorean Tree

Primitive Pythagorean triples form infinite ternary tree, where each triple occurs exactly once. Node is a column vector  $(a, b, c = \sqrt{a^2 + b^2})$ , root is  $(3, 4, 5)$ , and each child is given by a product of a parent and one of the:

$$\begin{bmatrix} 1 & -2 & 2 \\ 2 & -1 & 2 \\ 2 & -2 & 3 \end{bmatrix}, \begin{bmatrix} 1 & 2 & 2 \\ 2 & 1 & 2 \\ 2 & 2 & 3 \end{bmatrix}, \begin{bmatrix} -1 & 2 & 2 \\ -2 & 1 & 2 \\ -2 & 2 & 3 \end{bmatrix}$$

5.8 Primes

$p = 962592769$  is such that  $2^{21} \mid p - 1$ , which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power  $p^a$ , except for  $p = 2, a > 2$ , and there are  $\phi(\phi(p^a))$  many. For  $p = 2, a > 2$ , the group  $\mathbb{Z}_{2^a}^\times$  is instead isomorphic to  $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$ .

5.9 Estimates

$$\sum_{d|n} d = \mathcal{O}(n \log \log n).$$

The number of divisors of  $n$  is at most around 100 for  $n < 5e4$ , 500 for  $n < 1e7$ , 2000 for  $n < 1e10$ , 200 000 for  $n < 1e19$ .

5.10 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$$\sum_{d|n} \mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g(\lfloor \frac{n}{m} \rfloor)$$

Define Dirichlet convolution as  $f * g(n) = \sum_{d|n} f(d)g(n/d)$ . Let  $s_f(n) = \sum_{i=1}^n f(i)$ . Then  $s_f(n)g(1) = s_{f*g}(n) - \sum_{d=2}^n s_f(\lfloor \frac{n}{d} \rfloor)g(d)$ .

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

| <i>n</i>   | 1     | 2     | 3     | 4      | 5      | 6      | 7      | 8        | 9      | 10      |
|------------|-------|-------|-------|--------|--------|--------|--------|----------|--------|---------|
| <i>n</i> ! | 1     | 2     | 6     | 24     | 120    | 720    | 5040   | 40320    | 362880 | 3628800 |
| <i>n</i>   | 11    | 12    | 13    | 14     | 15     | 16     | 17     |          |        |         |
| <i>n</i> ! | 4.0e7 | 4.8e8 | 6.2e9 | 8.7e10 | 1.3e12 | 2.1e13 | 3.6e14 |          |        |         |
| <i>n</i>   | 20    | 25    | 30    | 40     | 50     | 100    | 150    | 171      |        |         |
| <i>n</i> ! | 2e18  | 2e25  | 3e32  | 8e47   | 3e64   | 9e157  | 6e262  | >DBL_MAX |        |         |

IntPerm.h

**Description:** Permutation -> integer conversion. (Not order preserving). Integer -> permutation can use a lookup table.

**Time:**  $\mathcal{O}(n)$

```
int permToInt(vi& v) {
    int use = 0, i = 0, r = 0;
    for(int x:v) r = r * ++i + __builtin_popcount(use & ~(1<<x)),
        use |= 1 << x;
    return r;
}
```

044568, 6 lines

6.1.2 Cycles

Let  $g_S(n)$  be the number of  $n$ -permutations whose cycle lengths all belong to the set  $S$ . Then

$$\sum_{n=0}^\infty g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n\in S} \frac{x^n}{n}\right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.1.4 Burnside’s lemma

Given a group  $G$  of symmetries and a set  $X$ , the number of elements of  $X$  *up to symmetry* equals

$$\frac{1}{|G|} \sum_{g\in G} |X^g|,$$

where  $X^g$  are the elements fixed by  $g$  ( $g.x = x$ ).

If  $f(n)$  counts “configurations” (of some sort) of length  $n$ , we can ignore rotational symmetry using  $G = \mathbb{Z}_n$  to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing  $n$  as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \; p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

| <i>n</i>              | 0 | 1 | 2 | 3 | 4 | 5 | 6  | 7  | 8  | 9  | 20  | 50   | 100  |
|-----------------------|---|---|---|---|---|---|----|----|----|----|-----|------|------|
| <i>p</i> ( <i>n</i> ) | 1 | 1 | 2 | 3 | 5 | 7 | 11 | 15 | 22 | 30 | 627 | ~2e5 | ~2e8 |

6.2.2 Lucas’ Theorem

Let  $n, m$  be non-negative integers and  $p$  a prime. Write  $n = n_k p^k + \dots + n_1 p + n_0$  and  $m = m_k p^k + \dots + m_1 p + m_0$ . Then  $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod{p}$ .

6.2.3 Binomials

multinomial.h

**Description:** Computes  $\binom{k_1 + \dots + k_n}{k_1, k_2, \dots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \dots k_n!}$ .

a0a312, 6 lines

```
ll multinomial(vi& v) {
    ll c = 1, m = v.empty() ? 1 : v[0];
    rep(i,1,sz(v)) rep(j,0,v[i])
        c = c * ++m / (j+1);
    return c;
}
```

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is  $B(t) = \frac{t}{e^t - 1}$  (FFT-able).  
 $B[0, \dots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \dots]$

Sums of powers:

$$\sum_{i=1}^n n^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\begin{aligned} \sum_{i=m}^\infty f(i) &= \int_m^\infty f(x) dx - \sum_{k=1}^\infty \frac{B_k}{k!} f^{(k-1)}(m) \\ &\approx \int_m^\infty f(x) dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m)) \end{aligned}$$

6.3.2 Stirling numbers of the first kind

Number of permutations on  $n$  items with  $k$  cycles.

$$c(n, k) = c(n-1, k-1) + (n-1)c(n-1, k), \; c(0, 0) = 1$$
$$\sum_{k=0}^n c(n, k) x^k = x(x+1) \dots (x+n-1)$$

$$c(8, k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$$
$$c(n, 2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$$

6.3.3 Eulerian numbers

Number of permutations  $\pi \in S_n$  in which exactly  $k$  elements are greater than the previous element.  $k$   $j$ :s s.t.  $\pi(j) > \pi(j+1)$ ,  $k+1$   $j$ :s s.t.  $\pi(j) \geq j$ ,  $k$   $j$ :s s.t.  $\pi(j) > j$ .

$$E(n, k) = (n-k)E(n-1, k-1) + (k+1)E(n-1, k)$$

$$E(n, 0) = E(n, n-1) = 1$$

$$E(n, k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of  $n$  distinct elements into exactly  $k$  groups.

$$S(n, k) = S(n-1, k-1) + kS(n-1, k)$$

$$S(n, 1) = S(n, n) = 1$$

$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

6.3.5 Bell numbers

Total number of partitions of  $n$  distinct elements.  $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$ . For  $p$  prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$$

6.3.6 Labeled unrooted trees

# on  $n$  vertices:  $n^{n-2}$   
# on  $k$  existing trees of size  $n_i$ :  $n_1 n_2 \dots n_k n^{k-2}$   
# with degrees  $d_i$ :  $(n-2)! / ((d_1-1)! \dots (d_n-1)!)$

6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1, \; C_{n+1} = \frac{2(2n+1)}{n+2} C_n, \; C_{n+1} = \sum C_i C_{n-i}$$

$$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$$

- sub-diagonal monotone paths in an  $n \times n$  grid.
- strings with  $n$  pairs of parenthesis, correctly nested.
- binary trees with with  $n+1$  leaves (0 or 2 children).
- ordered trees with  $n+1$  vertices.
- ways a convex polygon with  $n+2$  sides can be cut into triangles by connecting vertices with straight lines.
- permutations of  $[n]$  with no 3-term increasing subseq.

6.4 Other

DeBruijn.h

**Description:** Recursive FKM, given alphabet  $[0, k)$  constructs cyclic string of length  $k^n$  that contains every length  $n$  string as substr.

a7faa5, 13 lines

```
vi dseq(int k, int n) {
    if (k == 1) return {0};
    vi res, aux(n+1);
    function<void(int,int)> gen = [&](int t, int p) {
        if (t > n) { // consider lyndon word of len p
            if (n%p == 0) FOR(i,1,p+1) res.pb(aux[i]);
        } else {
            aux[t] = aux[t-p]; gen(t+1,p);
            FOR(i,aux[t-p]+1,k) aux[t] = i, gen(t+1,t);
        }
    };
    gen(1,1); return res;
}
```

### NimProduct.h

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <b>Description:</b> Nim Product.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 9bba25, 18 lines |
| <pre>using ull = uint64_t; ull _nimProd2[64][64]; ull nimProd2(int i, int j) {     if (_nimProd2[i][j]) return _nimProd2[i][j];     if ((i &amp; j) == 0) return _nimProd2[i][j] = 1ull &lt;&lt; (i j);     int a = (i&amp;j) &amp; ~(i&amp;j);     return _nimProd2[i][j] = nimProd2(i ^ a, j         ^ nimProd2((i ^ a)   (a-1), (j ^ a)   (i &amp; (a-1)))); } ull nimProd(ull x, ull y) {     ull res = 0;     for (int i = 0; (x &gt;&gt; i) &amp;&amp; i &lt; 64; i++)         if ((x &gt;&gt; i) &amp; 1)             for (int j = 0; (y &gt;&gt; j) &amp;&amp; j &lt; 64; j++)                 if ((y &gt;&gt; j) &amp; 1)                     res ^= nimProd2(i, j);     return res; }</pre> |                  |

### GrayCode.h

**Description:** Gray code: gray(0),...,gray(2<sup>n</sup> − 1) - permutation in which each two consecutive (cyclically) numbers. differ in exactly one bit

|                                                                                                                                                                                                                                                              |                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| <pre>using ull = unsigned long long; ull gray(ull i) { return i^i&gt;&gt;1; } ull invg(ull i) { <i>// i=invg(gray(i))=gray(inv(i))</i>     i^=i&gt;&gt;1; i^=i&gt;&gt;2; i^=i&gt;&gt;4;     i^=i&gt;&gt;8; i^=i&gt;&gt;16; i^=i&gt;&gt;32; return i; }</pre> | bitc82, 6 lines |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|

## Graph (7)

### 7.1 Fundamentals

#### Shapes.h

**Description:** Counts all subgraph shapes with at most 4 edges. No multi-edges / loops allowed;

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <b>Time:</b> $\mathcal{O}(m\sqrt{m})$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | f62c2b, 39 lines |
| <pre>struct Shapes {     ll tri = 0, rect = 0, path3 = 0, path4 = 0, star3 = 0, p = 0;     __int128_t y = 0, star4 = 0;     Shapes(vector&lt;vi&gt; &amp;g) {         int n = sz(g);         vector&lt;vi&gt; h(n);         vector&lt;ll&gt; f(n), c(n), s(n);         rep(v, n) f[v] = {s[v] = sz(g[v])} * n + v;         rep(v, n) {             ll x = 0;             star3 += s[v] * (s[v] - 1) * (s[v] - 2);             star4 += __int128_t(s[v] - 1) * s[v] * (s[v] - 2) * (s[v] - 3);             for (auto u : g[v]) {                 path4 += s[u] * x - x; x += s[u] - 1;                 y += (s[v] - 1) * (s[u] - 1) * (s[u] - 2) / 2;                 if (f[u] &lt; f[v]) h[v].pb(u);             }         }         rep(v, n) {             for (int u : h[v])                 for (int w : g[u]) if (f[v] &gt; f[w])                     rect += c[w] ++;             for(int u : h[v]) {                 tri += c[u]; c[u] -= 1;                 path3 += (s[v] - 1) * (s[u] - 1);                 for(int w : g[u])                     if (c[w] &lt; 0)                         p += s[v] + s[u] + s[w] - 6, c[w] ++;                     else if (c[w] &gt; 0)                         c[w] --;             }         }         path3 -= 3 * tri;         y -= 2 * p;         path4 -= 4 * rect + 2 * p + 3 * tri;         star3 /= 6;         star4 /= 24;     } };</pre> |                  |

#### SPFA.h

**Description:** SPFA with subtree erasure heuristic. Returns array of distances or empty array if negative cycle is reachable from source. par[v] = parent in shortest path tree

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <b>Time:</b> $\mathcal{O}(VE)$ but fast on random                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | bf4a3f, 27 lines |
| <pre>using Edge = pair&lt;int, ll&gt;; vector&lt;ll&gt; spfa(vector&lt;vector&lt;Edge&gt;&gt;&amp; G,                 vi&amp; par, int src) {     int n = sz(G); vi que, prv(n+1);     iota(all(prv), 0); vi nxt = prv;     vector&lt;ll&gt; dist(n, INT64_MAX);     par.assign(n, -1);     auto add = [&amp;](int v, int p, ll d) {         par[v] = p; dist[v] = d;         prv[n] = nxt[prv[v]] = prv[nxt[v] = n] = v;     }; auto del = [&amp;](int v) {         nxt[prv[nxt[v]]] = prv[v] = nxt[v];         prv[v] = nxt[v] = v;     }; for (add(src, -2, 0); nxt[n] != n;) {         int v = nxt[n]; del(v);         for (auto e : G[v]) {             ll alt = dist[v] + e.y;             if (alt &lt; dist[e.x]) {                 que = {e.x};                 rep(i, sz(que)) {                     int w = que[i]; par[w] = -1;                     del(w);                     for (auto f : G[w])                         if (par[f.x] == w) que.pb(f.x);                 } if (par[v] == -1) return {};                 add(e.x, v, alt);             }         }     } return dist; }</pre> |                  |

### 7.2 Network flow

#### MinCostMaxFlow.h

**Description:** Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.

**Time:**  $\mathcal{O}(FE\log(V))$  where F is max flow.  $\mathcal{O}(VE)$  for setpi. 58385b, 79 lines

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <pre>#include &lt;bits/extc++.h&gt; const ll INF = numeric_limits&lt;ll&gt;::max() / 4; struct MCMF {     struct edge {         int from, to, rev;         ll cap, cost, flow;     };     int N;     vector&lt;vector&lt;edge&gt;&gt; ed;     vi seen;     vector&lt;ll&gt; dist, pi;     vector&lt;edge*&gt; par;     MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}     void addEdge(int from, int to, ll cap, ll cost) {         if (from == to) return;         ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 });         ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 });     }     void path(int s) {         fill(all(seen), 0);         fill(all(dist), INF);         dist[s] = 0; ll di;         __gnu_pbds::priority_queue&lt;pair&lt;ll, int&gt;&gt; q;         vector&lt;decltype(q)::point_iterator&gt; its(N);         q.push({ 0, s });         while (!q.empty()) {             s = q.top().second; q.pop();             seen[s] = 1; di = dist[s] + pi[s];             for (edge&amp; e : ed[s]) if (!seen[e.to]) {                 ll val = di - pi[e.to] + e.cost;                 if (e.cap - e.flow &gt; 0 &amp;&amp; val &lt; dist[e.to]) {                     dist[e.to] = val;                     par[e.to] = &amp;e;                     if (its[e.to] == q.end())                         its[e.to] = q.push({ -dist[e.to], e.to });                     else                         q.modify(its[e.to], { -dist[e.to], e.to });                 }             }         }         rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);     }     pair&lt;ll, ll&gt; maxflow(int s, int t) {         ll totflow = 0, totcost = 0;         while (path(s), seen[t]) {             ll fl = INF;             for (edge* x = par[t]; x; x = par[x-&gt;from])                 fl = min(fl, x-&gt;cap - x-&gt;flow);             totflow += fl;             for (edge* x = par[t]; x; x = par[x-&gt;from]) {                 x-&gt;flow += fl;                 ed[x-&gt;to][x-&gt;rev].flow -= fl;             }         }</pre> |  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <pre>}     rep(i,0,N) for(edge&amp; e : ed[i]) totcost += e.cost * e.flow;     return {totflow, totcost/2}; } <i>// If some costs can be negative, call this before maxflow:</i> void setpi(int s) { <i>// (otherwise, leave this out)</i>     fill(all(pi), INF); pi[s] = 0;     int it = N, ch = 1; ll v;     while (ch-- &amp;&amp; it--)         rep(i,0,N) if (pi[i] != INF)             for (edge&amp; e : ed[i]) if (e.cap                 if ((v = pi[i] + e.cost) &lt; pi[e.to])                     pi[e.to] = v, ch = 1;             assert(it &gt;= 0); <i>// negative cost cycle</i>     } };</pre> |  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|

#### Dinic.h

**Description:** Flow algorithm with complexity  $\mathcal{O}(VE\log U)$  where  $U = \max|\text{cap}|$ .  $\mathcal{O}(\min(E^{1/2}, V^{2/3})E)$  if  $U = 1$ ;  $\mathcal{O}(\sqrt{V}E)$  for bipartite matching.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <pre>struct Dinic {     struct Edge {         int to, rev;         ll c, oc;         ll flow() { return max(oc - c, 0LL); } <i>// if you need flows</i>     };     vi lvl, ptr, q;     vector&lt;vector&lt;Edge&gt;&gt; adj;     Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}     void addEdge(int a, int b, ll c, ll rcap = 0) {         adj[a].push_back({b, sz(adj[b]), c, c});         adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});     } ll dfs(int v, int t, ll f) {         if (v == t    !f) return f;         for (int&amp; i = ptr[v]; i &lt; sz(adj[v]); i++) {             Edge&amp; e = adj[v][i];             if (lvl[e.to] == lvl[v] + 1)                 if (ll p = dfs(e.to, t, min(f, e.c))) {                     e.c -= p, adj[e.to][e.rev].c += p;                     return p;                 } return 0;         } ll calc(int s, int t) {             ll flow = 0; q[0] = s;             rep(L,0,31) do { <i>// 'int L=30' maybe faster for random data</i>                 lvl = ptr = vi(sz(q));                 int qi = 0, qe = lvl[s] = 1;                 while (qi &lt; qe &amp;&amp; !lvl[t]) {                     int v = q[qi++];                     for (Edge e : adj[v])                         if (!lvl[e.to] &amp;&amp; e.c &gt;&gt; (30 - L))                             q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;                 } while (lvl[p] = dfs(s, t, LLONG_MAX)) flow += p;             } while (lvl[t]); return flow;         } bool leftOfMinCut(int a) { return lvl[a] != 0; };     }</pre> | d7f0f1, 34 lines |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|

#### MinCut.h

**Description:** After running max-flow, the left side of a min-cut from  $s$  to  $t$  is given by all vertices reachable from  $s$ , only traversing edges with positive residual capacity.

### GlobalMinCut.h

**Description:** Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <b>Time:</b> $\mathcal{O}(V^3)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 8b0e19, 21 lines |
| <pre>pair&lt;int, vi&gt; globalMinCut(vector&lt;vi&gt; mat) {     pair&lt;int, vi&gt; best = {INT_MAX, {}};     int n = sz(mat);     vector&lt;vi&gt; co(n);     rep(i,0,n) co[i] = {i};     rep(ph,1,n) {         vi w = mat[0];         size_t s = 0, t = 0;         rep(it,0,n-ph) { <i>// O(V^2) -&gt; O(E log V) with prio. queue</i>             w[t] = INT_MIN;             s = t, t = max_element(all(w)) - w.begin();             rep(i,0,n) w[i] += mat[t][i];         }         best = min(best, {w[t] - mat[t][t], co[t]});         co[s].insert(co[s].end(), all(co[t]));         rep(i,0,n) mat[s][i] += mat[t][i];         rep(i,0,n) mat[i][s] = mat[s][i];         mat[0][t] = INT_MIN;     }</pre> |                  |

```
    }
    return best;
}
```

FlowDemands.h

**Description:** Flows with demands. ec1cd0d, 52 lines

```
//#include "flow_edmonds karp.h"
//include "flow_push_relabel.h" // if you need
//Flow with demands; time: O(maxflow)
struct FlowDemands {
    MaxFlow net;
    vector<vector<flow_t>> demands;
    flow_t total = 0;
    // Initialize for k vertices
    FlowDemands(int k = 0) : net(2) {
        while (k--) addVert();
    }
    // Add new vertex
    int addVert() {
        int v = net.addVert();
        demands.pb({});
        net.addEdge(0, v, 0);
        net.addEdge(v, 1, 0);
        return v-2;
    }
    // Add edge from u to v with demand dem
    // and capacity cap (dem <= flow <= cap).
    // Returns edge index in adjacency list of u.
    int addEdge(int u, int v,
                flow_t dem, flow_t cap) {
        demands[u].pb(dem);
        demands[v].pb(0);
        total += dem;
        net.G[0][v].cap += dem;
        net.G[u+2][1].cap += dem;
        return net.addEdge(u+2, v+2, cap-dem) - 2;
    }
    // Check if there exists a flow with value f
    // for source src and destination dst.
    // For circulation, you can set args to 0.
    bool canFlow(int src, int dst, flow_t f) {
        net.addEdge(dst + 2, src + 2, f);
        f = net.maxFlow(0, 1);
        net.G[src].pop_back();
        net.G[dst].pop_back();
        return f == total;
    }
    // Get flow through e-th edge of vertex v
    flow_t getFlow(int v, int e) {
        return net.getFlow(v+2,e+2)+demands[v][e];
    }
};
```

GomoryHu.h

**Description:** Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.

**Time:**  $\mathcal{O}(V)$  Flow Computations  
"PushRelabel.h" 0418b3, 13 lines

```
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
    vector<Edge> tree;
    vi par(N);
    rep(i,1,N) {
        PushRelabel D(N); // Dinic also works
        for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
        tree.push_back({i, par[i], D.calc(i, par[i])});
        rep(j,i+1,N)
            if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
    }
    return tree;
}
```

7.3 Flow with demands

Say we want  $d(e) \leq f(e) \leq c(e)$  for each edge. To find an arbitrary flow, add  $s', t'$  and the following edges:

- $\forall v \in V : c'((s', v)) = \sum_u d((u, v)), \quad c'((v, t')) = \sum_w d((v, w)),$
- $\forall (u, v) \in E : c'((u, v)) = c((u, v)) - d((u, v)),$
- $c'((t, s)) = \infty.$

For min flow, replace  $\infty$  with  $L$  and find smallest  $L$  such that flow is saturated.

7.4 Matching

hopcroftKarp.h

**Description:** Fast bipartite matching algorithm. Graph  $g$  should be a list of neighbors of the left partition, and  $btoa$  should be a vector full of -1's of the same size as the right partition. Returns the size of the matching.  $btoa[i]$  will be the match for vertex  $i$  on the right side, or  $-1$  if it's not matched.

**Usage:** vi btoa(m, -1); hopcroftKarp(g, btoa);

**Time:**  $\mathcal{O}(\sqrt{VE})$  56693c, 25 lines

```
bool dfs(int a, int L, vector<vi>& g, vi& btoa, vi& A, vi& B) {
    if (A[a] != L) return 0; A[a] = -1;
    for (int b : g[a]) if (B[b] == L + 1) { B[b] = 0;
        if (btoa[b] == -1 || dfs(btoa[b], L + 1, g, btoa, A, B))
            return btoa[b] = a, 1;
    } return 0; }
int hopcroftKarp(vector<vi>& g, vi& btoa) {
    int res = 0;
    vi A(g.size()), B(btoa.size()), cur, next;
    for (;;) {
        fill(all(A), 0); fill(all(B), 0); cur.clear();
        for (int a : btoa) if (a != -1) A[a] = -1;
        rep(a, 0, sz(g)) if (A[a] == 0) cur.push_back(a);
        for (int lay = 1; ; lay++) {
            bool islast = 0;
            next.clear();
            for (int a : cur) for (int b : g[a]) {
                if (btoa[b] == -1) B[b] = lay, islast = 1;
                else if (btoa[b] != a && !B[b])
                    B[b] = lay, next.push_back(btoa[b]);
            } if (islast) break;
            if (next.empty()) return res;
            for (int a : next) A[a] = lay; cur.swap(next);
        } rep(a, 0, sz(g)) res += dfs(a, 0, g, btoa, A, B);
    }
```

DFSMatching.h

**Description:** Simple bipartite matching algorithm. Graph  $g$  should be a list of neighbors of the left partition, and  $btoa$  should be a vector full of -1's of the same size as the right partition. Returns the size of the matching.  $btoa[i]$  will be the match for vertex  $i$  on the right side, or  $-1$  if it's not matched.

**Usage:** vi btoa(m, -1); dfsMatching(g, btoa);

**Time:**  $\mathcal{O}(VE)$  522b98, 22 lines

```
bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {
    if (btoa[j] == -1) return 1;
    vis[j] = 1; int di = btoa[j];
    for (int e : g[di])
        if (!vis[e] && find(e, g, btoa, vis)) {
            btoa[e] = di;
            return 1;
        }
    return 0;
}
int dfsMatching(vector<vi>& g, vi& btoa) {
    vi vis;
    rep(i, 0, sz(g)) {
        vis.assign(sz(btoa), 0);
        for (int j : g[i])
            if (find(j, g, btoa, vis)) {
                btoa[j] = i;
                break;
            }
    }
    return sz(btoa) - (int)count(all(btoa), -1);
}
```

MinimumVertexCover.h

**Description:** Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

"DFSMatching.h" da4196, 20 lines

```
vi cover(vector<vi>& g, int n, int m) {
    vi match(m, -1);
    int res = dfsMatching(g, match);
    vector<bool> lfound(n, true), seen(m);
    for (int it : match) if (it != -1) lfound[it] = false;
    vi q, cover;
    rep(i, 0, n) if (lfound[i]) q.push_back(i);
    while (!q.empty()) {
        int i = q.back(); q.pop_back();
        lfound[i] = 1;
        for (int e : g[i]) if (!seen[e] && match[e] != -1) {
            seen[e] = true;
            q.push_back(match[e]);
        }
    }
```

```
    }
}
rep(i, 0, n) if (!lfound[i]) cover.push_back(i);
rep(i, 0, m) if (seen[i]) cover.push_back(n+i);
assert(sz(cover) == res);
return cover;
}
```

WeightedMatching.h

**Description:** Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes  $\text{cost}[N][M]$ , where  $\text{cost}[i][j] = \text{cost}$  for  $L[i]$  to be matched with  $R[j]$  and returns (min cost, match), where  $L[i]$  is matched with  $R[\text{match}[i]]$ . Negate costs for max cost. Requires  $N \leq M$ .

**Time:**  $\mathcal{O}(N^2M)$  1e0fe9, 29 lines

```
pair<int, vi> hungarian(const vector<vi> &a) {
    if (a.empty()) return {0, {}};
    int n = sz(a) + 1, m = sz(a[0]) + 1;
    vi u(n), v(m), p(m), ans(n - 1);
    rep(i, 1, n) {
        p[0] = i;
        int j0 = 0; // add "dummy" worker 0
        vi dist(m, INT_MAX), pre(m, -1);
        vector<bool> done(m + 1);
        do { // dijkstra
            done[j0] = true;
            int i0 = p[j0], j1, delta = INT_MAX;
            rep(j, 1, m) if (!done[j]) {
                auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
                if (dist[j] < delta) delta = dist[j], j1 = j;
            }
            rep(j, 0, m) {
                if (done[j]) u[p[j]] += delta, v[j] -= delta;
                else dist[j] -= delta;
            }
            j0 = j1;
        } while (p[j0]);
        while (j0) { // update alternating path
            int j1 = pre[j0];
            p[j0] = p[j1], j0 = j1;
        } rep(j, 1, m) if (p[j]) ans[p[j] - 1] = j - 1;
    } return {-v[0], ans}; // min cost
}
```

GeneralMatching.h

**Description:** Unweighted matching for general graphs. If  $\text{white}[v] = 0$  at the end,  $v$  is part of every max matching.

**Time:**  $\mathcal{O}(NM)$ , faster in practice 28552a, 40 lines

```
struct MaxMatching { // 1-indexed
    vector<vi> G; int n;
    vi mate, par, white; vector<pii> l;
    MaxMatching(vector<vi> _G : G(_G), n(SZ(G)), mate(n),
                par(n), white(n), l(n) {}) {
        int group(int x) {
            return par[x] = (white[par[x]] ? group(par[x]) : par[x]);
        } void match(int p, int b) {
            swap(mate[p], b); if (mate[b] != p) return;
            if (!l[p].nd) mate[b] = l[p].st, match(l[p].st, b);
            else match(l[p].st, l[p].nd), match(l[p].nd, l[p].st);
        } bool augment(int a) {
            white[a] = 1; par[a] = 0; l[a] = {0, 0};
            queue<int> q; q.push(a);
            while (!q.empty()) {
                a = q.front(); q.pop();
                for (int b : G[a]) {
                    if (white[b]) {
                        int x = group(a), y = group(b), lca = 0;
                        while (x || y) {
                            if (y) swap(x, y);
                            if (l[x] == mp(a, b)) { lca = x; break; }
                            l[x] = {a, b}; x = group(l[mate[x]].st);
                        } for (int v : {group(a), group(b)}) while (v != lca) {
                            q.push(v); white[v] = 1; par[v] = lca;
                            v = group(l[mate[v]].st);
                        }
                    } else if (!mate[b]) {
                        mate[b] = a; match(a, b); fill(all(white), 0);
                        return 1;
                    } else if (!white[mate[b]]) {
                        white[mate[b]] = 1; par[mate[b]] = b;
                        l[b] = {0, 0}; l[mate[b]] = {a, 0};
                        q.push(mate[b]);
                    }
                }
            } return 0;
        } int max_matching() {
```

```
int ans = 0;
FOR(v, 1, n) if(!mate[v]) ans += augment(v);
return ans;
});
```

GeneralWeightedMatching.h  
Description: Weighted matching for general graphs.  
Time:  $\mathcal{O}(N^3)$ , fast in practice

8ba4a8, 128 lines

```
namespace WeightedBlossom { // 1-indexed
#define d(x) (lab[x.u] + lab[x.v] - e[x.u][x.v].w * 2)
const int N = 501 * 2;
const ll INF = 1e18;
struct Q { int u, v; ll w; } e[N][N]; vi p[N];
int n, m = 0, id, h, t, lk[N], sl[N], sk[N], f[N], b[N][N];
int s[N], ed[N], q[N]; ll lab[N];
void upd(int u, int v) {
    if(!sl[v] || d(e[u][v]) < d(e[sl[v]][v])) sl[v] = u;
}
void ss(int v) {
    sl[v] = 0;
    rep(u, 1, n)
        if(e[u][v].w > 0 && sk[u] != v && !s[sk[u]]) upd(u, v);
}
void ins(int u) {
    if(u <= n) q[++t] = u;
    else for(auto v: p[u]) ins(v);
}
void mdf(int u, int w) {
    sk[u] = w;
    if(u > n) for(auto v: p[u]) mdf(v, w);
}
int gr(int u, int v) {
    Q w = e[u][v]; int x = b[u][w.u], y = gr(u, x);
    FOR(i, 0, y) stm(p[u][i], p[u][i ^ 1]);
    stm(x, v);
    rotate(p[u].begin(), p[u].begin() + y, p[u].end());
}
void aug(int u, int v) {
    int w = sk[lk[u]]; stm(u, v); if(!w) return;
    stm(w, sk[f[w]]); aug(sk[f[w]], w);
}
int lca(int u, int v) {
    for(++id; u|v; swap(u, v)){
        if(!u) continue;
        if(ed[u] == id) return u;
        ed[u] = id; if(u = sk[lk[u]]) u = sk[f[u]]; // not ==
    }
    return 0;
}
void add(int uu, int a, int vv) {
    int x = n + 1; while(x <= m && sk[x]) x++;
    if(x > m) m++;
    lab[x] = s[x] = sk[x] = 0; lk[x] = lk[a];
    p[x].clear(); p[x].pb(a);
    for(auto i = uu, j = 0; i != a; i = sk[f[j]])
        p[x].pb(i), p[x].pb(j = sk[lk[i]]), ins(j);
    reverse(1 + all(p[x]));
    for(auto i = vv, j = 0; i != a; i = sk[f[j]])
        p[x].pb(i), p[x].pb(j = sk[lk[i]]), ins(j);
    mdf(x, x); rep(i, 1, m) e[x][i].w = e[i][x].w = 0;
    memset(b[x] + 1, 0, n * sizeof b[0][0]);
    for(auto u: p[x]){
        rep(v, 1, m) if(!e[x][v].w || d(e[u][v]) < d(e[x][v]))
            e[x][v] = e[u][v], e[v][x] = e[v][u];
        rep(v, 1, n) if(b[u][v]) b[x][v] = u;
    } ss(x);
}
void ex(int u) { // s[u] == 1
    for(auto x: p[u]) mdf(x, x);
    int a = b[u][e[u][f[u]].u], r = gr(u, a);
    FOR(i, 0, r) {
        int x = p[u][i], y = p[u][i + 1];
        f[x] = e[y][x].u; s[x] = 1; s[y] = sl[x] = 0;
        ss(y); ins(y); i++;
    } s[a] = 1; f[a] = f[u];
    FOR(i, r + 1, SZ(p[u])) s[p[u][i]] = -1, ss(p[u][i]);
    sk[u] = 0;
}
bool on(const Q &e) {
    int u = sk[ee.u], v = sk[ee.v], a;
    if(s[v] == -1) {
        f[v] = ee.u, s[v] = 1, a = sk[lk[v]];
        sl[v] = sl[a] = s[a] = 0, ins(a);
    } else if(!s[v]){
        a = lca(u, v);
        if(!a) return aug(u, v), aug(v, u), 1;
        else add(u, a, v);
    }
    return 0;
}
bool bfs() {
    memset(s + 1, -1, m * sizeof s[0]);
    memset(sl + 1, 0, m * sizeof sl[0]);
```

```
h = 1; t = 0;
rep(i, 1, m)
    if(sk[i] == i && !lk[i]) f[i] = s[i] = 0, ins(i);
if(h > t) return 0;
while(1) {
    while(h <= t) {
        int u = q[h++];
        if(s[sk[u]] != 1) rep(v, 1, n) {
            if(e[u][v].w > 0 && sk[u] != sk[v]) {
                if(d(e[u][v])) upd(u, sk[v]);
                else if(on(e[u][v])) return 1;
            }
        }
        ll x = INF;
        rep(i, n + 1, m) if(sk[i] == i && s[i] == 1)
            x = min(x, lab[i] >> 1);
        rep(i, 1, m) if(sk[i] == i && sl[i] && s[i] != 1)
            x = min(x, d(e[sl[i]][i]) >> (s[i] + 1));
        rep(i, 1, n) if(~s[sk[i]])
            if((lab[i] += (s[sk[i]] * 2 - 1) * x) <= 0) return 0;
        rep(i, n + 1, m) if(sk[i] == i && ~s[sk[i]])
            lab[i] += (2 - s[sk[i]] * 4) * x;
        h = 1; t = 0;
        rep(i, 1, m) if(sk[i] == i && sl[i] && sk[sl[i]] != i &&
            [d(e[sl[i]][i]) && on(e[sl[i]][i])) return 1;
        rep(i, n + 1, m) if(sk[i] == i && s[i] == 1 && !lab[i])
            ex(i);
    }
    return 0;
}
pair<ll, vector<pii>> run
(int _n, vector<tuple<int, int, ll>> edges) {
    memset(ed + 1, 0, m * sizeof ed[0]);
    memset(lk + 1, 0, m * sizeof lk[0]);
    n = m = _n; id = 0; iota(sk + 1, sk + n + 1, 1);
    ll wm = 0, weight = 0;
    rep(i, 1, n) rep(j, 1, n) e[i][j] = {i, j, 0};
    for(auto [u, v, w]: edges)
        wm = max(wm, e[v][u].w = e[u][v].w = max(e[u][v].w, w));
    rep(i, 1, n) p[i].clear();
    rep(i, 1, n) rep(j, 1, n) b[i][j] = i * (i == j);
    fill_n(lab + 1, n, wm); while(bfs());
    vector<pii> matching;
    rep(i, 1, n) if(i < lk[i])
        weight += e[i][lk[i]].w, matching.pb({i, lk[i]});
    return {weight, matching};
}
}
#undef d
}
```

MatroidIntersection.h  
Description: Find largest subset S of [n] such that S is independent in both matroid A and B, given by their oracles, see example implementations below. Returns vector V such that V[i] = 1 iff i-th element is included in found set. Time:  $\mathcal{O}(r^2 \cdot (init + n \cdot add))$ , where r is max independent sets.

8ed3b, 152 lines

```
template<class T, class U>
vector<bool> intersectMatroids(T& A, U& B, int n) {
    vector<bool> ans(n);
    bool ok = 1;
    // NOTE: for weighted matroid intersection find
    // shortest augmenting paths first by weight change,
    // then by length using Bellman-Ford,
    // Speedup trick (only for unweighted):
    A.init(ans); B.init(ans);
    rep(i, n)
        if (A.canAdd(i) && B.canAdd(i))
            ans[i] = 1, A.init(ans), B.init(ans);
    //End of speedup
    while (ok) {
        vector<vi> G(n);
        vector<bool> good(n);
        queue<int> que;
        vi prev(n, -1);
        A.init(ans); B.init(ans); ok = 0;
        rep(i, n) if (!ans[i]) {
            if (A.canAdd(i)) que.push(i), prev[i] = -2;
            good[i] = B.canAdd(i);
        }
        rep(i, n) if (ans[i]) {
            ans[i] = 0;
            A.init(ans); B.init(ans);
            rep(j, n) if (i != j && !ans[j]) {
                if (A.canAdd(j)) G[i].pb(j); // -cost[j]
                if (B.canAdd(j)) G[j].pb(i); // cost[i]
            }
            ans[i] = 1;
        }
        while (!que.empty()) {
            int i = que.front();
            que.pop();
            if (good[i]) { // best found (unweighted = shortest path)
```

```
ans[i] = 1;
while (prev[i] >= 0) { // alternate matching
    ans[i = prev[i]] = 0;
    ans[i = prev[i]] = 1;
}
ok = 1; break;
}
for(auto j: G[i]) if (prev[j] == -1)
    que.push(j), prev[j] = i;
}
}
return ans;
}
// Matroid where each element has color
// and set is independent iff for each color c
// #elements of color c<= maxAllowed[c].
struct LimOracle {
    vi color; // color[i] = color of i-th element
    vi maxAllowed; // Limits for colors
    vi tmp;
    // Init oracle for independent set S; O(n)
    void init(vector<bool>& S) {
        tmp = maxAllowed;
        rep(i, sz(S)) tmp[color[i]] -= S[i];
    }
    // Check if S+{k} is independent; time: O(1)
    bool canAdd(int k) { return tmp[color[k]] > 0; }
};
// Graphic matroid - each element is edge,
// set is independent iff subgraph is acyclic.
struct GraphOracle {
    vector<pii> elems; // Ground set: graph edges
    int n; // Number of vertices, indexed [0;n-1]
    vi par;
    int find(int i) {
        return par[i] == -1 ? i : par[i] = find(par[i]);
    }
    // Init oracle for independent set S; ~O(n)
    void init(vector<bool>& S) {
        par.assign(n, -1);
        rep(i, sz(S)) if (S[i])
            par[find(elems[i].st)] = find(elems[i].nd);
    }
    // Check if S+{k} is independent; time: ~O(1)
    bool canAdd(int k) {
        return find(elems[k].st) != find(elems[k].nd);
    }
};
// Co-graphic matroid - each element is edge,
// set is independent iff after removing edges
// from graph number of connected components
// doesn't change.
struct CographOracle {
    vector<pii> elems; // Ground set: graph edges
    int n; // Number of vertices, indexed [0;n-1]
    vector<vi> G;
    vi pre, low;
    int cnt;
    int dfs(int v, int p) {
        pre[v] = low[v] = ++cnt;
        for(auto e: G[v]) if (e != p)
            low[v] = min(low[v], pre[e] ? dfs(e, v));
        return low[v];
    }
    // Init oracle for independent set S; O(n)
    void init(vector<bool>& S) {
        G.assign(n, {});
        pre.assign(n, 0);
        low.resize(n);
        cnt = 0;
        rep(i, sz(S)) if (!S[i]) {
            pii e = elems[i];
            G[e.st].pb(e.nd);
            G[e.nd].pb(e.st);
        }
        rep(v, n) if (!pre[v]) dfs(v, -1);
    }
    // Check if S+{k} is independent; time: O(1)
    bool canAdd(int k) {
        pii e = elems[k];
        return max(pre[e.st], pre[e.nd]) != max(low[e.st], low[e.nd]);
    }
};
// Matroid equivalent to linear space with XOR
struct XorOracle {
    vector<ll> elems; // Ground set: numbers
    vector<ll> base;
    // Init for independent set S; O(n+r^2)
    void init(vector<bool>& S) {
        base.assign(63, 0);
        rep(i, sz(S)) if (S[i]) {
```

```

    ll e = elems[i];
    rep(j, sz(base)) if ((e >> j) & 1) {
        if (!base[j]) {
            base[j] = e;
            break;
        }
        e ^= base[j];
    }
}
// Check if S+{k} is independent; time: O(r)
bool canAdd(int k) {
    ll e = elems[k];
    rep(i, sz(base)) if ((e >> i) & 1) {
        if (!base[i]) return 1;
        e ^= base[i];
    }
    return 0;
}
};
```

7.5 DFS algorithms

kShortestWalks.h

**Description:** Given a non-negative weighted directed graph, computes lengths of K shortest walks (vertices can repeat). For graphs with negative weights, try your luck or change Dijkstra to SPFA.

**Time:**  $\mathcal{O}((M+K)\log N)$

```

5a47b8, 58 lines
struct PersistentHeap {
    struct Node {
        int id, l = -1, r = -1; ll cost;
        Node(int _id, ll _cost) : id(_id), cost(_cost) {}
    };
    vector<Node> ds;
    int add(int id, ll cost) {
        ds.pb(Node(id, cost)); return SZ(ds) - 1;
    }
    int ins(int v, int u) {
        if(v == -1) return u;
        ds.pb(ds[v]); v = SZ(ds) - 1; swap(ds[v].l, ds[v].r);
        if(ds[v].cost > ds[u].cost) {
            swap(ds[v].cost, ds[u].cost), swap(ds[v].id, ds[u].id);
        }
        ds[v].r = ins(ds[v].r, u); return v;
    }
    void insert(int &v, int u) { v = ins(v, u); }
};
vector<ll> kWalk(vector<vector<pii>> G, int s, int t, int k) {
    int n = SZ(G); vector<vector<pii>> GR(n);
    FOR(v, 0, n) for(auto &[u, d]: G[v]) GR[u].pb({v, d});
    const ll INF = 1e18; vector<ll> dist(n, INF); vi par(n, -1);
    using T = pair<ll, int>; dist[t] = 0;
    priority_queue<T, vector<T>, greater<T>> q; q.push({0, t});
    while(!q.empty()) {
        auto [dv, v] = q.top(); q.pop();
        if(dv != dist[v]) continue;
        for(auto &[u, d]: GR[v]) if(dv + d < dist[u]) {
            par[u] = v; dist[u] = dv + d; q.push({dist[u], u});
        }
    }
    vector<vi> tree(n);
    FOR(v, 0, n) if(~par[v]) tree[par[v]].pb(v);
    PersistentHeap heap; vi head(n, -1);
    function<void(int)> dfs = [&](int v) {
        bool skip = 0;
        for(auto &[u, d]: G[v]) if(dist[u] != INF) {
            if(dist[v] == dist[u] + d && par[v] == u && !skip)
                skip = 1;
            else
                heap.insert(head[v], heap.add(u, dist[u]-dist[v]+d));
        }
        for(int u: tree[v]) head[u] = head[v], dfs(u);
    }; dfs(t);
    vector<ll> ans(k, -1); q.push({dist[s], heap.add(s, 0)});
    FOR(i, 0, k) {
        if(q.empty() || dist[s] == INF) break;
        auto [dv, v] = q.top(); q.pop();
        ans[i] = dv; auto &node = heap.ds[v]; ll diff = 0;
        for(int u: {head[node.id], node.l, node.r}) {
            if(~u) q.push({dv + heap.ds[u].cost - diff, u});
            diff = node.cost;
        }
    }
    return ans;
}
```

SCC.h

**Description:** SCC's  
**Time:**  $\mathcal{O}(E+V)$

```

b71891, 10 lines
vector<int> ord; int comp, id[N], vis[N];
void dfs(int v){ vis[v] = 1;
    for(auto u : adj[v]) if(!vis[u]) dfs(u); ord.push_back(v);
} void dfs2(int v){ vis[v] = 1; id[v] = comp;
    for(auto u : adjr[v]) if(!vis[u]) dfs2(u);
} void init(){
    fill(vis, vis+n, 0);
    rep(i, 0, n) if(!vis[i]) dfs(i);
    fill(vis, vis+n, 0); rev(all(ord));
    for(auto v : ord) if(!vis[v]) dfs2(v), comp++;
}
```

BBC.h

**Description:** Finds all biconnected components in an undirected graph. In a biconnected component there are at least two distinct paths between any two nodes. Note that a node can be in several components. An edge which is not in a component is a bridge i.e., not part of any cycle. bls contains vertices of bbcs, eds edges of bbcs, bl[i] = max id of bls vertex i belongs to.

**Usage:** for each edge (a,b) {  
adj[a].emplace\_back(b, eid);  
adj[b].emplace\_back(a, eid++); }  
rep(i, 0, n) if(!tin[i]) {tim=1;st.clear();dfs(i,-1);}  
**Time:**  $\mathcal{O}(E+V)$

```

58520e, 12 lines
int tim=1, tin[N], low[N], art[N]; vi st;
vector<pii> ste, eds[N]; vector<vi> bl;
void dfs(int v, int p){ st.pb(v); tin[v] = low[v] = tim++;
    if(!adj[v].size()) bl.pb({v}), st.pop_back();
    for(auto& [u, e] : adj[v]){ if(e == p) continue; // if(u == p)
        if(tin[u] < tin[v]) ste.pb(v,u);
        if(tin[u] low[v] = min(low[v], tin[u]);
        eds(dfs(u, e); low[v] = min(low[v], low[u]);
        if(low[u] >= tin[v]){ art[v] = (tin[v] > 1 || tin[u] > 2);
            do(eds[sz(bl)].pb(ste.back()); ste.pop_back(););
            while(eds[sz(bl)].back() != mp(v,u)); bl.pb({v});
            while(bl.back().back() != u){bl.back().pb(ste.back()); st.pop_back
                ();}
        }
    }
}
```

2sat.h

**Description:** Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type (a||b)&&(!a||c)&&(d||!b)&&... becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).

**Usage:** TwoSat ts(number of boolean variables);  
ts.either(0, ~3); // Var 0 is true or var 3 is false  
ts.setValue(2); // Var 2 is true  
ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true  
ts.solve(); // Returns true iff it is solvable  
ts.values[0..N-1] holds the assigned values to the vars  
**Time:**  $\mathcal{O}(N+E)$ , where N is the number of boolean variables, and E is the number of clauses.

```

7897d8, 33 lines
struct TwoSat {
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true
    TwoSat(int n = 0) : N(n), gr(2*n) {}
    void either(int f, int j) {
        f = max(2*f, -1-2*f); j = max(2*j, -1-2*j);
        gr[f].push_back(j^1); gr[j].push_back(f^1);
    } int addVar() { // (optional)
        gr.emplace_back(); gr.emplace_back();
        return N++;
    } void setValue(int x) { either(x, x); }
    void atMostOne(const vi& li) {
        if (sz(li) <= 1) return;
        int cur = ~li[0];
        rep(i, 2, sz(li)) { int nxt = addVar();
            either(cur, ~li[i]); either(cur, nxt);
            either(~li[i], nxt); cur = ~nxt;
        } // (optional end)
    }
    vi val, comp, z; int time = 0;
    int dfs(int i) {
        int low = val[i] = ++time, x; z.push_back(i);
        for(int e : gr[i]) if (!comp[e]) low = min(low, val[e] ? : dfs(e));
        if (low == val[i]) do {
            x = z.back(); z.pop_back(); comp[x] = low;
            if (values[x]>1 == -1) values[x]>1 = x&1;
        } while (x != i); return val[i] = low;
    }
}
```

```

} bool solve() {
    values.assign(N, -1); val.assign(2*N, 0); comp = val;
    rep(i, 0, 2*N) if (!comp[i]) dfs(i);
    rep(i, 0, N) if (comp[2*i] == comp[2*i+1]) return 0;
    return 1; };
```

Dominators.h

**Description:** Tarjan's dominators in directed graph Returns tree (as array of parents) of immediate dominators idom. idom[root] = root, idom[v] = -1 if v is unreachable from root

**Time:**  $\mathcal{O}(E\log V)$

```

2613e6, 29 lines
vi dominators(vector<vi>& G, int root) {
    int n = sz(G); vector<vi> in(n), bucket(n);
    vi pre(n, -1), anc(n, -1), par(n), best(n);
    vi ord, idom(n, -1), sdom(n, -1), rdom(n);
    auto dfs = [&](auto f, int v, int p) -> void {
        if (pre[v] == -1) {
            par[v] = p; pre[v] = sz(ord);
            ord.pb(v);
            for (auto e : G[v])
                in[e].pb(v), f(f, e, v);
        }
    }; auto find = [&](auto f, int v) -> pii {
        if (anc[v] == -1) return {best[v], v};
        int b; tie(b, anc[v]) = f(f, anc[v]);
        if (sdom[b] < sdom[best[v]]) best[v] = b;
        return {best[v], anc[v]};
    }; rdom[root] = idom[root] = root;
    iota(all(best), 0); dfs(dfs, root, -1);
    rep(i, sz(ord)) {
        int v = ord[sz(ord)-i-1], b = pre[v];
        for (auto e : in[v])
            b = min(b, pre[e] < pre[v] ? pre[e] :
                sdom[find(find, e).st]);
        for (auto u : bucket[v]) rdom[u]=find(find,u).st;
        sdom[v] = b; anc[v] = par[v];
        bucket[ord[sdom[v]]].pb(v);
    } for (auto v : ord) idom[v] = {rdom[v] == v ?
        ord[sdom[v]] : idom[rdom[v]]};
    return idom; }
```

KthShortest.h

**Description:** Given directed weighted graph with non-negative edge weights gets K-th shortest walk (not necessarily simple) in  $\mathcal{O}(\log|E|)$ . -1 if no next path (can only happen in DAG). WARNING: USES KLOGM memory and persistent heaps!

```

2d9393, 57 lines
constexpr ll INF = 1e18;
struct Eppstein {
    using T = ll; using Edge = pair<int, T>;
    struct Node { int E[2] = {};, s = 0; Edge x; };
    T shortest; // Shortest path length
    priority_queue<pair<T, int>> Q;
    vector<Node> P[1]; vi h;
    Eppstein(vector<vector<Edge>>& G, int s, int t) {
        int n = sz(G); vector<vector<Edge>> H(n);
        rep(i, n) for(auto &e : G[i])
            H[e.st].pb({i, e.nd});
        vi ord, par(n, -1); vector<T> d(n, -INF);
        Q.push({d[t] = 0, t});
        while (!Q.empty()) {
            auto v = Q.top(); Q.pop();
            if (d[v.nd] == v.st) {
                ord.pb(v.nd);
                for(auto &e : H[v.nd])
                    if (v.st-e.nd > d[e.st]) {
                        Q.push({d[e.st] = v.st-e.nd, e.st});
                        par[e.st] = v.nd;
                    }
            }
        }
        if ((shortest = -d[s]) >= INF) return;
        h.resize(n);
        for(auto &v : ord) {
            int p = par[v]; if (p+1) h[v] = h[p];
            for(auto &e : G[v]) if (d[e.st] > -INF) {
                T k = e.nd - d[e.st] + d[v];
                if (k || e.st != p)
                    h[v] = push(h[v], {e.st, k});
                else p = -1;
            }
        }
        P[0].x.st = s; Q.push({0, 0});
    }
    int push(int t, Edge x) {
        P.pb(P[t]);
        if (!P[t = sz(P)-1].s || P[t].x.nd >= x.nd)
    }
}
```



```
        swap(x, P[t].x);
    if (P[t].s) {
        int i = P[t].E[0], j = P[t].E[1];
        int d = P[i].s > P[j].s;
        int k = push(d ? j : i, x);
        P[t].E[d] = k; // Don't inline k!
    }
    P[t].s++; return t;
}
11 nextPath() { // next length, -1 if no next path
    if (Q.empty()) return -1;
    auto v = Q.top(); Q.pop();
    for (int i : P[v.nd].E) if (i)
        Q.push({ v.st-P[i].x.nd+P[v.nd].x.nd, i });
    int t = h[P[v.nd].x.st];
    if (t) Q.push({v.st - P[t].x.nd, t });
    return shortest - v.st; }
};

PlanarFaces.h
Description: Read desc below. a391b4, 102 lines

/*
 * complexity mlogm, assumes that you are given an embedding
 * graph is drawn straightline non-intersecting
 * returns combinatorial embedding (inner face vertices clockwise, outer
   counter clockwise).
 * WAZNE czasem trzeba zlaczyz wszystkie sciany zewnetrzne (chodzi o kmine
   do konkretnego zadania)
 * (ktorych moze byc kilka, gdy jest wiele spojnych) w jedna sciane.
 * Zewnetrzne sciany moga wygladac jak kaktusy, a wewnetrzne zawsze sa
   niezdegenerowanym wielokatem.
 */
struct Edge {
    int e, from, to;
    // face is on the right of "from -> to"
};
ostream& operator<<(ostream &o, Edge e) {
    return o << vector{e.e, e.from, e.to};
}
struct Face {
    bool is_outside;
    vector<Edge> sorted_edges;
    // edges are sorted clockwise for inside and cc for outside faces
};
ostream& operator<<(ostream &o, Face f) {
    return o << pair(f.is_outside, f.sorted_edges);
}
vector<Face> split_planar_to_faces(vector<pii> coord, vector<pii> edges) {
    int n = sz(coord);
    int E = sz(edges);
    vector<vi> graph(n);
    rep(e, E) {
        auto [v, u] = edges[e];
        graph[v].eb(e);
        graph[u].eb(e);
    }
    vi lead(2 * E);
    iota(lead.begin(), lead.end(), 0);
    function<int (int)> find = [&](int v) {
        return lead[v] == v ? v : lead[v] = find(lead[v]);
    };
    auto side_of_edge = [&](int e, int v, bool outward) {
        return 2 * e + ((v != min(edges[e].first, edges[e].second)) ^ outward)
            ;
    };
    rep(v, n) {
        vector<pair<pii, int>> sorted;
        for(int e : graph[v]) {
            auto p = coord[edges[e].first ^ edges[e].second ^ v];
            auto center = coord[v];
            sorted.eb(pair(p.first - center.first, p.second - center.second), e)
                ;
        }
        sort(all(sorted), [&](pair<pii, int> l0, pair<pii, int> r0) {
            auto l = l0.first;
            auto r = r0.first;
            bool half_l = l > pair(0, 0);
            bool half_r = r > pair(0, 0);
            if(half_l != half_r)
                return half_l;
            return l.first * LL(r.second) - l.second * LL(r.first) > 0;
        });
        rep(i, sz(sorted)) {
            int e0 = sorted[i].second;
            int e1 = sorted[(i + 1) % sz(sorted)].second;
            int side_e0 = side_of_edge(e0, v, true);
            int side_e1 = side_of_edge(e1, v, false);
            lead[find(side_e0)] = find(side_e1);
        }
    }
}
```

## PlanarFaces PlanarityCheck Meduze EdgeColoring

```
vector<vi> comps(2 * E);
rep(i, 2 * E)
    comps[find(i)].eb(i);
vector<Face> polygons;
vector<vector<pii>> outgoing_for_face(n);
rep(leader, 2 * E)
    if(sz(comps[leader])) {
        for(int id : comps[leader]) {
            int v = edges[id / 2].first;
            int u = edges[id / 2].second;
            if(v > u)
                swap(v, u);
            if(id % 2 == 1)
                swap(v, u);
            outgoing_for_face[v].eb(u, id / 2);
        }
        vector<Edge> sorted_edges;
        function<void (int)> dfs = [&](int v) {
            while(sz(outgoing_for_face[v])) {
                auto [u, e] = outgoing_for_face[v].back();
                outgoing_for_face[v].pop_back();
                dfs(u);
                sorted_edges.eb(e, v, u);
            }
        };
        dfs(edges[comps[leader].front() / 2].first);
        reverse(all(sorted_edges));
        LL area = 0;
        for(auto edge : sorted_edges) {
            auto l = coord[edge.from];
            auto r = coord[edge.to];
            area += l.first * LL(r.second) - l.second * LL(r.first);
        }
        polygons.eb(area >= 0, sorted_edges);
    }
// Remember that there can be multiple outside faces.
return polygons;
}
```

## PlanarityCheck.h

```
Description: Read desc below. cc4508, 93 lines

/*
 * Opis: O(szybko) ale istnieja przyklady O(n2), przyjmuje graf
   nieskierowany bez petelek i multikrawedzi.
 */
bool is_planar(vector<vi> g) {
    int n = sz(g), m = 0;
    rep(v, n) m += sz(g[v]);
    m /= 2;
    if(n <= 3) return true;
    if(m > 3 * n - 6) return false;
    vector<vi> up(n), dn(n);
    vi low(n, -1), pre(n);
    rep(start, n)
        if(low[start] == -1) {
            vector<pii> e_up;
            int tm = 0;
            function<void (int, int)> dfs_low = [&](int v, int p) {
                low[v] = pre[v] = tm++;
                for(int u : g[v])
                    if(u != p and low[u] == -1) {
                        dn[v].eb(u);
                        dfs_low(u, v);
                        low[v] = min(low[v], low[u]);
                    }
                else if(u != p and pre[u] < pre[v]) {
                    up[v].eb(ssize(e_up));
                    e_up.eb(v, u);
                    low[v] = min(low[v], pre[u]);
                }
            };
            dfs_low(start, -1);
            vector<pair<int, bool>> dsu(sz(e_up));
            rep(v, sz(dsu)) dsu[v].first = v;
            function<pair<int, bool> (int)> find = [&](int v) {
                if(dsu[v].first == v)
                    return pair(v, false);
                auto [u, ub] = find(dsu[v].first);
                return dsu[v] = pair(u, ub ^ dsu[v].second);
            };
            auto onion = [&](int x, int y, bool flip) {
                auto [v, vb] = find(x);
                auto [u, ub] = find(y);
                if(v == u)
                    return not (vb ^ ub ^ flip);
                dsu[v] = {u, vb ^ ub ^ flip};
                return true;
            };
            auto interlace = [&](const vi &ids, int lo) {
                vi ans;
```

```
        for(int e : ids)
            if(pre[e_up[e].second] > lo)
                ans.eb(e);
        return ans;
    };
    auto add_fu = [&](const vi &a, const vi &b) {
        fwd(k, 1, sz(a))
            if(not onion(a[k - 1], a[k], 0))
                return false;
        fwd(k, 1, sz(b))
            if(not onion(b[k - 1], b[k], 0))
                return false;
        return a.empty() or b.empty() or onion(a[0], b[0], 1);
    };
    function<bool (int, int)> dfs_planar = [&](int v, int p) {
        for(int u : dn[v])
            if(not dfs_planar(u, v))
                return false;
        rep(i, sz(dn[v])) {
            fwd(j, i + 1, sz(dn[v]))
                if(not add_fu(interlace(up[dn[v][i]], low[dn[v][j]]),
                    interlace(up[dn[v][j]], low[dn[v][i]])))
                    return false;
        }
        for(int j : up[v]) {
            if(e_up[j].first != v)
                continue;
            if(not add_fu(interlace(up[dn[v][i]], pre[e_up[j].second]),
                interlace({j}, low[dn[v][i]])))
                return false;
        }
        for(int u : dn[v]) {
            for(int idx : up[u])
                if(pre[e_up[idx].second] < pre[p])
                    up[v].eb(idx);
                    exchange(up[u], {});
        }
        return true;
    };
    if(not dfs_planar(start, -1))
        return false;
    }
    return true;
}
```

## Meduze.h

**Description:** Utility for functional graphs. Computes following tables: cs - vertices of ith comp's cycle, ci - index of comp, ti - index of tree in comp, idx - same as ti but -1 if not on cycle (i.e. not root), cl - length of ith component's cycle

**Time:**  $\mathcal{O}(V)$  or  $\mathcal{O}(V \log V)$  if lca 955d6b, 22 lines

```
vector<int> adjr[N], cs[N];
int vis[N], cn, tn, ci[N], id[N], ti[N], cl[N];
// int h[N], tin[N], tout[N], tim, up[N][K]; // lca
void dfs(int v, int p){
    ci[v] = cn; ti[v] = tn;
    // if(v != adj[p]) h[v] = h[p]+1; tin[v] = tim++; // lca
    // up[v][0] = (v == adj[p] ? v : p); // lca
    // for(int k=1;k<K;k++) up[v][k] = up[up[v][k-1]][k-1]; // lca
    for(auto u : adjr[v]){
        if(u == p) continue;
        dfs(u, v);
    } // tout[v] = tim++; // lca
}
void init(){
    fill(ci,ci+n,-1); fill(id,id+n,-1);
    rep(i,0,n) adjr[adj[i]].push_back(i);
    rep(i,0,n) if(!vis[i]) {
        int v = i;
        while(!vis[v]) { vis[v] = 1; v = adj[v]; }
        if(ci[v] == -1){
            int u = adj[v]; tn = 0;
            do{dfs(u,v); id[u]=tn++; cs[cn].push_back(u); v=u; u=adj[u];}
                while(ci[u]==-1); cl[cn++] = tn; }
    }
```

## 7.6 Coloring

### EdgeColoring.h

**Description:** Given a simple, undirected graph with max degree  $D$ , computes a  $(D + 1)$ -coloring of the edges such that no neighboring edges share a color. ( $D$ -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)

**Time:**  $\mathcal{O}(NM)$  e210e2, 28 lines

```
vi edgeColoring(int N, vector<pii> eds) {
    vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
    for (pii e : eds) ++cc[e.first], ++cc[e.second];
    int u, v, ncols = *max_element(all(cc)) + 1;
```

```
vector<vi> adj(N, vi(ncols, -1));
for (pii e : eds) {
    tie(u, v) = e;
    fan[0] = v;
    loc.assign(ncols, 0);
    int at = u, end = u, d, c = free[u], ind = 0, i = 0;
    while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
        loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
    cc[loc[d]] = c;
    for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
        swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
    while (adj[fan[i]][d] != -1) {
        int left = fan[i], right = fan[++i], e = cc[i];
        adj[u][e] = left;
        adj[left][e] = u;
        adj[right][e] = -1;
        free[right] = e;
    } adj[u][d] = fan[i];
    adj[fan[i]][d] = u;
    for (int y : {fan[0], u, end})
        for (int z : free[y] = 0; adj[y][z] != -1; z++);
} rep(i,0,sz(eds))
for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
return ret; }
```

### ChordalGraph.h

**Description:** A graph is chordal if any cycle  $C \geq 4$  has a chord i.e. an edge  $(u, v)$  where  $u$  and  $v$  is in the cycle but  $(u, v)$  is not A perfect elimination ordering (PEO) in a graph is an ordering of the vertices of the graph such that,  $\forall v : v$  and its neighbors that occur after  $v$  in the order (later) form a clique. A graph is chordal if and only if it has a perfect elimination ordering. Optimal vertex coloring of the graph: first fit:  $col[i]$  = smallest color that is not used by any of the neighbours earlier in PEO. Max clique = Chromatic number =  $1 + \max$  over number of later neighbours for all vertices. Chromatic polynomial =  $(x - d_1)(x - d_2) \dots (x - d_n)$  where  $d_i$  = number of neighbors of  $i$  later in PEO.

**Time:**  $\mathcal{O}(n + m)$

```
34f560, 38 lines
vi perfectEliminationOrder(vector<vi>& g) { // 0-indexed, adj list
    int top = 0, n = sz(g);
    vi ord, vis(n), indeg(n);
    vector<vi> bucket(n);
    rep(i, n) bucket[0].pb(i);
    for(int i = 0; i < n; ) {
        while(bucket[top].empty()) --top;
        int u = bucket[top].back();
        bucket[top].pop_back();
        if(vis[u]) continue;
        ord.pb(u);
        vis[u] = 1;
        ++i;
        for(int v : g[u]) {
            if(vis[v]) continue;
            bucket[++indeg[v]].pb(v);
            top = max(top, indeg[v]);
        }
    }
    reverse(all(ord));
    return ord;
}

bool isChordal(vector<vi>& g, vi ord) { //ord = perfectEliminationOrder(g)
    int n = sz(g);
    set<pii> edg;
    rep(i, n) for(auto v:g[i]) edg.insert({i,v});
    vi pos(n); rep(i, n) pos[ord[i]] = i;
    rep(u, n){
        int mn = n;
        for(auto v : g[u]) if(pos[u] < pos[v]) mn = min(mn, pos[v]);
        if(mn != n) {
            int p = ord[mn];
            for(auto v : g[u]) if(pos[v] > pos[u] && v != p && !edg.count({v, p}))
                return 0;
        }
    }
    return 1;
}
```

### ChromaticNumber.h

**Description:** Calculates chromatic number of a graph represented by a vector of bitmasks. Self loops are not allowed.

**Usage:** chromaticNumber({6, 5, 3}) // 3-clique

**Time:**  $\mathcal{O}(2^n n)$

```
07ea3d, 17 lines
const int MOD = 1000500103; // big prime
int chromaticNumber(vi g) {
```

```
    int n = sz(g);
    if (!n) return 0;
    vi ind(1 << n, 1), s(1 << n);
    rep(i, 0, 1<n) s[i] = __popcount(i) & 1 ? -1 : 1;
    rep(i, 1, 1<n) {
        int ctz = __builtin_ctz(i);
        ind[i] = ind[i - (1 << ctz)] + ind[(i - (1 << ctz)) & ~g[ctz]];
        if (ind[i] >= MOD) ind[i] -= MOD;
    } rep(k, 1, n) {
        ll sum = 0;
        rep(i, 0, 1<n) {
            s[i] = int((ll)s[i] * ind[i] % MOD);
            sum += s[i];
        } if (sum % MOD) return k;
    } return n; }
```

### 7.7 Heuristics

#### MaximalCliques.h

**Description:** Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

**Time:**  $\mathcal{O}\left(3^{n/3}\right)$ , much faster for sparse graphs

```
b0d5b1, 12 lines
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
    if (!P.any()) { if (!X.any()) f(R); return; }
    auto q = (P | X)._find_first();
    auto cands = P & ~eds[q];
    rep(i,0,sz(eds)) if (cands[i]) {
        R[i] = 1;
        cliques(eds, f, P & eds[i], X & eds[i], R);
        R[i] = P[i] = 0; X[i] = 1;
    }
}
```

#### MaximumClique.h

**Description:** Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.

**Time:** Runs in about 1s for  $n=155$  and worst case random graphs ( $p=.90$ ). Runs faster for sparse graphs.

```
f7c0bc, 49 lines
typedef vector<bitset<200>> vb;
struct Maxclique {
    double limit=0.025, pk=0;
    struct Vertex { int i, d=0; };
    typedef vector<Vertex> vv;
    vb e;
    vv V;
    vector<vi> C;
    vi qmax, q, S, old;
    void init(vv& r) {
        for (auto& v : r) v.d = 0;
        for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
        sort(all(r), [](auto a, auto b) { return a.d > b.d; });
        int mxD = r[0].d;
        rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
    }
    void expand(vv& R, int lev = 1) {
        S[lev] += S[lev - 1] - old[lev];
        old[lev] = S[lev - 1];
        while (sz(R)) {
            if (sz(q) + R.back().d <= sz(qmax)) return;
            q.push_back(R.back().i);
            vv T;
            for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
            if (sz(T)) {
                if (S[lev]++ / ++pk < limit) init(T);
                int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
                C[1].clear(), C[2].clear();
                for (auto v : T) {
                    int k = 1;
                    auto f = [&](int i) { return e[v.i][i]; };
                    while (any_of(all(C[k]), f)) k++;
                    if (k > mxk) mxk = k, C[mxk + 1].clear();
                    if (k < mnk) T[j++].i = v.i;
                    C[k].push_back(v.i);
                }
                if (j > 0) T[j - 1].d = 0;
                rep(k,mnk,mxk + 1) for (int i : C[k])
                    T[j].i = i, T[j++].d = k;
                expand(T, lev + 1);
            } else if (sz(q) > sz(qmax)) qmax = q;
            q.pop_back(), R.pop_back();
        }
    }
}
```

```
    }
    vi maxClique() { init(V), expand(V); return qmax; }
    Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
        rep(i,0,sz(e)) V.push_back({i});
    }
};
```

### 7.8 Trees

#### CompressTree.h

**Description:** Given a rooted tree and a subset  $S$  of nodes, compute the minimal subtree that contains all the nodes by adding all (at most  $|S| - 1$ ) pairwise LCA's and compressing edges. Returns a list of (par, orig\_index) representing a tree rooted at 0. The root points to itself.

**Time:**  $\mathcal{O}(|S| \log |S|)$

```
"LCA.h" 9775a0, 18 lines
typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vi& subset) {
    static vi rev; rev.resize(sz(lca.time));
    vi li = subset, &T = lca.time;
    auto cmp = [&](int a, int b) { return T[a] < T[b]; };
    sort(all(li), cmp);
    int m = sz(li)-1;
    rep(i,0,m) {
        int a = li[i], b = li[i+1];
        li.push_back(lca.lca(a, b));
    } sort(all(li), cmp);
    li.erase(unique(all(li)), li.end());
    rep(i,0,sz(li)) rev[li[i]] = i;
    vpi ret = {pii(0, li[0])};
    rep(i,0,sz(li)-1) {
        int a = li[i], b = li[i+1];
        ret.emplace_back(rev[lca.lca(a, b)], b);
    } return ret; }
```

#### HLD.h

**Description:** Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most  $\log(n)$  light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS\_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.

**Time:**  $\mathcal{O}((\log N)^2)$

```
ab0cb2, 32 lines
struct HLD { // 0-indexed
    int N, tim = 0, VALS_EDGES = 0; // change to 1 if needed
    vector<vi> adj; vi par, sz, depth, rt, pos;
    HLD(vector<vi> _adj) : N(SZ(_adj)), adj(_adj), par(N, -1),
        sz(N, 1), depth(N), rt(N), pos(N) { dfsSz(0); dfsHld(0); }
    void dfsSz(int v) {
        if(par[v] != -1) adj[v].erase(find(all(adj[v]), par[v]));
        for(int &u: adj[v]) {
            par[u] = v, depth[u] = depth[v] + 1;
            dfsSz(u); sz[v] += sz[u];
            if(sz[u] > sz[adj[v][0]]) swap(u, adj[v][0]);
        }
    }
    void dfsHld(int v) {
        pos[v] = tim++;
        for(int u: adj[v]) {
            rt[u] = (u == adj[v][0] ? rt[v] : u); dfsHld(u); }
    }
    vector<pii> path(int u, int v) {
        vector<pii> paths;
        for(; rt[u] != rt[v]; v = par[rt[v]]) {
            if(depth[rt[u]] > depth[rt[v]]) swap(u, v);
            paths.pb({pos[rt[v]], pos[v]});
        }
        if(depth[u] > depth[v]) swap(u, v);
        paths.pb({pos[u] + VALS_EDGES, pos[v]});
        return paths;
    }
    pii subtree(int v) {
        return {pos[v] + VALS_EDGES, pos[v] + sz[v] - 1};
    }
};
```

### DickCumTree.h

**Description:** One-indexed. Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree. Also you can update a value in a vertex, and do standard path and subtree queries. Operation on subtrees must be invertible.

**Time:** All operations take amortized  $\mathcal{O}(\log N)$ .

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <pre>struct SplayTree {     struct Node {         int p = 0, ch[2] = {0, 0};         ll self = 0, path = 0; // Path aggregates         ll sub = 0, vir = 0; // Subtree aggregates         bool flip = 0; // Lazy tags     }; vector&lt;Node&gt; t;     SplayTree(int n) : t(n + 1) {}     void push(int v) {         if(!v    !t[v].flip) return;         auto &amp;l, r] = t[v].ch;         t[l].flip ^= 1, t[r].flip ^= 1;         swap(l, r), t[v].flip = 0;     }     void pull(int v) {         auto [l, r] = t[v].ch; push(l), push(r);         t[v].path = t[l].path + t[v].self + t[r].path;         t[v].sub = t[v].vir + t[l].sub + t[v].self + t[r].sub;     }     void set(int u, int d, int v) {         t[u].ch[d] = v, t[v].p = u, pull(u);     }     void splay(int v) {         auto dir = [&amp;](int x) {             int u = t[x].p;             return t[u].ch[0] == x ? 0 : t[u].ch[1] == x ? 1 : -1;         }; auto rotate = [&amp;](int x) {             int y = t[x].p, z = t[y].p, dx = dir(x), dy = dir(y);             set(y, dx, t[x].ch[!dx]), set(x, !dx, y);             if(~dy) set(z, dy, x);             t[x].p = z;         }; for(push(v); ~dir(v); ) {             int y = t[v].p, z = t[y].p;             push(z), push(y), push(v);             int dv = dir(v), dy = dir(y);             if(~dy) rotate(dv == dy ? y : v);             rotate(v); });     }     struct LinkCut : SplayTree { // 1-indexed         LinkCut(int n) : SplayTree(n) {}         int access(int v) {             int u = v, x = 0;             for(; u; x = u, u = t[u].p) {                 splay(u);                 int &amp;ox = t[u].ch[1];                 t[u].vir += t[ox].sub;                 t[u].vir -= t[x].sub;                 ox = x, pull(u);             }             return splay(v), x;         }         void reroot(int v) { access(v), t[v].flip ^= 1, push(v); }         void link(int u, int v) {             reroot(u), access(v);             t[v].vir += t[u].sub; t[u].p = v, pull(v);         }         void cut(int u, int v) {             reroot(u), access(v); t[v].ch[0] = t[u].p = 0, pull(v);         }         // Rooted tree LCA. Returns 0 if u and v are not connected.         int lca(int u, int v) {             if(u == v) return u;             access(u); int ret = access(v); return t[u].p ? ret : 0;         }         // Query subtree of u where v is outside the subtree.         ll getSub(int u, int v) {             reroot(v), access(u); return t[u].vir + t[u].self;         }         ll getPath(int u, int v) {             reroot(u), access(v); return t[v].path;         }         // Update vertex u with value val         void update(int u, ll val) {             access(u), t[u].self = val, pull(u); });     }</pre> | 42b461, 64 lines |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <pre>    }     Edge top() { prop(); return key; } }; Node *merge(Node *a, Node *b) {     if (!a    !b) return a ? b;     a-&gt;prop(), b-&gt;prop();     if (a-&gt;key.w &gt; b-&gt;key.w) swap(a, b);     swap(a-&gt;l, {a-&gt;r = merge(b, a-&gt;r)});     return a; } void pop(Node&amp; a) { a-&gt;prop(); a = merge(a-&gt;l, a-&gt;r); } pair&lt;ll, vi&gt; dmst(int n, int r, vector&lt;Edge&gt; &amp;g) {     RollbackUF uf(n);     vector&lt;Node&gt; heap(n);     for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node(e));     ll res = 0;     vi seen(n, -1), path(n), par(n);     seen[r] = r;     vector&lt;Edge&gt; Q(n), in(n, {-1,-1}), comp;     deque&lt;tuple&lt;int, int, vector&lt;Edge&gt;&gt;&gt; cycs;     rep(s,0,n) {         int u = s, qi = 0, w;         while (seen[u] &lt; 0) {             if (!heap[u]) return {-1,{} };             Edge e = heap[u]-&gt;top();             heap[u]-&gt;delta -= e.w, pop(heap[u]);             Q[qi] = e, path[qi++] = u, seen[u] = s;             res += e.w, u = uf.find(e.a);             if (seen[u] == s) {                 Node* cyc = 0;                 int end = qi, time = uf.time();                 do cyc = merge(cyc, heap[w = path[--qi]]);                 while (uf.join(u, w));                 u = uf.find(u), heap[u] = cyc, seen[u] = -1;                 cycs.push_front({u, time, {Q[qi], &amp;Q[end]}});             }             rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];         }         for (auto [u,t,comp] : cycs) { // restore sol (optional)             uf.rollback(t);             Edge inEdge = in[u];             for (auto e : comp) in[uf.find(e.b)] = e;             in[uf.find(inEdge.b)] = inEdge;         }         rep(i,0,n) par[i] = in[i].a; // restore sol (end)         return {res, par}; }     }</pre> | 17609f, 19 lines |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|

### GreenHackenbush.h

**Description:** Given a rooted graph computes the number of Green Hackenbush game. Rule one: for a tree, number of a root is a xorsum of (son’s number + 1). Rule two: when u and v lies on a cycle, then they can be contracted.

**Time:**  $\mathcal{O}(N)$

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <pre>struct GreenHack {     vector&lt;vi&gt; G; vi pre, low; int T = 0;     GreenHack(vector&lt;vi&gt; _G) : G(_G), pre(SZ(G)), low(SZ(G)) {}     int dfs(int v, int p) {         pre[v] = low[v] = ++T; int ans = 0;         for(int u: G[v]) {             if(u == p) { p += SZ(G) + 1; continue; }             if(!pre[u]) {                 int son = dfs(u, v); low[v] = min(low[v], low[u]);                 if(pre[v] &lt; low[u]) ans ^= (son + 1);                 else ans ^= son ^ 1;             }             else if(pre[v] &lt;= pre[u]) ans ^= 1;             else low[v] = min(low[v], pre[u]);         }         return ans;     }     int run() { return dfs(0, -1); } };</pre> | 72d802, 48 lines |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|

### RedBlueHackenbush.h

**Description:** Given a rooted tree computes the result of Red-Blue Hackenbush game. If result != 0 then {result} always wins, otherwise the second player to go wins.

**Time:**  $\mathcal{O}(N \log N)$

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <pre>struct Surreal {     int value = 0, offset = 0; set&lt;int&gt; powers;     int sign() {         int tmp = 2 * value + !powers.empty();         return tmp &lt; 0 ? -1 : (tmp &gt; 0);     }     int add_power(int power) {         while(power) {             if(!powers.count(power - offset)) {                 powers.insert(power - offset); break;             }             powers.erase(power - offset); power--;         }         return !power;     }     void operator+=(const Surreal &amp;v) {</pre> | 602d80, 48 lines |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <pre>        value += v.value;         for(int power: v.powers)             value += add_power(power + v.offset);     }     void divide(int power) {         offset += power; int to_add = 0;         FOR(i, 0, power) {             if(value &amp; 1) to_add += add_power(power - i);             value &gt;&gt;= 1;         }         value += to_add;     }     void get_next(int t) {         int power = max(0, -t * value); value += t * (power + 1);         if(value == -1    (value == 1 &amp;&amp; powers.empty())) {             power++, value += t;         }         divide(power);     } }; struct RedBlueHack { /* Weights on edges should be -1 or 1 */     vector&lt;vector&lt;pii&gt;&gt; G; vector&lt;Surreal&gt; ans;     RedBlueHack(vector&lt;vector&lt;pii&gt;&gt; _G) : G(_G), ans(SZ(G)) {}     void dfs(int u, int p) {         for(auto &amp;[v, w]: G[u] if(v != p) {             dfs(v, u); ans[v].get_next(w);             if(SZ(ans[u].powers) &lt; SZ(ans[v].powers)) {                 swap(ans[u], ans[v]);             }             ans[u] += ans[v];         }     }     int run() { dfs(0, 0); return ans[0].sign(); } };</pre> | 5ba6c3, 47 lines |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|

### Centroid.h

**Description:** Computes centroid tree for a given (0-indexed) tree, memory  $\mathcal{O}(n \log n)$  • child[v] = children of v in centroid tree • par[v] = parent of v in centroid tree (-1 for root) • depth[v] = depth of v in centroid tree (0 for root) = sz(ind[v])-1 • size[v] = size of centroid subtree of v • ind[v][i] = index of vertex v in i-th centroid subtree from root, preorder • subtree[v] = list of vertices in centroid subtree of v • dists[v] = distances from v to vertices in its centroid subtree (in the order of subtree[v]) • neigh[v] = neighbours of v in its centroid subtree • dir[v][i] = index of centroid neighbour that is first vertex on path from centroid v to i-th vertex of centroid subtree (-1 for centroid)

**Time:**  $\mathcal{O}(n \log n)$

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <pre>struct CentroidTree {     vector&lt;vi&gt; child, ind, dists, subtree, neigh, dir;     vi par, depth, size;     int root; // Root centroid     CentroidTree() {}     CentroidTree(vector&lt;vi&gt; &amp;G)         : child{(sz(G)), ind{(sz(G)), dists{(sz(G)), subtree{(sz(G)), neigh{(sz(G))             , dir{(sz(G)), par{(sz(G), -2), depth{(sz(G)), size{(sz(G))         { root = decomp(G, 0, 0); }     void dfs(vector&lt;vi&gt; &amp;G, int v, int p) {         size[v] = 1;         for(auto e: G[v]) if (e != p &amp;&amp; par[e] == -2)             dfs(G, e, v), size[v] += size[e];     }     void layer(vector&lt;vi&gt; &amp;G, int v, int p, int c, int d) {         ind[v].pb(sz(subtree[c]));         subtree[c].pb(v); dists[c].pb(d);         dir[c].pb(sz(neigh[c])-1); // possibly add extra functionalities here         for(auto e: G[v]) if (e != p &amp;&amp; par[e] == -2) {             if (v == c) neigh[c].pb(e);             layer(G, e, v, c, d+1);         }     }     int decomp(vector&lt;vi&gt; &amp;G, int v, int d) {         dfs(G,v, -1);         int p = -1, s = size[v];         loop:         for(auto e: G[v]) {             if (e != p &amp;&amp; par[e] == -2 &amp;&amp; size[e] &gt; s/2) {                 p = v; v = e; goto loop;             }         }         par[v] = -1; size[v] = s; depth[v] = d;         layer(G, v, -1, v, 0);         for(auto e: G[v]) if (par[e] == -2) {             int j = decomp(G, e, d+1);             child[v].pb(j);             par[j] = v;         }         return v;     } };</pre> | 5ba6c3, 47 lines |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|

7.9 Math

7.9.1 Number of Spanning Trees

Create an  $N \times N$  matrix `mat`, and for each edge  $a \rightarrow b \in G$ , do `mat[a][b]-`, `mat[b][b]++` (and `mat[b][a]-`, `mat[a][a]++` if  $G$  is undirected). Remove the  $i$ th row and column and take the determinant; this yields the number of directed spanning trees rooted at  $i$  (if  $G$  is undirected, remove any row/column).

7.9.2 Erdős–Gallai theorem

A simple graph with node degrees  $d_1 \geq \dots \geq d_n$  exists iff  $d_1 + \dots + d_n$  is even and for every  $k = 1 \dots n$ ,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Geometry (8)

8.1 Primitives

Point.h

| Description: Point primitives                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | aafe69, 35 lines |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <pre>using D=ld; using P=pair&lt;D,D&gt;; #define x st #define y nd const D eps = 1e-6L; int sgn(D v){ return (v &gt; +eps) - (v &lt; -eps); } P operator+(P a, P b){ return {a.x+b.x,a.y+b.y}; } P operator-(P a, P b){ return {a.x-b.x,a.y-b.y}; } P operator*(P a, D v){ return {a.x*v,a.y*v}; } P operator/(P a, D v){ return {a.x/v,a.y/v}; } D dot(P a, P b) { return a.x*b.x+a.y*b.y; } D det(P a, P b) { return a.x*b.y-a.y*b.x; } D det(P a, P b, P c) { return det(b-a, c-a); } D len2(P v) { return dot(v, v); } D dist2(P a, P b) { return len2(a-b); } D len(P v) { return sqrt(len2(v)); } D dist(P a, P b) { return len(a-b); } P unit(P v) { return v / len(v); } P rot90(P v) { return {-v.y, v.x}; } P rotate(P v, D alpha) {     D c = cos(alpha), s = sin(alpha);     return {v.x*c-v.y*s, v.x*s+v.y*c}; } D angle(P v) { return atan2(v.y, v.x); } int side(P a, P b, P c) { return sgn(det(a,b,c)); } bool cmp(P a, P b) {     return sgn(a.x-b.x)==0?sgn(a.y-b.y)&lt;0:sgn(a.x-b.x)&lt;0; } int half(P v) { return sgn(v.y)&gt;0   (sgn(v.y)==0&amp;&amp;sgn(v.x)&gt;0); } bool angle_cmp(P a, P b) {     return half(a)!=half(b)?half(a)&gt;half(b):sgn(det(a,b))&gt;0; }</pre> |                  |

Line.h

| Description: Line primitives                                                                                                                                                                                                                                                                                     | 97ff16, 6 lines |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| <pre>"Point.h" P intersect(P a, P b, P c, P d) {     return a+(b-a)*(det(c-a,d-c)/det(b-a,d-c)); } D lineDist(P a, P b, P p) { return abs(det(a,b,p))/len(b-a); } P project(P a, P b, P p) {     return a+(b-a)*(dot(p-a,b-a)/dot(b-a,b-a)); } P reflect(P a, P b, P p) { return project(a,b,p) * 2 - p; }</pre> |                 |

Segment.h

| Description: Segment primitives                                                                                                                                                                                                                                                                                                                     | 239c32, 22 lines |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <pre>"Point.h", "Line.h" bool onSegment(P a, P b, P p) {     return side(a, b, p)==0 &amp;&amp; sgn(dot(a-p, b-p))&lt;=0; } vector&lt;P&gt; intersectSegment(P a, P b, P c, P d) {     auto da = det(c,d,a), db = det(c,d,b),         dc = det(a,b,c), dd = det(a,b,d);     if (sgn(da) * sgn(db) &lt; 0 &amp;&amp; sgn(dc) * sgn(dd) &lt; 0)</pre> |                  |

|                                                                                                                                                                                                                                                                                                                                                                                                                                    |  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <pre>        return intersect(a,b,c,d));     set&lt;P&gt; s;     if (onSegment(a,b,c)) s.insert(c);     if (onSegment(a,b,d)) s.insert(d);     if (onSegment(c,d,a)) s.insert(a);     if (onSegment(c,d,b)) s.insert(b);     return {all(s)}; } D segDist(P a, P b, P p) {     if (dot(b-a, p-a) &lt; 0)         return dist(a, p);     if (dot(a-b, p-b) &lt; 0)         return dist(b, p);     return lineDist(a, b, p); }</pre> |  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|

Circle.h

| Description: Circle primitives                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | cd87d2, 119 lines |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| <pre>"Point.h", "Line.h" const D pi = acos(-1); vector&lt;P&gt; intersect(P a, D r, P b, D R) {     if (sgn(dist2(a,b))==0) return {};     P v = b-a;     D d2 = len2(v), sum = r+R, dif = r-R,         p = (d2+r*r-R*R)/(d2*2), h2 = r*r-p*p*d2;     if (sum*sum &lt; d2    dif*dif &gt; d2) return {};     P m = a+v*p, per = rot90(v) * sqrt(fmax(0, h2)/d2);     return {m+per, m-per}; } D intersectArea(P a, D r, P b, D R) {     if (r &lt; R) swap(a, b), swap(r, R);     D d = len(a-b);     if (sgn(d-r-R)&gt;=0) return 0;     if (sgn(d-r+R)&lt;=0) return pi*r*R;     D u = 2*acos((d+d+r*r-R*R)/(2*d*r));     D v = 2*acos((d+d+R*R-r*r)/(2*d*R));     return 0.5*(r*r*(u-sin(u))+R*R*(v-sin(v))); } D intersectArea(P c, D r, vector&lt;P&gt; poly) {     auto arg = [&amp;](P p, P q) { return atan2(det(p,q),dot(p,q)); };     auto tri = [&amp;](P p, P q) {         P w = q-p;         auto r2 = r*r/2;         auto a = dot(w,p)/dot(w,w), b=(dot(p,p)-r*r)/dot(w,w);         auto d=a-a-b;         if (d&lt;=0) return arg(p,q)*r2;         auto s=fmax(0,-a-sqrt(d)), t=fmin(1,-a+sqrt(d));         if (t&lt;0  1&lt;=s) return arg(p,q)*r2;         P u=p+w*s, v=p+w*t;         return arg(p,u)*r2+det(u,v)/2+arg(v,q)*r2;     };     D res=0;     rep(i,0,sz(poly))         res += tri(poly[i]-c,poly[(i+1)%sz(poly)]-c);     return res; }</pre> |                   |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <pre>/// external R &gt; 0, internal R &lt; 0, point R = 0 vector&lt;pair&lt;P,P&gt;&gt; tangents(P a, D r, P b, D R) {     P d = b-a;     D dr = r-R, d2 = len2(d), h2 = d2-dr*dr;     if (sgn(d2) == 0    h2 &lt; 0) return {};     vector&lt;pair&lt;P,P&gt;&gt; res;     for (D sign : {-1, +1}) {         P v = (d*dr+rot90(d)*sqrt(h2)*sign)/d2;         res.pb((a+v*r, b+v*R));     }     if (h2==0) res.pop_back();     return res; } vector&lt;P&gt; circleLine(P c, D r, P a, P b) {     P ab = b-a, p = a+ab*dot(c-a,ab)/len2(ab);     D s = det(a,b,c), h2 = r*r-s*s/len2(ab);     if (h2 &lt; 0) return {};     if (h2 == 0) return {p};     P h = unit(ab) * sqrt(h2);     return {p+h, p-h}; } pair&lt;P,D&gt; circumCircle(P a, P b, P c) {     P v = c-a, w = b-a;     P o = a + rot90(v*len2(w)-w*len2(v))/det(v,w)/2;     return {o,dist(a,o)}; } pair&lt;P, D&gt; enclosingCircle(vector&lt;P&gt; p) {     shuffle(all(p), mt19937(2137));     P o = pts[0];     D r = 0, EPS = 1 + 1e-8;     rep(i,0,sz(p)) if (dist(o, p[i]) &gt; r * EPS) {         o = p[i], r = 0;         rep(j,0,i) if (dist(o, p[j]) &gt; r * EPS) {             o = (p[i] + p[j]) / 2; </pre> |  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <pre>        r = dist(o, p[i]);         rep(k,0,j) if (dist(o, p[k]) &gt; r * EPS) {             tie(o, r) = circumCircle(p[i], p[j], p[k]);         }     }     return {o, r}; } D circlesUnionArea(vector&lt;pair&lt;P, D&gt;&gt; c) {     sort(all(c)); c.erase(unique(all(c)), c.end());     D res = 0;     for(auto &amp;[p, r]: c) {         int cnt = 0, cover = 0;         vector&lt;pair&lt;D, int&gt;&gt; eve = {{-pi, 0}};         for(auto &amp;[q, s]: c) if(mp(p, r) != mp(q, s)) {             D dst = len(p-q);             if (r + dst &lt;= s) { cover = 1; break; }             vector&lt;P&gt; inters=intersect(p,r,q,s);             if (inters.empty()) continue;             D le = angle(inters[0] - p);             D re = angle(inters[1] - p);             cnt += le &gt; re;             eve.pb({le, 1}), eve.pb({re, -1});         }         if(cover) continue;         sort(eve.begin() + 1, eve.end());         eve.pb({pi, 0});         D loc = 0;         rep(i, 1, sz(eve)) {             if(!cnt) {                 D a = eve[i-1].st, b = eve[i].st;                 loc += r * (b - a) +                     det(p, P(cos(b)-cos(a), sin(b)-sin(a)));             }             cnt += eve[i].nd;         }         res += r * loc;     }     return res / 2; }</pre> |  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|

8.2 Polygons

Convex.h

| Description: Convex polygon primitives                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 0d0b0b, 70 lines |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <pre>"Segment.h" vector&lt;P&gt; convexHull(vector&lt;P&gt; pts) {     sort(all(pts),cmp);     if (sz(pts) &lt;= 2) return pts;     vector&lt;P&gt; h;     rep(i,0,2) {         int s = sz(h)+2;         for (P p:pts){             while ((sz(h)&gt;=s&amp;&amp;side(end(h)[-2],end(h)[-1],p)&lt;=0)                 h.pop_back();             h.pb(p);}         h.pop_back(), reverse(all(pts));     }     return h;} bool insideConvex(vector&lt;P&gt; const&amp; q, P p) {     if (sz(q) &lt;= 2)         return onSegment(q[0],q.back(),p);     if (side(q[0],q[1],p) &lt; 0            side(q.back(),q[0],p) &lt; 0)         return false;     int l=1,r=sz(q)-1;     while (r - l &gt; 1) {         int m=(l+r)&gt;&gt;1;         (side(q[0],q[m],p)&gt;=0?l:r)=m;     }     return side(q[1],q[r],p)&gt;=0;} #define pdir(i) (ph ? p - poly[(i)%n] : poly[(i)%n] - p) #define cmp(i,j) sgn(pdir(i).cross(poly[(i)%n]-poly[(j)%n])) #define extr(i) cmp(i + 1, i) &gt;= 0 &amp;&amp; cmp(i, i - 1 + n) &lt; 0 template &lt;class P&gt; array&lt;int, 2&gt; polygonTangents(vector&lt;P&gt;&amp; poly, P p) {     auto bs = [&amp;](int ph) {         int n = sz(poly), lo = 0, hi = n;         if(extr(0)) return 0;         while(lo + 1 &lt; hi) {             int m = (lo + hi) / 2;             if(extr(m)) return m;             int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);             (ls &lt; ms    (ls == ms &amp;&amp; ls == cmp(lo, m)) ? hi:lo) = m;         }         return lo;     };     array&lt;int, 2&gt; res = {bs(0), bs(1)};     if(res[0] == res[1]) res[0] = (res[0] + 1) % SZ(poly);     if(poly[res[0]] == p) res[0] = (res[0] + 1) % SZ(poly);     return res; }</pre> |                  |

```
}
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = (n >= 2);
    pair<D, array<P, 2>> res({0, {S[0], S[0]}});
    rep(i,0,j)
        for (; j = (j + 1) % n) {
            res = max(res, {len2(S[i] - S[j]), {S[i], S[j]}});
            if (det(S[(j + 1) % n] - S[j], S[i + 1] - S[i]) >= 0)
                break;
        }
    return res.second;
}
vector<P> minkowski(vector<P> a, vector<P> b) {
    rotate(a.begin(), min_element(all(a)), a.end());
    rotate(b.begin(), min_element(all(b)), b.end());
    rep(i,0,2) a.pb(a[i]), b.pb(b[i]);
    vector<P> res;
    for(int i = 0, j = 0; i < sz(a) - 2 || j < sz(b) - 2; ) {
        res.pb(a[i] + b[j]);
        auto cross = sgn(det(a[i + 1] - a[i], b[j + 1] - b[j]));
        if(cross >= 0 && i < sz(a)) i++;
        if(cross <= 0 && j < sz(b)) j++;
    }
    return res;
}
```

ConvexHullOnline.h

**Description:** Allows online point insertion. If exists, left vertical segment is included; right one is excluded. To get a lower hull add (-x, -y) instead of (x, y).

**Time:** amortized  $\mathcal{O}(\log n)$  per add

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| "Point.h"                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 10c55b, 16 lines |
| <pre>using P = Point&lt;ll&gt;; struct UpperHull : set&lt;P&gt; {     bool rm(auto it) {         if (it==begin()    it==end()    next(it)==end()                it-&gt;cross(*prev(it), *next(it)) &gt; 0)             return false;         erase(it); return true;     }     bool add(P p) { // true iff added         auto [it, ok] = emplace(p);         if (!ok    rm(it)) return false;         while (rm(next(it)));         while (it != begin() &amp;&amp; rm(prev(it)));         return true;     } };</pre> |                  |

ConvexHullOnline2.h

**Description:** Fully dynamic upper / lower convex hull, can be used for computing onion layers. All points should be known in advance. Points on the edges are included in the hull. Return indices are the same as in the input.

**Time:**  $\mathcal{O}(\log^2 n)$ , as fast as other  $\mathcal{O}(\log n)$  hulls

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| "Point.h"                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 21e548, 70 lines |
| <pre>template&lt;class T&gt; struct DynHull {     using P = Point&lt;T&gt;;     struct Node { int l, r; }; vector&lt;P&gt; ps;     int n; vi in, id; int s; vector&lt;Node&gt; t; vector&lt;T&gt; m;     DynHull(vector&lt;P&gt; _ps, bool lower = 0, int start = -1)         : ps(_ps), n(sz(ps)), in(n), id(n) {         if(start == -1) start = n;         s = 1; while(s &lt; n) s *= 2;         t.resize(s * 2, {-1, -1}); m.resize(s);         vector&lt;pair&lt;P, int&gt;&gt; pts;         rep(i, n) pts.pb({ps[i] * (lower ? -1 : 1), i});         sort(all(pts));         rep(i, n) {             tie(ps[i], id[i]) = pts[i]; in[id[i]] = i;             int p = i + s; while((p &amp; 1) ^ 1) p &gt;&gt;= 1;             m[p &gt;&gt; 1] = ps[i].x;         }         rep(i, start) t[s + in[i]] = {in[i], in[i]};         for (int i = s - 1; i &gt;= 1; i --) pull(i);     }     int go(int v) {         while(t[v].l &lt; 0) v = v * 2 + t[v].l + 3;         return v;     }     void pull(int v) {         auto crossNegX = [](P a, P b, P c, P d, T x) {             // change __int128 if using doubles!             __int128 p = a.cross(b, c), q = b.cross(a, d);             return p + q == 0    (d.x - x) * p + (c.x - x) * q &lt;= 0;         };</pre> |                  |

```
int p = v * 2, q = p + 1;
if(t[p].l == -1 && t[q].l == -1) t[v] = {-1, -1};
else if(t[p].l == -1) t[v] = {-2, -2};
else if(t[q].l == -1) t[v] = {-3, -3};
else {
    p = go(p), q = go(q);
    while(p < s || q < s) {
        auto [a, b] = t[p]; auto [c, d] = t[q];
        if(a != b && ps[a].cross(ps[b], ps[c]) > 0) {
            p = go(p * 2);
        }
        else if(c != d && ps[b].cross(ps[c], ps[d]) > 0) {
            q = go(q * 2 + 1);
        }
        else if(a == b) q = go(q * 2);
        else if(c == d) {
            crossNegX(ps[a], ps[b], ps[c], ps[d], m[v]) {
                p = go(p * 2 + 1);
                else q = go(q * 2);
            }
        }
        t[v] = {p - s, q - s};
    }
}
void add(int i) {
    i = in[i]; int v = i + s; t[v] = {i, i};
    while(v >= 1) pull(v);
}
void del(int i) {
    i = in[i]; int v = i + s; t[v] = {-1, -1};
    while(v >= 1) {
        if(t[v].l < 0 || t[v].l == i || t[v].r == i) pull(v);
    }
}
void dfs(int v, int l, int r, vi &h) {
    if(v >= s) return h.pb(id[t[v].l]);
    if(l <= t[v].l) dfs(go(v * 2), l, min(t[v].l, r), h);
    if(t[v].r <= r) dfs(go(v * 2 + 1), max(t[v].r, l), r, h);
}
vi hull() {
    vi h; if(~t[1].l) dfs(go(1), 0, n - 1, h); return h;
};
```

Polygon.h

**Description:** Polygon primitives

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| "Point.h", "Line.h", "Segment.h"                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 5858d1, 85 lines |
| <pre>bool inPolygon(vector&lt;P&gt; const&amp; p, P a, bool strict=true) {     int cnt = 0, n = sz(p);     rep(i,0,n) {         P q = p[(i+1)%n];         if (onSegment(p[i], q, a)) return !strict;         cnt ^= ((a.y &lt; p[i].y) - (a.y &lt; q.y)) * det(a,p[i],q) &gt; 0;     }     return cnt; } P polygonCenter(vector&lt;P&gt; const&amp; p) {     P res{0, 0}; D area = 0;     for (int i = 0, j = sz(p) - 1; i &lt; sz(p); j = i++) {         res = res+(p[i]+p[j])*det(p[i],p[j]);         area += det(v[i], v[j]);     }     return res/area/3; } vector&lt;P&gt; polygonCut(vector&lt;P&gt; const&amp; p, P a, P b) {     vector&lt;P&gt; res;     for (int i = 0, j = sz(p) - 1; i &lt; sz(p); j = i++) {         bool side = det(a, b, p[i]) &lt; 0;         if (side != (det(a, b, p[j]) &lt; 0))             res.pb(intersect(a, b, p[i], p[j]));         if (side)             res.pb(p[i]);     }     return res; } D rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y; } D polyUnion(vector&lt;vector&lt;P&gt;&gt; const&amp; p) {     D ret = 0;     rep(i,0,sz(poly)) rep(v,0,sz(p[i])) {         P A = p[i][v], B = p[i][(v + 1) % sz(p[i])];         vector&lt;pair&lt;D, int&gt;&gt; segs = {{0, 0}, {1, 0}};         rep(j,0,sz(p)) if (i != j) {             rep(u,0,sz(p[j])) {                 P C = p[j][u], D = p[j][(u + 1) % sz(p[j])];                 int sc = sideOf(A, B, C), sd = sideOf(A, B, D);                 if (sc != sd) {                     D sa = det(C, D, A), sb = det(C, D, B);                     if (min(sc, sd) &lt; 0)                         segs.eb(sa / (sa - sb), sgn(sc - sd));                 }                 else if (!sc &amp;&amp; !sd &amp;&amp; j &lt; i &amp;&amp; sgn(dot(B-A, D-C))&gt;0) {                     segs.eb(rat(C - A, B - A), 1);                     segs.eb(rat(D - A, B - A), -1);                 }             }         }     } };</pre> |                  |

```
}
sort(all(segs));
for (auto& s : segs) s.st = fmin(fmax(s.st, 0.0), 1.0);
D sum = 0;
int cnt = segs[0].nd;
rep(j,1,sz(segs)) {
    if (!cnt) sum += segs[j].st - segs[j - 1].st;
    cnt += segs[j].nd;
}
ret += det(A, B) * sum;
return ret / 2;
}
D triangulate(vector<P> const& p) {
    int n = p.size();
    vi next(n);
    rep(i,0,n-1) next[i] = i+1;
    auto is_ear = [&](int i, int j, int k) {
        if (side(p[i], p[j], p[k]) <= 0) return false;
        for (int l = next[k]; l != i; l = next[l])
            if (side(p[l], p[i], p[j]) >= 0 &&
                side(p[l], p[j], p[k]) >= 0 &&
                side(p[l], p[k], p[i]) >= 0) return false;
        return true;
    };
    D area = 0;
    for (int i = 0; next[next[i]] != i; ) {
        if (is_ear(i, next[i], next[next[i]]) ) {
            D += abs(det(p[i], p[next[i]], p[next[next[i]]])) / 2;
            next[i] = next[next[i]];
        } else i = next[i];
    }
    return area;
}
```

DelaunayTriangulation.h

**Description:** Computes the Delaunay triangulation of a set of points. Each circumcircle contains none of the input points. If any three points are collinear or any four are on the same circle, behavior is undefined.

**Time:**  $\mathcal{O}(n^2)$

|                                                                                                                                                                                                                                                                                                                                                                                                                       |                  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| "Point.h", "Hull3D.h"                                                                                                                                                                                                                                                                                                                                                                                                 | a4ba4e, 10 lines |
| <pre>template&lt;class P, class F&gt; void delaunay(vector&lt;P&gt;&amp; ps, F trfun) {     if (sz(ps) == 3) { int d = (det(ps[0], ps[1], ps[2]) &lt; 0);         trfun(0,1+d,2-d); }     vector&lt;P&gt; p3;     for (P p : ps) p3.emplace_back(p.x, p.y, dot(p,p));     if (sz(ps) &gt; 3) for(auto t:hull3d(p3)) if (det(p3[t.b]-p3[t.a],         p3[t.c]-p3[t.a]).z &lt; 0)         trfun(t.a, t.c, t.b); }</pre> |                  |

FastDelaunay.h

**Description:** Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.

**Time:**  $\mathcal{O}(n \log n)$

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| "Point.h"                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | cefd5f, 88 lines |
| <pre>typedef Point&lt;ll&gt; P; typedef struct Quad* Q; typedef __int128_t ll1; // (can be ll if coords are &lt; 2e4) P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point struct Quad {     Q rot, o; P p = arb; bool mark;     P&amp; F() { return r()-&gt;p; }     Q&amp; r() { return rot-&gt;rot; }     Q prev() { return rot-&gt;o-&gt;rot; }     Q next() { return r()-&gt;prev(); } } *H; bool circ(P p, P a, P b, P c) { // is p in the circumcircle?     ll1 p2 = p.dist2(), A = a.dist2()-p2,         B = b.dist2()-p2, C = c.dist2()-p2;     return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B &gt; 0; } Q makeEdge(P orig, P dest) {     Q r = H ? H : new Quad(new Quad(new Quad{0}));     H = r-&gt;o; r-&gt;r()-&gt;r() = r;     rep(i,0,4) r = r-&gt;rot, r-&gt;p = arb, r-&gt;o = i &amp; 1 ? r : r-&gt;r(i);     r-&gt;p = orig; r-&gt;F() = dest;     return r; } void splice(Q a, Q b) {     swap(a-&gt;o-&gt;rot-&gt;o, b-&gt;o-&gt;rot-&gt;o); swap(a-&gt;o, b-&gt;o);</pre> |                  |

```
}
Q connect(Q a, Q b) {
  Q q = makeEdge(a->F(), b->p);
  splice(q, a->next());
  splice(q->r(), b);
  return q;
}
pair<Q,Q> rec(const vector<P>& s) {
  if (sz(s) <= 3) {
    Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
    if (sz(s) == 2) return { a, a->r() };
    splice(a->r(), b);
    auto side = s[0].cross(s[1], s[2]);
    Q c = side ? connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
  }
#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
  Q A, B, ra, rb;
  int half = sz(s) / 2;
  tie(ra, A) = rec((all(s) - half));
  tie(B, rb) = rec((sz(s) - half + all(s)));
  while ((B->p.cross(H(A)) < 0 && (A = A->next()) ||
    (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
  Q base = connect(B->r(), A);
  if (A->p == ra->p) ra = base->r();
  if (B->p == rb->p) rb = base;
#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
  while (circ(e->dir->F(), H(base), e->F())) { \
    Q t = e->dir; \
    splice(e, e->prev()); \
    splice(e->r(), e->r()->prev()); \
    e->o = H; H = e; e = t; \
  }
  for (;;) {
    DEL(LC, base->r(), o); DEL(RC, base, prev());
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
      base = connect(RC, base->r());
    else
      base = connect(base->r(), LC->r());
  }
  return { ra, rb };
}
vector<P> triangulate(vector<P> pts) {
  sort(all(pts)); assert(unique(all(pts)) == pts.end());
  if (sz(pts) < 2) return {};
  Q e = rec(pts).first;
  vector<Q> q = {e};
  int qi = 0;
  while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
  q.push_back(c->r()); c = c->next(); } while (c != e); }
  ADD; pts.clear();
  while (qi < sz(q) if (!(e = q[qi++])->mark) ADD;
  return pts;
}
```

## LineHullIntersection.h

**Description:** Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon:  $\bullet$   $(-1, -1)$  if no collision,  $\bullet$   $(i, -1)$  if touching the corner  $i$ ,  $\bullet$   $(i, i)$  if along side  $(i, i + 1)$ ,  $\bullet$   $(i, j)$  if crossing sides  $(i, i + 1)$  and  $(j, j + 1)$ . In the last case, if a corner  $i$  is crossed, this is treated as happening on side  $(i, i + 1)$ . The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.

**Time:**  $\mathcal{O}(\log n)$

```
"Point.h"
7cf45b, 39 lines

#define cmp(i, j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template<class P> int extrVertex(vector<P>& poly, P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
  }
  return lo;
}
#define cmpL(i) sgn(a.cross(poly[i], b))
template<class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp());
```

```
if (cmpL(endA) < 0 || cmpL(endB) > 0)
  return {-1, -1};
array<int, 2> res;
rep(i, 0, 2) {
  int lo = endB, hi = endA, n = sz(poly);
  while ((lo + 1) % n != hi) {
    int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
    (cmpL(m) == cmpL(endB) ? lo : hi) = m;
  }
  res[i] = (lo + !cmpL(hi)) % n;
  swap(endA, endB);
}
if (res[0] == res[1]) return {res[0], -1};
if (!cmpL(res[0]) && !cmpL(res[1]))
  switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
    case 0: return {res[0], res[0]};
    case 2: return {res[1], res[1]};
  }
return res;
}
```

## 8.3 Misc.

### ClosestPair.h

**Description:** Finds the closest pair of points.

**Time:**  $\mathcal{O}(n \log n)$

```
"Point.h"
39cf125, 16 lines

pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret(LLONG_MAX, {P(), P()});
  int j = 0;
  for (P p : v) {
    P d(1 + (ll)sqrt(ret.first), 0);
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {len2(*lo - p), (*lo, p)});
    S.insert(p);
  }
  return ret.second;
}
```

### PointLocation.h

**Description:** Computes (not necessarily convex) polygon tree structure. Also for each query point computes its location (including boundaries).

**Time:**  $\mathcal{O}(n \log n)$

```
"directedSegment.h"
39f30f, 47 lines

template<class P>
pair<vi, vi> pointLoc(vector<vector<P>> polys, vector<P> pts) {
  vector<tuple<P, int, int>> eve; // {point, event type, id}
  vector<pair<dirSeg<P>, int>> segs; // {s, e, poly id}
  rep(i, sz(polys)) rep(j, sz(polys[i])) {
    dirSeg<P> seg(polys[i][j], polys[i][j+1]*sz(polys[i]));
    eve.pb({seg.s, 0, sz(segs)}), eve.pb({seg.e, 2, sz(segs)});
    segs.pb({seg, i});
  }
  rep(i, sz(pts)) eve.pb({pts[i], 1, i});
  sort(all(eve));
  vi par(sz(polys), -2), ans(sz(pts), -1);
  auto cmp = [](auto a, auto b) {
    return make_pair(a.st.cmp(b.st), a.nd) < make_pair(0, b.nd);
  };
  set<pair<dirSeg<P>, int>, decltype(cmp)> s(cmp);
  for(auto &[_ , eve_tp, id]: eve) {
    if(eve_tp == 1) { // point query
      P p = pts[id];
      auto it = s.lower_bound({dirSeg(p, p), 0});
      if(it != s.begin()) { // on vertical segment?
        auto prv = prev(it);
        if(!sgn(p.cross(prv->st.s, prv->st.e)) it--;
      }
      if(it == s.end()) ans[id] = -1;
      else {
        auto [seg, seg_id] = *it;
        int poly_id = segs[seg_id].nd; // strictness there!
        ans[id] = !seg.rev && sgn(p.cross(seg.s, seg.e))
          ? par[poly_id] : poly_id;
      }
    }
    if(eve_tp == 0) { // add segment
      auto it = next(s.insert({segs[id].st, id}).st);
      int poly_id = segs[id].nd;
      if(par[poly_id] == -2) {
        if(it == s.end()) par[poly_id] = -1;
        else {
          int up_rev = it->st.rev, up_id = segs[it->nd].nd;
```

```
          par[poly_id] = !up_rev ? par[up_id] : up_id;
        }
      }
      if(eve_tp == 2) s.erase({segs[id].st, id}); // del segment
    }
    return {par, ans};
  }
}
```

## SegmentInterDetection.h

**Description:** Finds one of segments intersections. You should change dirSeg's comparator, to compare segments at their left end.

**Time:**  $\mathcal{O}(N \log N)$

```
"SegmentIntersection.h"
a87bce, 49 lines

template<class P>
struct dirSeg {
  P s, e; int rev;
  dirSeg(P _s, P _e) : s(_s), e(_e), rev(0) {
    if(e < s) swap(s, e), rev = 1;
  }
  P getY(P X) { // takes x * 2, returns y * 2 as a fraction
    P d = (e - s);
    return !sgn(d.x) ? P(s.y+e.y, 1) : P(d.cross(s*2-X), d.x);
  }
  int cmp(dirSeg b) { // needs ~64 * M^3 !
    P X(max(s.x, b.s.x) + min(e.x, b.e.x), 0);
    return sgn(getY(X).cross(b.getY(X)));
  }
};

template<class P>
pii segmentsIntersect(vector<pair<P, P>> segments) {
  vector<tuple<P, int, int>> eve; // {point, event_type, id}
  vector<dirSeg<P>> segs;
  for(auto &[s, e]: segments) {
    dirSeg<P> seg(s, e);
    eve.pb({seg.s, 0, sz(segs)}), eve.pb({seg.e, 1, sz(segs)});
    segs.pb(seg);
  }
  sort(all(eve));
  auto inter = [](auto a, auto b) {
    return sz(segInter(a->st.s, a->st.e, b->st.s, b->st.e));
  };
  auto cmp = [](auto a, auto b) {
    return mp(a.st.cmp(b.st), a.nd) < mp(0, b.nd);
  };
  set<pair<dirSeg<P>, int>, decltype(cmp)> s(cmp);
  for(auto &[_ , eve_tp, id]: eve) {
    if(eve_tp == 0) { // add segment
      auto it = s.insert({segs[id], id}).st;
      if(next(it) != s.end()) && inter(it, next(it))
        return {it->nd, next(it)->nd};
      if(it != s.begin()) && inter(it, prev(it))
        return {it->nd, prev(it)->nd};
    }
    if(eve_tp == 1) { // del segment
      auto it = s.erase(s.find({segs[id], id}));
      if(it!=s.begin() && it!=s.end()) && inter(it, prev(it))
        return {it->nd, prev(it)->nd};
    }
  }
  return {-1, -1};
}
```

## Halfplane.h

**Description:** Halfplane intersection

**Time:**  $\mathcal{O}(n \log n)$

```
"Point.h", "Line.h"
27e354, 24 lines

using L=pair<P,P>;
#define dir(l) l.nd-l.st
vector<P> halfplaneIntersection(vector<L> ls) {
  sort(all(ls), [&](L a, L b){return angle_cmp(dir(a),dir(b));});
  auto bad = [&](L a, L b, L c) {
    return sgn(det(a.st, a.nd, intersect(b.st,b.nd,c.st,c.nd))) < 0;
  };
  deque<L> q; for (L h : ls) {
    while (sz(q)>=2 && bad(h,end(q)[-1],end(q)[-2])) q.pop_back();
    while (sz(q)>=2 && bad(h,q[0],q[1])) q.pop_front();
    if (sz(q) && sgn(det(dir(h),dir(q.back())))==0){
      if (sgn(dot(dir(h),dir(q.back())))<0)
        return {};
      if (side(h.st,h.nd,q.back().st)<0)
        q.back() = h;
    } else q.pb(h);
  }
  while (sz(q)>3 && bad(q[0],end(q)[-1],end(q)[-2])) q.pop_back();
  while (sz(q)>3 && bad(end(q)[-1],q[0],q[1])) q.pop_front();
  if(sz(q)<3)return{};
  vector<P> h;
```

```

    rep(i,0,sz(q)){
        int j=(i+1)%sz(q);
        h.pb(intersect(q[i].st,q[i].nd,q[j].st,q[j].nd)); }
    return h; }


```

## HalfplaneOnline.h

**Description:** Data structure that dynamically keeps track of the intersection of half-planes.

**Time:** amortized  $\mathcal{O}(\log n)$

```

"new_geometry/Point.h"
e489e5, 50 lines

int hf(P a) { return a.y < 0 || (a.y == 0 && a.x < 0); }
struct polarCmp {
    bool operator()(const P &a, const P &b) const {
        return hf(a) == hf(b) ? det(a, b) > 0 : hf(a) < hf(b);
    }
};
struct HalfplaneSet : map<P, P, polarCmp> {
    D INF = 1e6, area = 8 * INF * INF;
    HalfplaneSet() {
        P p(-INF, -INF), d(1, 0);
        rep(k, 0, 4) {
            insert({d, p}); p = p + d * 2 * INF; d = rot90(d); }
        auto fix(auto it) { return it == end() ? begin() : it; }
        auto getNext(auto it) { return fix(next(it)); }
        auto getPrev(auto it) {
            return it == begin() ? prev(end()) : prev(it); }
        auto uSide(auto it, int change) {
            area += change * det(it->nd, getNext(it)->nd); }
        auto del(auto it) {
            uSide(getPrev(it), -1), uSide(it, -1);
            it = fix(erase(it));
            if(size()) uSide(getPrev(it), 1);
            return it;
        }
    }
    void add(P s, P e) {
        auto eval = [&](auto it) { return sgn(det(s, e, it->nd)); };
        auto ii = [&](auto it) {
            return intersect(s, e, it->nd, it->st + it->nd); };
        auto it = fix(lower_bound(e - s));
        if(empty() || eval(it) >= 0) return;
        while(size() && eval(getPrev(it)) < 0) del(getPrev(it));
        while(size() && eval(getNext(it)) < 0) it = del(it);
        if(empty()) return;
        if(eval(getNext(it)) > 0) {
            uSide(getPrev(it), -1), uSide(it, -1);
            it->nd = ii(it);
            uSide(getPrev(it), 1), uSide(it, 1);
        }
        else it = del(it);
        it = getPrev(it);
        uSide(it, -1); insert(it, {e - s, ii(it)});
        uSide(it, 1), uSide(getNext(it), 1);
        if(eval(it) == 0) del(it);
    }
    D maxDot(P a) {
        return dot(a, fix(lower_bound(rot90(a)))->nd); }
    D getArea() { return area / 2; }
};


```

## kdTree.h

**Description:** KD-tree (2d, can be extended to 3d)

```

"Point.h"
af33f9, 80 lines

const int MXN = 100005;
struct KDTree {
    struct Nd {
        int x,y,x1,y1,x2,y2;
        int id,f;
        Nd *L,*R;
    }tree[MXN];
    int n;
    Nd *root;
    LL dis2(int x1, int y1, int x2, int y2) {
        LL dx = x1-x2; LL dy = y1-y2;
        return dx*dx+dy*dy;
    }
    static bool cmpx(Nd& a, Nd& b){ return a.x<b.x; }
    static bool cmpy(Nd& a, Nd& b){ return a.y<b.y; }
    void init(vector<pair<int,int>> ip) {
        n = ip.size();
        for (int i=0; i<n; i++) {
            tree[i].id = i;
            tree[i].x = ip[i].first;
            tree[i].y = ip[i].second;
        }
        root = build_tree(0, n-1, 0);
    }
    Nd* build_tree(int L, int R, int dep) {


```

```

        if (L>R) return nullptr;
        int M = (L+R)/2;
        tree[M].f = dep%2;
        nth_element(tree+L, tree+M, tree+R+1,
            tree[M].f ? cmpy : cmpx);
        tree[M].x1 = tree[M].x2 = tree[M].x;
        tree[M].y1 = tree[M].y2 = tree[M].y;
        tree[M].L = build_tree(L, M-1, dep+1);
        if (tree[M].L) {
            tree[M].x1 = min(tree[M].x1, tree[M].L->x1);
            tree[M].x2 = max(tree[M].x2, tree[M].L->x2);
            tree[M].y1 = min(tree[M].y1, tree[M].L->y1);
            tree[M].y2 = max(tree[M].y2, tree[M].L->y2);
        }
        tree[M].R = build_tree(M+1, R, dep+1);
        if (tree[M].R) {
            tree[M].x1 = min(tree[M].x1, tree[M].R->x1);
            tree[M].x2 = max(tree[M].x2, tree[M].R->x2);
            tree[M].y1 = min(tree[M].y1, tree[M].R->y1);
            tree[M].y2 = max(tree[M].y2, tree[M].R->y2);
        }
        return tree+M;
    }
    int touch(Nd* r, int x, int y, LL d2){
        LL dis = sqrt(d2)+1;
        if (x<r->x1-dis || x>r->x2+dis ||
            y<r->y1-dis || y>r->y2+dis)
            return 0;
        return 1;
    }
    void nearest(Nd* r, int x, int y, int &mID, LL &md2){
        if (!r || !touch(r, x, y, md2)) return;
        LL d2 = dis2(r->x, r->y, x, y);
        if (d2 < md2 || (d2 == md2 && mID < r->id)) {
            mID = r->id; md2 = d2;
        }
        // search order depends on split dim
        if ((r->f == 0 && x < r->x) ||
            (r->f == 1 && y < r->y)) {
            nearest(r->L, x, y, mID, md2);
            nearest(r->R, x, y, mID, md2);
        } else {
            nearest(r->R, x, y, mID, md2);
            nearest(r->L, x, y, mID, md2);
        }
    }
    int query(int x, int y) {
        int id = 1029384756;
        LL d2 = 102938475612345678LL;
        nearest(root, x, y, id, d2);
        return id;
    }
}tree;


```

## ManhattanMST.h

**Description:** Given N points, returns up to 4\*N edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights  $w(p, q) = |p.x - q.x| + |p.y - q.y|$ . Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST.

**Time:**  $\mathcal{O}(N \log N)$

```

"Point.h"
df6f59, 23 lines

typedef Point<int> P;
vector<array<int, 3>> manhattanMST(vector<P> ps) {
    vi id(sz(ps));
    iota(all(id), 0);
    vector<array<int, 3>> edges;
    rep(k,0,4) {
        sort(all(id), [&](int i, int j) {
            return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;});
        map<int, int> sweep;
        for (int i: id) {
            for (auto it = sweep.lower_bound(~ps[i].y);
                it != sweep.end(); sweep.erase(it++)) {
                int j = it->second;
                P d = ps[i] - ps[j];
                if (d.y > d.x) break;
                edges.push_back({d.y + d.x, i, j});
            }
            sweep[-ps[i].y] = i;
        }
        for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p.y);
    }
    return edges;
}


```

## RectUnion.h

**Description:** Rectangle union

2401ce, 44 lines

```

#define fst first
#define snd second
#define all(c) ((c).begin()), ((c).end())
struct rectangle { int x1, y1, xh, yh; };
long long rectangle_area(vector<rectangle> rs) {
    vector<int> ys; // coordinate compression
    for (int i = 0; i < rs.size(); ++i) {
        ys.push_back(rs[i].y1);
        ys.push_back(rs[i].yh);
    }
    sort(all(ys)); ys.erase(unique(all(ys)), ys.end());
    int n = ys.size(); // measure tree
    vector<int> C(8*n), A(8*n);
    function<void (int,int,int,int,int,int)> aux =
    [&](int a, int b, int c, int l, int r, int k) {
        if ((a = max(a,1)) >= (b = min(b,r))) return;
        if (a == 1 && b == r) C[k] += c;
        else {
            aux(a, b, c, l, (l+r)/2, 2*k+1);
            aux(a, b, c, (l+r)/2, r, 2*k+2);
        }
        if (C[k]) A[k] = ys[r] - ys[l];
        else A[k] = A[2*k+1] + A[2*k+2];
    };
    struct event { int x, l, h, c; }; // plane sweep
    vector<event> es;
    for (auto r: rs) {
        int l = distance(ys.begin(), lower_bound(all(ys), r.y1));
        int h = distance(ys.begin(), lower_bound(all(ys), r.yh));
        es.push_back({r.x1, l, h, +1});
        es.push_back({r.xh, l, h, -1});
    }
    sort(all(es), [](event a, event b) { return a.x != b.x ? a.x < b.x : a.c
        > b.c; });
    long long area = 0, prev = 0;
    for (auto &e: es) {
        area += (e.x - prev) * A[0];
        prev = e.x;
        aux(e.l,e.h,e.c,0,n,0);
    }
    return area;
}


```

## 8.4 3D

## Point3D.h

**Description:** 3D Point primitives

de4463, 26 lines

```

using D=ld;
struct P {
    array<D, 3> v;
    auto operator<=> (P const&p) const = default;
};
#define x v[0]
#define y v[1]
#define z v[2]
const D eps = 1e-9L;
int sgn(D v) { return (v > +eps) - (v < -eps); }
P operator+ (P a, P b) { rep(i,0,3) a.v[i] += b.v[i]; return a; }
P operator- (P a, P b) { rep(i,0,3) a.v[i] -= b.v[i]; return a; }
P operator* (P a, D v) { rep(i,0,3) a.v[i] *= v; return a; }
P operator/ (P a, D v) { rep(i,0,3) a.v[i] /= v; return a; }
D dot(P a, P b) { return a.x*b.x + a.y*b.y + a.z*b.z; }
P det(P a, P b) { return a.y*b.z-a.z*b.y, a.z*b.x-a.x*b.z, a.x*b.y-a.y*b
    .x; };
D len(P v) { return sqrt(dot(a,a)); }
D phi(P v) { return atan2(v.y, v.x); }
D theta(P v) { return atan2(hypot(x,y),z); }
P unit(P v) { return v / len(v); }
P rotate(P v, P axis, P angle) {
    D s = sin(angle), c = cos(angle); P u = unit(axis);
    return u*dot(v,u)*(1-c)+v*c-det(v,u)*s; }


```

## Hull3D.h

**Description:** Computes all faces of the 3-dimension hull of a point set. \*No four points must be coplanar\*, or else random results will be returned. All faces will point outwards.

**Time:**  $\mathcal{O}(n^2)$

```

"Point3D.h"
2c3706, 54 lines

struct PR {
    void ins(int x) { (a == -1 ? a : b) = x; }
    void rem(int x) { (a == x ? a : b) = -1; }
    int cnt() { return (a != -1) + (b != -1); }
    int a, b;
};
struct F { P3 q; int a, b, c; };


```

```
vector<F> hull3d(const vector<P3>& A) {
    assert(sz(A) >= 4);
    vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
    #define E(x,y) E[f.x][f.y]
    vector<F> FS;
    auto mf = [=](int i, int j, int k, int l) {
        P3 q = det(A[j] - A[i], A[k] - A[i]);
        if (dot(q,A[l]) > dot(q,A[i]))
            q = q * -1;
        F f(q, i, j, k);
        E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
        FS.push_back(f);
    };
    rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
        mf(i, j, k, 6 - i - j - k);
    rep(i,4,sz(A)) {
        rep(j,0,sz(FS)) {
            F f = FS[j];
            if(dot(f.q,A[i]) > dot(f.q,A[f.a])) {
                E(a,b).rem(f.c);
                E(a,c).rem(f.b);
                E(b,c).rem(f.a);
                swap(FS[j--], FS.back());
                FS.pop_back();
            }
        }
        int nw = sz(FS);
        rep(j,0,nw){
            F f = FS[j];
            #define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
            C(a, b, c); C(a, c, b); C(b, c, a);
        }
        for (F& it : FS) if (dot(det(A[it.b] - A[it.a]
            A[it.c] - A[it.a]),it.q) <= 0) swap(it.c, it.b);
        return FS;
    };
    template<class V, class L>
    D signedPolyVolume(const V& p, const L& trilst) {
        double v = 0;
        for (auto i : trilst) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
        return v / 6;
    }
}
```

## Sphere.h

**Description:** Sphere primitives

```
"Point3D.h" d4bf2a, 23 lines

D sphericalDistance(D f1, D t1, D f2, D t2, D radius) {
    D dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
    D dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
    D dz = cos(t2) - cos(t1);
    D d = sqrt(dx*dx + dy*dy + dz*dz);
    return radius*2*asin(d/2);
}

vector<P> trilaterate(P a, P b, P c, D p, D q, D r) {
    b = b - a, c = c - a;
    P e_x = unit(b); D i = dot(e_x, c);
    P e_y = unit(c - e_x * i), e_z = det(e_x, e_y);
    D d = len(b), j = dot(e_y,c);
    D x = (p * p - q * q + d * d) / 2 / d;
    D y = (p * p - r * r - 2 * i * x + i * i + j * j) / 2 / j;
    D z2 = p * p - x * x - y * y;
    const D EPS = 1e-8; // take care!
    if(z2 < -EPS) return {};
    D z = sqrt(fmax(z2, 0));
    P sol = a + e_x * x + e_y * y;
    if(z2 < EPS) return {sol};
    return {sol - e_z * z, sol + e_z * z};
}
```

## Strings (9)

### KMP.h

**Description:** pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.

**Time:**  $\mathcal{O}(n)$  d4375c, 16 lines

```
vi pi(const string& s) {
    vi p(sz(s));
    rep(i,1,sz(s)) {
        int g = p[i-1];
        while (g && s[i] != s[g]) g = p[g-1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}
```

```
vi match(const string& s, const string& pat) {
    vi p = pi(pat + '\0' + s), res;
    rep(i,sz(p)-sz(s),sz(p))
        if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
    return res;
}
```

### Zfunc.h

**Description:** z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)

**Time:**  $\mathcal{O}(n)$  ee09e2, 12 lines

```
vi Z(const string& S) {
    vi z(sz(S));
    int l = -1, r = -1;
    rep(i,1,sz(S)) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
            z[i]++;
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
    return z;
}
```

### Manacher.h

**Description:** For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).

**Time:**  $\mathcal{O}(N)$  e7ad79, 13 lines

```
array<vi, 2> manacher(const string& s) {
    int n = sz(s);
    array<vi,2> p = {vi(n+1), vi(n)};
    rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
        int t = r-i+!z;
        if (i<r) p[z][i] = min(t, p[z][l+t]);
        int L = i-p[z][i], R = i+p[z][i]-!z;
        while (L>=1 && R+1<n && s[L-1] == s[R+1])
            p[z][i]++, L--, R++;
        if (R>r) l=L, r=R;
    }
    return p;
}
```

### MainLorentz.h

**Description:** Main-Lorentz algorithm for finding all squares in given word; Results are in compressed form: (b, e, l) means that for each b <= i < e there is square at position i of size 2l. Each square is present in only one interval.

**Time:**  $\mathcal{O}(nlgn)$  3e9f59, 46 lines

```
struct Sqr {
    int begin, end, len;
};

vector<Sqr> lorentz(const string &s) {
    vector<Sqr> ans;
    vi pos(sz(s) / 2 + 2, -1);
    rep(mid, 1, sz(s)) {
        int part = mid & ~(mid - 1), off = mid - part;
        int end = min(mid + part, sz(s));
        auto a = s.substr(off, part);
        auto b = s.substr(mid, end - mid);
        string ra(a.rbegin(), a.rend());
        string rb(b.rbegin(), b.rend());
        rep(j, 0, 2) {
            // Set # to some unused character!
            vi z1 = Z(ra, true);
            vi z2 = Z(b + "#" + a, true);
            z1.pb(0);
            z2.pb(0);
            rep(c, 0, sz(a)) {
                int l = sz(a) - c;
                int x = c - min(l - 1, z1[l]);
                int y = c - max(l - z2[sz(b) + c + 1], j);
                if (x > y)
                    continue;
                int sb = (j ? end - y - l * 2 : off + x);
                int se = (j ? end - x - l * 2 + 1 : off + y + 1);
                int &p = pos[l];
                if (p != -1 && ans[p].end == sb)
                    ans[p].end = se;
                else
                    p = sz(ans), ans.pb({sb, se, l});
            }
            a.swap(rb);
            b.swap(ra);
        }
    }
```

```
    }
    }
    return ans;
}
```

### Lyndon.h

**Description:** Compute Lyndon factorization for s; Word is simple iff it's stricly smaller than any of it's nontrivial suffixes. Lyndon factorization is division of string into non-increasing simple words. It is unique.

**Time:**  $\mathcal{O}(n)$  688c1c, 12 lines

```
vector<string> duval(const string &s) {
    int n = sz(s), i = 0;
    vector<string> ret;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j])
            k = (s[k] < s[j] ? i : k + 1), j++;
        while (i <= k)
            ret.pb(s.substr(i, j - k)), i += j - k;
    }
    return ret;
}
```

### MinRotation.h

**Description:** Finds the lexicographically smallest rotation of a string. **Usage:** rotate(v.begin(), v.begin()+minRotation(v), v.end()); **Time:**  $\mathcal{O}(N)$  d07a42, 8 lines

```
int minRotation(string s) {
    int a=0, N=sz(s); s += s;
    rep(b,0,N) rep(k,0,N) {
        if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
        if (s[a+k] > s[b+k]) {a = b; break;}
    }
    return a;
}
```

### SuffixArray.h

**Description:** Builds suffix array for a string. sa[i] is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size n + 1, and sa[0] = n. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. The input string must not contain any zero bytes.

**Time:**  $\mathcal{O}(n \log n)$  769289, 23 lines

```
struct SuffixArray {
    vi sa, lcp;
    SuffixArray(string& s, int lim=256) { // or basic_string<int>
        int n = sz(s) + 1, k = 0, a, b;
        vi x(all(s)), y(n), ws(max(n, lim)), rank(n);
        x.push_back(0), sa = lcp = y, iota(all(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
            p = j, iota(all(y), n - j);
            rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i,0,n) ws[x[i]]++;
            rep(i,1,lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
        }
        rep(i,1,n) rank[sa[i]] = i;
        for (int i = 0, j; i < n - 1; lcp[rank[i++]] = k)
            for (k && k--, j = sa[rank[i] - 1];
                s[i + k] == s[j + k]; k++);
    }
};
```

### SuffixTree.h

**Description:** Ukkonen's algorithm for online suffix tree construction. Each node contains indices [l, r) into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining [l, r) substrings. The root is 0 (has l = -1, r = 0), non-existent children are -1. To get a complete tree, append a dummy symbol - otherwise it may contain an incomplete path (still useful for substring matching, though).

**Time:**  $\mathcal{O}(26N)$  aae0b8, 50 lines

```
struct SuffixTree {
    enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
    int toi(char c) { return c - 'a'; }
```



```
string a; // v = cur node, q = cur position
int t[N][ALPHA], l[N], r[N], p[N], s[N], v=0, q=0, m=2;
void ukkadd(int i, int c) { suff:
    if (r[v]<=q) {
        if (t[v][c]==-1) { t[v][c]=m; l[m]=i;
            p[m++]=v; v=s[v]; q=r[v]; goto suff; }
        v=t[v][c]; q=l[v];
    }
    if (q==-1 || c==toi(a[q])) q++; else {
        l[m+1]=i; p[m+1]=m; l[m]=l[v]; r[m]=q;
        p[m]=p[v]; t[m][c]=m+1; t[m][toi(a[q])]=v;
        l[v]=q; p[v]=m; t[p[m]][toi(a[l[m]])]=m;
        v=s[p[m]]; q=l[m];
        while (q<r[m]) { v=t[v][toi(a[q])]; q+=r[v]-l[v]; }
        if (q==r[m]) s[m]=v; else s[m]=m+2;
        q=r[v]-(q-r[m]); m+=2; goto suff;
    }
}
SuffixTree(string a) : a(a) {
    fill(r,r+N,sz(a));
    memset(s, 0, sizeof s);
    memset(t, -1, sizeof t);
    fill(t[1],t[1]+ALPHA,0);
    s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] = 0;
    rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
}
// example: find longest common substrng (uses ALPHA = 28)
pii best;
int lcs(int node, int i1, int i2, int olen) {
    if (l[node] <= i1 && i1 < r[node]) return 1;
    if (l[node] <= i2 && i2 < r[node]) return 2;
    int mask = 0, len = node ? olen + (r[node] - l[node]) : 0;
    rep(c,0,ALPHA) if (t[node][c] != -1)
        mask |= lcs(t[node][c], i1, i2, len);
    if (mask == 3)
        best = max(best, {len, r[node] - len});
    return mask;
}
static pii LCS(string s, string t) {
    SuffixTree st(s + (char)('z'+1) + t + (char)('z'+2));
    st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
    return st.best;
}
};
```

**Runs.h**  
**Description:** Find all  $(i,p)$  such that  $s.substr(i,p) == s.substr(i+p,p)$ . No two intervals with the same period intersect or touch.  
**Usage:** solve("aaabababa") // {{0, 1, 1}, {2, 5, 2}}  
**Time:**  $\mathcal{O}(N \log N)$  cf17d5, 14 lines

```
vector<array<int, 3>> solve(string s) {
    int N = sz(s); SuffixArray A, B;
    A.init(s); reverse(all(s)); B.init(s);
    vector<array<int, 3>> runs;
    for(int p = 1; 2*p <= N; ++p) { // do in O(N/p) for period p
        for(int i = 0, lst = -1; i+p <= N; i += p) {
            int l = i-B.getLCP(N-i-p,N-i), r = i-p+A.getLCP(i,i+p);
            // getLCP(i,j) is lcp of suffixes starting at i and j
            if (l > r || l == lst) continue;
            runs.pb({lst = l,r,p}); // for each i in [l,r],
        } // s.substr(i,p) == s.substr(i+p,p)
    }
    return runs;
}
```

**AhoCorasick.h**  
**Description:** Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(–, word) finds all words (up to  $N\sqrt{N}$  many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries.  
**Time:** construction takes  $\mathcal{O}(26N)$ , where  $N$  = sum of length of patterns. find(x) is  $\mathcal{O}(N)$ , where  $N$  = length of x. findAll is  $\mathcal{O}(NM)$ . f35677, 66 lines

```
struct AhoCorasick {
    enum {alpha = 26, first = 'A'}; // change this!
    struct Node {
        // (nmatches is optional)
        int back, next[alpha], start = -1, end = -1, nmatches = 0;
        Node(int v) { memset(next, v, sizeof(next)); }
```

```
};
vector<Node> N;
vi backp;
void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
        int& m = N[n].next[c - first];
        if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
        else n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
}
AhoCorasick(vector<string>& pat) : N(1, -1) {
    rep(i,0,sz(pat)) insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);
    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
        int n = q.front(), prev = N[n].back;
        rep(i,0,alpha) {
            int &ed = N[n].next[i], y = N[prev].next[i];
            if (ed == -1) ed = y;
            else {
                N[ed].back = y;
                (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
                    = N[y].end;
                N[ed].nmatches += N[y].nmatches;
                q.push(ed);
            }
        }
    }
}
vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
        n = N[n].next[c - first];
        res.push_back(N[n].end);
        // count += N[n].nmatches;
    }
    return res;
}
vector<vi> findAll(vector<string>& pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i,0,sz(word)) {
        int ind = r[i];
        while (ind != -1) {
            res[i - sz(pat[ind]) + 1].push_back(ind);
            ind = backp[ind];
        }
    }
    return res;
}
};
```

**ALCS.h**  
**Description:** All-substrings common sequences algorithm. Given strings A and B, algorithm computes:  $C(i,j,k) = |LCS(A[:i], B[j:k])|$  in compressed form; To describe the compression, note that: 1.  $C(i,j,k-1) \leq C(i,j,k) \leq C(i,j,k-1)+2$ . If  $j < k$  and  $C(i,j,k) = C(i,j,k-1)+1$ , then  $C(i,j+1,k) = C(i,j+1,k-1)+1$  3. If  $j \geq k$ , then  $C(i,j,k) = 0$  This allows us to store just the following:  $ih(i,k) = \min j$  s.t.  $C(i,j,k-1) < C(i,j,k)$   
**Time:**  $\mathcal{O}(nm)$  87acac, 58 lines

```
struct ALCS {
    string A, B;
    vector<vi> ih;
    // Precompute compressed matrix; time: O(nm)
    ALCS(string s, string t) : A(s), B(t) {
        int n = sz(A), m = sz(B);
        ih.resize(n + 1, vi(m + 1));
        iota(all(ih[0]), 0);
        rep(l, 1, n + 1) {
            int iv = 0;
            rep(j, 1, m + 1) {
                if (A[l - 1] != B[j - 1]) {
                    ih[l][j] = max(ih[l - 1][j], iv);
                    iv = min(ih[l - 1][j], iv);
                } else {
                    ih[l][j] = iv;
                    iv = ih[l - 1][j];
                }
            }
        }
    }
};
```

```
}
// Compute |LCS(A[:i], B[j:k])|; time: O(k-j)
// Note: You can precompute data structure
// to answer these queries in O(log n)
// or compute all answers for fixed 'i'.
int operator()(int i, int j, int k) {
    int ret = 0;
    rep(q, j, k) ret += (ih[i][q + 1] <= j);
    return ret;
}
// Compute subsequence LCS(A[:i], B[j:k]);
// time: O(k-j)
string recover(int i, int j, int k) {
    string ret;
    while (i > 0 && j < k) {
        if (ih[i][k-1] <= j) {
            ret.pb(B[k]);
            while (A[--i] != B[k])
                ;
        }
        reverse(all(ret));
        return ret;
    }
}
// Compute LCS'es of given prefix of A,
// and all prefixes of given suffix of B.
// Returns vector L of length |B|+1 s.t.
// L[k] = |LCS(A[:i], B[j:k])|; time: O(|B|)
vi row(int i, int j) {
    vi ret(sz(B) + 1);
    rep(k, j + 1, sz(ret)) ret[k] = ret[k - 1] + (ih[i][k] <= j);
    return ret;
}
};
```

**PalindromicTree.h**  
**Description:** Computes plaindromic tree: for each end position in the string we store longest palindrome ending in that position. link is the suffix palindrome links, eg ababa -> aba. Can be used to compute shortest decomposition of strings to palindromes in  $\mathcal{O}(n \log n)$  time - use [DP] lines.

```
Time:  $\mathcal{O}(N)$  eb3607, 38 lines
constexpr int ALPHA = 26;
struct PalTree {
    vi txt; //;Node 0=empty pal (root of even), 1=-1" pal (of odd)
    vi len{0, -1}; // Lengths of palindromes
    vi link{1, 0}; // Suffix palindrome links, eg [ababa] -> [aba]
    vector<array<int, ALPHA>> to{{}, {}}; // egdes, ex: aba -c> cabac
    int last{0}; // Current node (max suffix pal)
    vi diff{0, 0}; // [DP] len[i]-len[link[i]]
    vi slink{0, 0}; // [DP] like link but to having different 'diff'
    vi series{0, 0}; // [DP] dp for series (groups of pals with =diff)
    vi ans{0}; // [DP] ans for prefix
    int ext(int i) {
        while (len[i]+2>sz(txt) || txt[sz(txt)-len[i]-2]!=txt.back())
            i = link[i];
        return i;
    }
    void add(int x) { //x in [0,ALPHA), time O(1) or O(lg n) for DP
        txt.pb(x); last = ext(last);
        if (!to[last][x]) {
            len.pb(len[last] + 2);
            link.pb(to[ext(link[last])][x]);
            to[last][x] = sz(to);
            to.pb({});
            diff.pb(len.back() - len[link.back()]); // [DP]
            slink.pb(diff.back() == diff[link.back()] ? slink[link.back()] :
                link.back()); // [DP]
            series.pb(0); // [DP]
        }
        last = to[last][x];
        ans.pb(INT_MAX); // [DP]
        for (int i = last; len[i] > 0; i = slink[i]) { // [DP]
            series[i] = ans[sz(ans) - len[slink[i]] - diff[i] - 1]; // [DP]
            if (diff[i] == diff[link[i]]) // [DP]
                series[i] = min(series[i], series[link[i]]); // [DP]
            //For even only palindromes set ans only for even sz(txt) // [DP]
            ans.back() = min(ans.back(), series[i] + 1); // [DP]
        }
    }
};
```

Various (10)

10.1 Misc. algorithms

ConstantIntervals.h

**Description:** Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.

**Usage:** constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});

**Time:**  $\mathcal{O}(k \log \frac{n}{k})$  753a4c, 19 lines

```
template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
    if (p == q) return;
    if (from == to) {
        g(i, to, p);
        i = to; p = q;
    } else {
        int mid = (from + to) >> 1;
        rec(from, mid, f, g, i, p, f(mid));
        rec(mid+1, to, f, g, i, p, q);
    }
}

template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
    if (to <= from) return;
    int i = from; auto p = f(i), q = f(to-1);
    rec(from, to-1, f, g, i, p, q);
    g(i, to, q);
}
```

FastMod.h

**Description:** Compute  $a\%b$  about 5 times faster than usual, where  $b$  is constant but not known at compile time. Returns a value congruent to  $a \pmod b$  in the range  $[0, 2b)$ .

751a02, 8 lines

```
typedef unsigned long long ull;
struct FastMod {
    ull b, m;
    FastMod(ull b) : b(b), m((-1ULL / b) {}
    ull reduce(ull a) { // a % b + (0 or b)
        return a - (ull)((__uint128_t(m) * a) >> 64) * b;
    }
};
```

FastInput.h

**Description:** Read an integer from stdin. Usage requires your program to pipe in input from file.

**Usage:** ./a.out < input.txt

**Time:** About 5x as fast as cin/scanf. 7b3c70, 17 lines

```
inline char gc() { // like getchar()
    static char buf[1 << 16];
    static size_t bc, be;
    if (bc >= be) {
        buf[0] = 0, bc = 0;
        be = fread(buf, 1, sizeof(buf), stdin);
    }
    return buf[bc++]; // returns 0 on EOF
}

int readInt() {
    int a, c;
    while ((a = gc()) < 40);
    if (a == '-') return -readInt();
    while ((c = gc()) >= 48) a = a * 10 + c - 48;
    return a - 48;
}
```

10.2 Dynamic programming

KnuthDP.h

**Description:** When doing DP on intervals:  $a[i][j] = \min_{i < k < j} (a[i][k] + a[k][j]) + f(i, j)$ , where the (minimal) optimal  $k$  increases with both  $i$  and  $j$ , one can solve intervals in increasing order of length, and search  $k = p[i][j]$  for  $a[i][j]$  only between  $p[i][j - 1]$  and  $p[i + 1][j]$ . This is known as Knuth DP. Sufficient criteria for this are if  $f(b, c) \leq f(a, d)$  and  $f(a, c) + f(b, d) \leq f(a, d) + f(b, c)$  for all  $a \leq b \leq c \leq d$ . Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.

**Time:**  $\mathcal{O}(N^2)$

DivideAndConquerDP.h

**Description:** Given  $a[i] = \min_{lo(i) \leq k < hi(i)} (f(i, k))$  where the (minimal) optimal  $k$  increases with  $i$ , computes  $a[i]$  for  $i = L..R - 1$ .

**Time:**  $\mathcal{O}((N + (hi - lo)) \log N)$  d38d2b, 18 lines

```
struct DP { // Modify at will:
    int lo(int ind) { return 0; }
    int hi(int ind) { return ind; }
    ll f(int ind, int k) { return dp[ind][k]; }
    void store(int ind, int k, ll v) { res[ind] = pii(k, v); }
    void rec(int L, int R, int LO, int HI) {
        if (L >= R) return;
        int mid = (L + R) >> 1;
        pair<ll, int> best(LLONG_MAX, LO);
        rep(k, max(LO, lo(mid)), min(HI, hi(mid)))
            best = min(best, make_pair(f(mid, k), k));
        store(mid, best.second, best.first);
        rec(L, mid, LO, best.second+1);
        rec(mid+1, R, best.second, HI);
    }
    void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

AliensTrick.h

**Description:** Optimize dp where you want "k things with minimal cost". The slope of f(k) must be non increasing. Provide a function g(lambda) that computes the best answer for any k with costs increased by lambda.

13ea3, 8 lines

```
ll aliens(ll k, auto g) { // returns f(k)
    ll l = 0, r = 1e11; // make sure lambda range [l, r) is ok (r > max
        slope etc)
    while (l + 1 < r) {
        ll m = (l + r) / 2;
        (g(m - 1) + k <= g(m) ? l : r) = m;
    }
    return g(l) - 1 * k; // return l if you want the optimal lambda
}
```